



Security Assessment Report

Exponent CLMM

May 11, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Exponent CLMM smart contract.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 59 issues or questions.

Task	Type	Commit/PR
exponent_clmm	Solana	818a6b8 (Dec 15, 2025), PR3446 (Feb 11, 2026), PR3608 (Mar 20, 2026), PR3682 (Apr 14, 2026)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview	3
Findings in Detail	5
[H-01] Incorrect liquidity decrease applied to pending cross ticks	5
[H-02] Swap exact-in missing interval guards	10
[H-03] Checked math breaks v3 style fee accounting	12
[H-04] Incorrect kappa calculation in swap_exact_out	15
[H-05] Improper boundary handling in PtToSy exact-in swap	18
[H-06] Incorrect reversal in partial liquidity removal	21
[H-07] Incorrect crossing split reversal mechanism	25
[H-08] Stale lp_balance_crossing breaks removal accounting	29
[M-01] Cumulative rounding loss in sub tick token calculation	36
[M-02] Split discontinuity at lower and upper boundaries	39
[M-03] kappa not clamped in exact-in swap calculation	40
[M-04] Incorrect rounding direction in user provided amount calculation	42
[M-05] Stranded emissions from LP fee tokens	44
[M-06] Tick skip breaks liquidity invariant	47
[M-07] Incorrect max_sy_in parameter in wrapper_buy_yt	51
[M-08] Exact SY swap out check breaks buy_yt	53
[M-09] Rounding assertion between pt_out and yt_out can fail	55
[M-10] DoS add_liquidity calls by zero lp_balance_crossing	57
[M-11] Zero min_l_to_activate disables tick initialization protection	59
[M-12] Dust-sized shares can DoS subsequent add_liquidity calls	61
[M-13] Potentially inaccurate equalization estimates	63
[L-01] Unsafe u64 to i64 type casting	66
[L-02] Incorrect compute_ticks_amount_in_range implementation	70
[L-03] create_token_account DoS	72
[L-04] The floor operation on pt_in_segment leads to trade_pt failure	74

[L-05] Trade fee event misreports total fee	76
[L-06] Inconsistent epsilon clamp check in SyToPt mode	77
[L-07] Missing tick cross when price hits boundary	79
[L-08] Incorrect tick cross condition in swap_exact_out	81
[L-09] Incorrect SY slice traversal due to index-based start node selection	82
[L-10] Undistributed emissions due to lp_share split rounding	84
[L-11] Zero PT contribution on crossing leads to DoS	86
[L-12] swap_exact_out input accounting undercounts max_in	88
[L-13] Incorrect pt_asset_value formula	90
[L-14] Hardcoded parameters in mock liquidity calculation	92
[L-15] Minimum node liquidity check bypass	93
[L-16] Extra tick crossing due to inconsistent swap loop conditions	94
[L-17] Unclaimable CrossingSplit rewards due to mismatched domain	96
[L-18] Equalization calculation ignored in liquidity deposit	98
[L-19] Missing cap on secant_step in pt_heavy path	100
[I-01] Missing alignment check in emissions tracker update	102
[I-02] Inconsistent MarketThree::size_of	104
[I-03] Inaccurate tick key ordering check	106
[I-04] Redundant successor lookup	107
[I-05] Unsafe account closure procedure	108
[I-06] Rounding causes zero mint with principal loss	109
[I-07] Dead liquidity from epsilon clamp	111
[I-08] Unchecked liquidity_to_remove in remove_liquidity	114
[I-09] Inaccurate PT balance check in sell_yt	115
[I-10] Incorrect amount_pt_in event parameter	116
[I-11] Boundary condition mismatches in liquidity and fee checks	117
[I-12] Off-by-one farm index in AddFarmEvent	119
[I-13] Tick node exhaustion DoS risk	120
[I-14] Overly restrictive sy_to_borrow check in buy_yt	121
[I-15] Unused emission_index instruction parameter	123

[I-16] Issues introduced by the new add_liquidity feature	124
[I-17] Division precision loss leads to state inconsistency	126
[I-18] Exact input dust is reported as consumed	129
[I-19] Inconsistent comments in add_liquidity	131
Appendix: Methodology and Scope of Work	132

Result Overview

Issue	Impact	Status
EXPONENT_CLMM		
[H-01] Incorrect liquidity decrease applied to pending cross ticks	High	Resolved
[H-02] Swap exact-in missing interval guards	High	Resolved
[H-03] Checked math breaks v3 style fee accounting	High	Resolved
[H-04] Incorrect kappa calculation in swap_exact_out	High	Resolved
[H-05] Improper boundary handling in PtToSy exact-in swap	High	Resolved
[H-06] Incorrect reversal in partial liquidity removal	High	Resolved
[H-07] Incorrect crossing split reversal mechanism	High	Resolved
[H-08] Stale lp_balance_crossing breaks removal accounting	High	Resolved
[M-01] Cumulative rounding loss in sub tick token calculation	Medium	Resolved
[M-02] Split discontinuity at lower and upper boundaries	Medium	Resolved
[M-03] kappa not clamped in exact-in swap calculation	Medium	Resolved
[M-04] Incorrect rounding direction in user provided amount calculation	Medium	Resolved
[M-05] Stranded emissions from LP fee tokens	Medium	Resolved
[M-06] Tick skip breaks liquidity invariant	Medium	Resolved
[M-07] Incorrect max_sy_in parameter in wrapper_buy_yt	Medium	Resolved
[M-08] Exact SY swap out check breaks buy_yt	Medium	Resolved
[M-09] Rounding assertion between pt_out and yt_out can fail	Medium	Resolved
[M-10] DoS add_liquidity calls by zero lp_balance_crossing	Medium	Resolved
[M-11] Zero min_l_to_activate disables tick initialization protection	Medium	Resolved
[M-12] Dust-sized shares can DoS subsequent add_liquidity calls	Medium	Resolved
[M-13] Potentially inaccurate equalization estimates	Medium	Resolved
[L-01] Unsafe u64 to i64 type casting	Low	Resolved
[L-02] Incorrect compute_ticks_amount_in_range implementation	Low	Resolved
[L-03] create_token_account DoS	Low	Resolved
[L-04] The floor operation on pt_in_segment leads to trade_pt failure	Low	Resolved
[L-05] Trade fee event misreports total fee	Low	Resolved
[L-06] Inconsistent epsilon clamp check in SyToPt mode	Low	Resolved
[L-07] Missing tick cross when price hits boundary	Low	Resolved
[L-08] Incorrect tick cross condition in swap_exact_out	Low	Resolved
[L-09] Incorrect SY slice traversal due to index-based start node selection	Low	Resolved
[L-10] Undistributed emissions due to lp_share split rounding	Low	Resolved
[L-11] Zero PT contribution on crossing leads to DoS	Low	Resolved
[L-12] swap_exact_out input accounting undercounts max_in	Low	Resolved
[L-13] Incorrect pt_asset_value formula	Low	Resolved
[L-14] Hardcoded parameters in mock liquidity calculation	Low	Resolved
[L-15] Minimum node liquidity check bypass	Low	Resolved
[L-16] Extra tick crossing due to inconsistent swap loop conditions	Low	Resolved

[L-17] Unclaimable CrossingSplit rewards due to mismatched domain	Low	Resolved
[L-18] Equalization calculation ignored in liquidity deposit	Low	Resolved
[L-19] Missing cap on secant_step in pt_heavy path	Low	Resolved
[I-01] Missing alignment check in emissions tracker update	Info	Resolved
[I-02] Inconsistent MarketThree::size_of	Info	Resolved
[I-03] Inaccurate tick key ordering check	Info	Resolved
[I-04] Redundant successor lookup	Info	Resolved
[I-05] Unsafe account closure procedure	Info	Acknowledged
[I-06] Rounding causes zero mint with principal loss	Info	Resolved
[I-07] Dead liquidity from epsilon clamp	Info	Resolved
[I-08] Unchecked liquidity_to_remove in remove_liquidity	Info	Resolved
[I-09] Inaccurate PT balance check in sell_yt	Info	Resolved
[I-10] Incorrect amount_pt_in event parameter	Info	Resolved
[I-11] Boundary condition mismatches in liquidity and fee checks	Info	Resolved
[I-12] Off-by-one farm index in AddFarmEvent	Info	Resolved
[I-13] Tick node exhaustion DoS risk	Info	Resolved
[I-14] Overly restrictive sy_to_borrow check in buy_yt	Info	Resolved
[I-15] Unused emission_index instruction parameter	Info	Resolved
[I-16] Issues introduced by the new add_liquidity feature	Info	Resolved
[I-17] Division precision loss leads to state inconsistency	Info	Acknowledged
[I-18] Exact input dust is reported as consumed	Info	Acknowledged
[I-19] Inconsistent comments in add_liquidity	Info	Resolved

Findings in Detail

EXPONENT_CLMM

[H-01] Incorrect liquidity decrease applied to pending cross ticks

Identified in commit [818a6b8](#).

During the add liquidity process, `compute_liquidity_target_and_token_needs` calculates the liquidity amount (`liquidity_needs.liquidity_target`) that the user should receive for providing `PT/SY` within the specified tick range [`lower_tick_idx`, `upper_tick_idx`).

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
093 | // Decide region and per-budget ΔL
094 | let liquidity_needs = compute_liquidity_target_and_token_needs(
095 |     snap,
096 |     current_spot,
097 |     price_eff_lower,
098 |     price_eff_upper,
099 |     lower_price,
100 |     upper_price,
101 |     lower_tick_idx,
102 |     upper_tick_idx,
103 |     ticks.current_tick,
104 |     args.max_sy,
105 |     args.max_pt,
106 |     self.configuration_options.epsilon_clamp,
107 | );
```

This liquidity amount is then added to the boundary ticks of the user-specified range, i.e., both `lower_tick_idx` and `upper_tick_idx`, by increasing their `liquidity_gross` and updating their `liquidity_net`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
109 | // 5) Materialize integer ΔL units and update liquidity_net at boundaries
110 | let delta_l_i64 = liquidity_needs.liquidity_target as i64;
111 | ticks.tick_add_liquidity_net_gross(
112 |     lower_tick_idx,
113 |     delta_l_i64,
114 |     true,
115 |     self.configuration_options.min_lp_tick_amount,
116 | ); // +ΔL at lower
117 | ticks.tick_add_liquidity_net_gross(
118 |     upper_tick_idx,
119 |     delta_l_i64,
120 |     false,
121 |     self.configuration_options.min_lp_tick_amount,
122 | ); // -ΔL at upper
```

In the actual liquidity addition phase, `accrue_principal_for_deposit` splits the user-provided `PT/SY` across

all existing sub tick ranges within the user-specified range. The computed `liquidity_needs.liquidity_target` is passed as `position_liquidity` and is used to help determine how much `PT` or `SY` should be allocated to each sub tick range.

For sub tick ranges:

- Below `spot_price_current`, only `PT` is added.
- Above `spot_price_current`, only `SY` is added.
- If `spot_price_current` lies within a sub tick range, that range receives both `PT` and `SY`.

The sub tick range that contains `spot_price_current` is referred to as the pending-cross tick range.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
198 | pub fn accrue_principal_for_deposit(
199 |     &mut self,
200 |     snapshot: EffSnap,
201 |     price_spot_lower: f64,
202 |     price_spot_upper: f64,
203 |     price_split_for_need: f64,
204 |     price_split_tick_idx: u32,
205 |     lower_price_idx: u32,
206 |     position_liquidity: u64,
207 |     lp_position: &mut LpPosition,
208 |     market_emission_indices: [precise_number::Number; 3],
209 | ) -> Result<(u64, u64)> {
210 |     let pt_slice_start = price_spot_lower;
211 |     let pt_slice_end = price_split_for_need.min(price_spot_upper);
212 |     let mut total_sy_spend: u64 = 0;
213 |     let mut total_pt_spend: u64 = 0;
214 |     let mut pending_cross: Option<(u32, u64, f64)> = None;
215 |
216 |     if pt_slice_start < pt_slice_end {
217 |         self.for_each_interval_between(
218 |             lower_price_idx,
219 |             pt_slice_start,
220 |             pt_slice_end,
221 |             |segment: IntervalSegment| {
222 |                 let segment_start = segment.price_spot_left.max(pt_slice_start);
223 |                 let segment_end = segment.price_spot_right.min(pt_slice_end);
224 |                 let segment_length_u = segment_end - segment_start;
225 |                 if segment_length_u > 0.0 {
226 |                     let is_crossing = segment.price_spot_left < price_split_for_need
227 |                         && segment.price_spot_right > price_split_for_need;
228 |
229 |                     let principal_pt_delta =
230 |                         (position_liquidity as f64 * segment_length_u).floor() as u64;
231 |
232 |                     if principal_pt_delta > 0 {
233 |                         if is_crossing {
234 |                             // save for later resolve
235 |                             pending_cross = Some((
236 |                                 segment.left_index,
237 |                                 principal_pt_delta,
238 |                                 segment_length_u,
239 |                             ));

```

```

240 |         return Ok(());
241 |     }

```

For the pending-cross tick range, the protocol recalculates the actual liquidity added (`l_actual`) and compares it against `position_liquidity`. If `l_actual` is less than `position_liquidity`, the protocol subtracts the difference (`position_liquidity - l_actual`) from the `liquidity_gross` and `liquidity_net` of the boundary ticks of the pending-cross range.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
284 | if is_crossing {
285 |     // active tick case we deposit liquidity based on proportions of current distribution
286 |     let (pt_left_index, dpt_in, du_pt_part) =
287 |         pending_cross.take().unwrap();
288 |
289 |     let (_minted, used_pt, used_sy) =
290 |         segment.ticks.mint_shares_for_tick_prestate_calc_used(
291 |             pt_left_index,
292 |             dpt_in,
293 |             principal_sy_delta,
294 |             lp_position,
295 |             market_emission_indices,
296 |         )?;
297 |
298 |     if used_pt > 0 {
299 |         add_principal_pt(segment.ticks, pt_left_index, used_pt)?;
300 |         total_pt_spend = total_pt_spend.checked_add(used_pt).unwrap();
301 |     }
302 |     if used_sy > 0 {
303 |         add_principal_sy(
304 |             segment.ticks,
305 |             segment.left_index,
306 |             used_sy,
307 |             market_emission_indices,
308 |         )?;
309 |
310 |         total_sy_spend = total_sy_spend.checked_add(used_sy).unwrap();
311 |     }
312 |
313 |     // recalc actual L active for this tick
314 |     let l_target = position_liquidity as f64;
315 |     let l_pt_cap = (used_pt as f64) / du_pt_part;
316 |     let l_sy_cap = (used_sy as f64) / sy_per_l_1;
317 |
318 |     let l_actual = l_pt_cap.min(l_sy_cap).min(l_target);
319 |
320 |     // 4) if l_actual < l_target - we should reduce L in tick
321 |     if l_actual < l_target {
322 |         let delta_l: i64 = (l_target - l_actual).floor() as i64; // positive
323 |         let delta_l_u64 = delta_l as u64;
324 |
325 |         segment.ticks.tick_sub_liquidity_net_gross(
326 |             pt_left_index as u32,
327 |             delta_l,
328 |             true,
329 |         )?;
330 |

```

```

331 |         segment.ticks.tick_sub_liquidity_net_gross(
332 |             segment.right_index,
333 |             delta_l,
334 |             false,
335 |         )?;

```

However, this logic is incorrect when the user's tick range contains multiple sub tick ranges. The pending-cross tick range is only a sub-range of the user-specified range, and its boundary ticks are not the same as the original boundary ticks (`lower_tick_idx` and `upper_tick_idx`) where liquidity was initially added. The recalculation logic incorrectly treats the pending-cross tick range as if it represented the entire user-specified range.

Additionally, when such a reduction occurs, `lp_position.lp_balance` and `liquidity_needs.liquidity_target` are not reduced accordingly. The position's recorded liquidity and `MarketFinancials.liquidity_balance` remain `delta_l_i64`, even though the tick level liquidity and `current_prefix_sum` have been decreased by `delta_l`, resulting in an inconsistency between the position state and the tick state.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
178 | lp_position.lp_balance = delta_l_i64 as u64;

```

Example:

The user provides liquidity in the tick range `[10, 40)`. As a result, ticks 10 and 40 increase `liquidity_gross` and update `liquidity_net`.

Existing ticks are at `10, 20, 30`, and `40`, and `spot_price_current` lies within `[20, 30)`.

Liquidity is split as follows:

- `[10, 20)`: PT only
- `[20, 30)`: PT + SY (pending-cross range)
- `[30, 40)`: SY only

In the `[20, 30)` pending-cross range, ticks `20` and `30` will be decreased in `liquidity_gross` and `liquidity_net` when the `position_liquidity` is greater than `l_actual`, even though liquidity was originally applied to ticks `10` and `40`.

It is recommended to ensure that any liquidity decrease is applied to the original position boundary ticks (`lower_tick_idx` and `upper_tick_idx`) and that `lp_position.lp_balance` is reduced consistently.

Resolution

This issue has been fixed by [e041477](#).

EXPONENT_CLMM

[H-02] Swap exact-in missing interval guards

Identified in commit 818a6b8.

The `swap` (exact-in) implementation in `swap.rs` performs interval trading without the same guard logic used in `swap_exact_out`. It computes `kappa` without clamping to `1.0` and does not skip intervals when `kappa <= 0` or when denominators such as `pt_max_to_left_f` or `y_max_to_boundary_f` are non-positive.

In addition, the swap calculation does not safely handle cross-tick execution and one-sided or zero liquidity ranges. When `current_price_spot` reaches a tick boundary and the swap enters the next tick range, the expression `(c_eff_old - c_eff_at_boundary) / current_price_spot - anchor_u_left` can be zero. This makes `y_max_to_boundary_f / pt_max_to_left_f` zero, which then causes a division-by-zero error and stops the swap instead of continuing into the next interval.

If the current interval has only PT liquidity or only SY liquidity, `principal_pt_in_interval` or `principal_sy_in_interval` becomes zero, causing `kappa` to be zero and halting the swap. Similarly, when the swap enters a zero liquidity range, `active_liquidity_f64` becomes zero, `kappa == 0`, and the logic stops rather than skipping ahead to the next tick range with liquidity.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
196 | if kappa <= 0.0 {
197 |     self.cross_one_boundary_and_update_core(
198 |         ticks,
199 |         SwapDirection::SyToPt,
200 |         current_left_boundary_index,
201 |         &mut current_left_boundary_index,
202 |         &mut current_price_spot,
203 |         &mut active_liquidity_u64,
204 |         &mut active_liquidity_f64,
205 |         mutate_tick_state,
206 |         market_farm_indices,
207 |         args.modified_ticks,
208 |     );
209 |     continue;
210 | }

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
507 | let kappa = kappa_pt.min(kappa_sy);
508 |
509 | match args.direction {
510 |     SwapDirection::PtToSy => {
511 |         let l_trade_f64 = active_liquidity_f64 * kappa;
512 |         // Final du in this interval
513 |         let du_by_input = (amount_in_left as f64) / l_trade_f64;
514 |         let du_to_boundary = anchor_u_right - current_price_spot;
```

This mismatch can result in division-by-zero errors or invalid price progression during exact-in swaps.

As a consequence, trades may fail unexpectedly when the current interval contains no usable principal or when the price is positioned exactly at a tick boundary, leading to denial-of-service on certain execution paths and inconsistent swap behavior.

In particular, a user's `trade_pt` instruction can fail when `current_price_spot` is exactly at the right boundary tick. In this case, `c_eff_old == c_eff_at_boundary`, which causes `y_max_to_boundary_f` to evaluate to zero and propagates `NaN` or infinite values through the swap calculations.

To address this, the `swap` implementation should be aligned with `swap_exact_out` by clamping `kappa` and adding interval-skipping guards before computing `l_trade_f64`.

The logic should also explicitly handle cross-tick execution, one-sided liquidity, and zero-liquidity ranges by skipping forward to the next valid interval whenever denominators or `l_trade_f64` are non-finite or non-positive.

```
if y_max_to_boundary_f <= eps || pt_max_to_left_f <= eps {
    // cross boundary or return an error
    self.cross_one_boundary_and_update_core(...)?;
    continue;
}
let kappa_sy = principal_sy_in_interval as f64 / y_max_to_boundary_f;
let kappa_pt = (principal_pt_in_interval as f64) / (pt_max_to_left_f);
let kappa = kappa_pt.min(kappa_sy).min(1.0);
let l_trade_f64 = active_liquidity_f64 * kappa;
if !l_trade_f64.is_finite() || l_trade_f64 <= eps {
    // cross boundary or return an error
    self.cross_one_boundary_and_update_core(...)?;
    continue;
}
if kappa <= 0.0 { /* cross boundary and continue */ }
```

Resolution

This issue has been fixed by [867468d](#).

EXPONENT_CLMM

[H-03] Checked math breaks v3 style fee accounting

Identified in commit [818a6b8](#).

In the CLMM protocol, LP swap fees and farm emissions are accounted for using a Uniswap-v3-style piecewise index mechanism.

For any LP position, the "inside" fee (or emission) index is calculated as: `global_fee_index - lower_tick.outside_fee_index - upper_tick.outside_fee_index`.

Over time, the LP's accrued fees or emissions are determined by the difference between the latest and initial inside indices: `latest_inside_index - initial_inside_index`.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
287 | /// v3-style piecewise "inside" for reinvest index (hard-compounding only)
288 | pub fn compute_fee_inside_for_range(
289 |     &self,
290 |     lower_idx: u32,
291 |     upper_idx: u32,
292 | ) -> Result<(u128, u128)> {
293 |     // Fetch current tick and outside snapshots at range boundaries
294 |     let current = self.current_tick;
295 |     let lo_out = self.ticks_tree.get_node(lower_idx).value;
296 |     let hi_out = self.ticks_tree.get_node(upper_idx).value;
297 |
298 |     let current_tick_price = self.ticks_tree.get_node(current).key;
299 |     let lower_tick_price = self.ticks_tree.get_node(lower_idx).key;
300 |     let upper_tick_price = self.ticks_tree.get_node(upper_idx).key;
301 |
302 |     // Piecewise inside:
303 |     // if current < lower:     inside = outside(lower) - outside(upper)
304 |     // if lower <= current < upper: inside = global - outside(lower) - outside(upper)
305 |     // if current >= upper:   inside = outside(upper) - outside(lower)
306 |     let inside_sy = if current_tick_price < lower_tick_price {
307 |         lo_out
308 |         .fee_growth_outside_sy
309 |         .checked_sub(hi_out.fee_growth_outside_sy)
310 |         .unwrap()
311 |     } else if current_tick_price >= upper_tick_price {
312 |         hi_out
313 |         .fee_growth_outside_sy
314 |         .checked_sub(lo_out.fee_growth_outside_sy)
315 |         .unwrap()
316 |     } else {
317 |         // (reinvest_global - lo_out - hi_out) with order check
318 |         let tmp = self
319 |             .fee_growth_index_global_sy
320 |             .checked_sub(lo_out.fee_growth_outside_sy)
321 |             .unwrap();
322 |         tmp.checked_sub(hi_out.fee_growth_outside_sy).unwrap()
323 |     };
324 | }
```

```

325 |     let inside_pt = if current_tick_price < lower_tick_price {
326 |         lo_out
327 |         .fee_growth_outside_pt
328 |         .checked_sub(hi_out.fee_growth_outside_pt)
329 |         .unwrap()
330 |     } else if current_tick_price >= upper_tick_price {
331 |         hi_out
332 |         .fee_growth_outside_pt
333 |         .checked_sub(lo_out.fee_growth_outside_pt)
334 |         .unwrap()
335 |     } else {
336 |         self.fee_growth_index_global_pt
337 |         .checked_sub(lo_out.fee_growth_outside_pt)
338 |         .unwrap()
339 |         .checked_sub(hi_out.fee_growth_outside_pt)
340 |         .unwrap()
341 |     };
342 |
343 |     Ok((inside_sy, inside_pt))
344 | }

/* solana/programs/exponent_clmm/src/state/ticks.rs */
362 | pub fn compute_farm_inside_for_range(
363 |     &self,
364 |     lower_idx: u32,
365 |     upper_idx: u32,
366 |     market_farm_indices: [Number; PERSONAL_TICK_YIELD_TRACKER_SIZE],
367 | ) -> Result<Vec<Number>> {
368 |     let left_node = self.ticks_tree.get_node(lower_idx);
369 |     let right_node = self.ticks_tree.get_node(upper_idx);
370 |     let current_key = self.ticks_tree.get_node(self.current_tick).key;
371 |     // v3-fromula calculate liquidity
372 |     let mut inside = vec![Number::ZERO; PERSONAL_TICK_YIELD_TRACKER_SIZE];
373 |     for i in 0..PERSONAL_TICK_YIELD_TRACKER_SIZE {
374 |         let below = if current_key >= left_node.key {
375 |             left_node.value.farms.trackers[i].last_seen_index
376 |         } else {
377 |             market_farm_indices[i]
378 |             .checked_sub(&left_node.value.farms.trackers[i].last_seen_index)
379 |             .unwrap()
380 |         };
381 |         let above = if current_key < right_node.key {
382 |             right_node.value.farms.trackers[i].last_seen_index
383 |         } else {
384 |             market_farm_indices[i]
385 |             .checked_sub(&right_node.value.farms.trackers[i].last_seen_index)
386 |             .unwrap()
387 |         };
388 |         inside[i] = market_farm_indices[i]
389 |             .checked_sub(&below)
390 |             .unwrap()
391 |             .checked_sub(&above)
392 |             .unwrap();
393 |     }
394 |     Ok(inside)
395 | }

```

Example

Assume the current spot price is at tick **15**, and the current global fee index is **20**. Tick **20** is already initialized with an outside fee index of **10**.

An LP opens a position in the range **[10, 20]**. Since tick **10** is newly initialized, its outside fee index is set to the current global fee index **20**.

The position's initial inside fee index is therefore: $20 - 20 - 10 = -10$.

Later, swaps move the price so that tick **20** is crossed, and the global fee index increases to **30**. As a result, tick 20's outside fee index is updated to: $30 - 10 = 20$.

The latest inside fee index becomes $20 - 20 = 0$ (simply $30 - 20 - (30 - 20) = 0$).

So the accumulated inside index for the position is: $0 - (-10) = 10$.

This is the correct result.

Key Observation

In the v3-style piecewise accounting model, negative intermediate values - and arithmetic underflow - are not only expected, but essential for correctness. Since accumulated fees are derived from the difference between two inside fee indexes, intermediate underflows do not affect correctness. On the contrary, the correctness depends on arithmetic wrapping semantics.

However, the CLMM protocol currently uses checked arithmetic for these v3-style calculations. This causes legitimate underflows (such as the **-10** in the example above) to panic, breaking fee and emission accounting and introducing a potential DoS risk to core functionality.

All v3-style piecewise fee and emission calculations should use a wrapping math library (e.g., consistent with other [CLMM](#) implementations), instead of using checked version arithmetic operators.

Resolution

This issue has been fixed by **7d1403e**.

EXPONENT_CLMM

[H-04] Incorrect kappa calculation in swap_exact_out

Identified in commit [818a6b8](#).

In `swap_exact_out`, the swap execution within a tick range computes two kappa values:

- `kappa_pt` represents the PT-side liquidity available over the price interval from the current spot price to the tick range's left boundary.
- `kappa_sy` is intended to represent the SY-side liquidity.

The implementation then takes `min(kappa_pt, kappa_sy)` as the effective liquidity for executing the swap within that tick range.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
100 | let anchor_u_left = ticks.get_spot_price(current_left_boundary_index);
101 | let c_eff_old = snapshot.get_effective_price(current_price_spot);
102 | let c_eff_left = snapshot.get_effective_price(anchor_u_left);
103 | let delta_c_to_left_boundary = c_eff_left - c_eff_old;

176 | let y_max_to_boundary_f = active_liquidity_f64 * delta_c_to_left_boundary;

193 | let kappa_pt = (principal_pt_in_interval as f64) / pt_max_to_left_f;
194 | let kappa_sy = (principal_sy_in_interval as f64) / y_max_to_boundary_f;
195 | let kappa = kappa_pt.min(kappa_sy).min(1.0);
196 | if kappa <= 0.0 {
197 |     self.cross_one_boundary_and_update_core(
198 |         ticks,
199 |         SwapDirection::SyToPt,
200 |         current_left_boundary_index,
201 |         &mut current_left_boundary_index,
202 |         &mut current_price_spot,
203 |         &mut active_liquidity_u64,
204 |         &mut active_liquidity_f64,
205 |         mutate_tick_state,
206 |         market_farm_indices,
207 |         args.modified_ticks,
208 |     )?;
209 |     continue;
210 | }
```

However, this approach has two issues.

1. Incorrect kappa_sy calculation

Under the CLMM model (consistent with the `add_liquidity` logic), liquidity is one-sided relative to the current spot price:

- Liquidity below the spot price is `PT` only.

- Liquidity above the spot price is **SY** only.

```

/* solana/programs/exponent_clmm/src/utils/math.rs */
082 | // Inside range: both sides
083 | let du_left = (spot_price_current - lower_price).max(epsilon_clamp);
084 | let price_eff_current = snap.get_effective_price(spot_price_current);
085 | let delta_c_right = (price_eff_current - price_eff_upper).max(epsilon_clamp);
086 |
087 | let liquidity_from_pt = (max_pt as f64) / du_left;
088 | let liquidity_from_sy = (max_sy as f64) / delta_c_right;
089 |
090 | let liquidity_target = liquidity_from_pt.min(liquidity_from_sy);
091 | let pt_need = liquidity_target * du_left;
092 | let sy_need = liquidity_target * delta_c_right;
093 |
094 | return LiquidityNeeds {
095 |     liquidity_target: liquidity_target.floor() as u64,
096 |     sy_needed: sy_need.floor() as u64,
097 |     pt_needed: pt_need.floor() as u64,
098 |     price_split_for_need: spot_price_current,
099 |     price_split_tick_idx: current_index,
100 | };
101 | }

```

However, in `swap_exact_out`, both `kappa_pt` and `kappa_sy` are computed using the same price interval from the current spot price to the tick range's left boundary, which only represents the price region below the current spot price. In a tick range that can hold both **PT** and **SY**, this interval only corresponds to the PT-side region. Using it to compute `kappa_sy` therefore does not reflect the SY-side liquidity and can underestimate the effective liquidity, leading to incorrect swap results.

It's recommended to compute the `kappa_sy` using the price interval from the current spot price to the tick range's right boundary, consistent with the logic used in the `swap` function.

2. Missing handling for one-sided liquidity tick ranges

The effective liquidity is always computed as `min(kappa_pt, kappa_sy)`. However, when the swap enters a fully PT-only tick range, `principal_sy_in_interval` is zero, which makes `kappa_sy` equal to zero.

This causes the effective liquidity to be zero, leading the swap logic to incorrectly skip a tick range that should still be usable for **SyToPt** swaps. As a result, valid liquidity is ignored, and swap execution becomes incorrect.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
121 | let (principal_pt_in_interval, principal_sy_in_interval) = {
122 |     let node = ticks.ticks_tree.get_node(current_left_boundary_index);
123 |     (node.value.principal_pt, node.value.principal_sy)
124 | };
193 | let kappa_pt = (principal_pt_in_interval as f64) / pt_max_to_left_f;

```

```
194 | let kappa_sy = (principal_sy_in_interval as f64) / y_max_to_boundary_f;  
195 | let kappa = kappa_pt.min(kappa_sy).min(1.0);
```

It's recommended to add explicit handling for fully one-sided liquidity tick ranges.

Resolution

This issue has been fixed by [20c90d0](#).

EXPONENT_CLMM

[H-05] Improper boundary handling in PtToSy exact-in swap

Identified in commit [818a6b8](#).

In the `PtToSy` exact-in swap path, both `current_price_spot` and the node corresponding to `current_left_boundary_index` are updated only when `sy_out_gross_u64 > 0`.

When the input amount is very small or the current interval contains insufficient `SY` principal, with `sy_out_gross_u64 = 0`, the updates at L573 and L593 are skipped, leaving both the left node and `current_price_spot` in an incorrect, stale state.

Because the price is not advanced, the unchanged state results in the following behavior: if the initial condition `(anchor_u_right - current_price_spot) > eps` holds, even when the current interval cannot produce any swappable `SY`, the price does not advance to the right. As a result, subsequent iterations remain trapped in the same interval.

Furthermore, because the node at the current left boundary is not updated, simply moving the `current_price_spot` update outside the `sy_out_gross_u64 > 0` branch is insufficient. When `sy_out_gross_u64 == 0` but `pt_in_segment > 0`, the node still fails to record the user's `pt_in_segment`, so the accounting remains incorrect.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
547 | if sy_out_gross_u64 > 0 {
...
573 |     if mutate_tick_state {
574 |         let node_mut =
575 |             ticks.ticks_tree.get_node_mut(current_left_boundary_index);
576 |         node_mut.value.principal_pt = node_mut
577 |             .value
578 |             .principal_pt
579 |             .checked_add(pt_in_segment)
580 |             .unwrap();
581 |         node_mut
582 |             .value
583 |             .update_tick_emissions(market_emission_indices);
584 |         node_mut.value.principal_sy = node_mut
585 |             .value
586 |             .principal_sy
587 |             .checked_sub(sy_out_gross_u64)
588 |             .unwrap();
589 |     };
592 |     amount_in_left = amount_in_left.checked_sub(pt_in_segment).unwrap();
593 |     current_price_spot = anchor_u_new;
594 | }
596 | // If we hit boundary, cross
597 | if (anchor_u_right - current_price_spot) <= eps && amount_in_left > 0 {
```

```

598 |         self.cross_one_boundary_and_update_core(
599 |             ticks,
600 |             SwapDirection::PtToSy,
601 |             right_boundary_index,
602 |             &mut current_left_boundary_index,
603 |             &mut current_price_spot,
604 |             &mut active_liquidity_u64,
605 |             &mut active_liquidity_f64,
606 |             mutate_tick_state,
607 |             market_farm_indices,
608 |             args.modified_ticks,
609 |         )?;
610 |     }
611 | }

```

Example. Consider intervals `[10, 20, 30]` with `current_price_spot = 10`. During `add_liquidity`, a user supplies `SY` liquidity in the `[20, 30]` interval. When executing a `PtToSy` swap:

- `l_trade_f64 = 0`, `current_price_spot = 10`, and `anchor_u_right = 20`.
- `sy_out_gross_u64 = 0`, so the swap should advance into the `[20, 30]` interval to access available `SY` liquidity.
- However, because L593 is not executed, `current_price_spot` remains unchanged.

The subsequent boundary-cross check still uses the unadvanced `current_price_spot`. Since `20 - 10 > eps`, the condition at L597 fails. The `cross_one_boundary_and_update_core` is not invoked, and `amount_in_left` remains unchanged. The `while` loop then repeatedly re-enters the `[10, 20]` interval, recomputes identical values, and makes no progress until the transaction eventually fails.

This issue can be triggered in the following scenarios:

- Small `principal_sy_in_interval`. When the current tick interval holds a small `principal_sy_in_interval`, `sy_out_gross_u64 = min(floor(...), principal_sy_in_interval)` becomes `0`. In this scenario, **all users'** `PtToSy` direction swaps will be unable to update `current_price_spot`, effectively blocking `PtToSy` execution for **all users** in that interval.
- Small `amount_in_left`. When a user's remaining funds are very small, and both the `du_actual` interval and `c_eff_old - c_eff_new` are sufficiently small such that `floor(l_trade × (c_eff_old - c_eff_new))` becomes `0`. Then, `sy_out_gross_u64` becomes `0`. In this case, the user can mitigate the issue by increasing the `amount_in`.

Therefore, when the current price is within a tick range with low `SY` principal or the user's provided amount is insufficient, the `PtToSy` precise swap cannot advance `current_price_spot` to execute the swap successfully. Additionally, when `pt_in_segment` is not `0`, fees cannot be charged from the user.

To resolve this, it is recommended to align the logic with the `SyToPt` implementation by moving the `current_price_spot` update and node update outside the `sy_out_gross_u64 > 0` branch. This ensures forward price progression and correct accounting even when no `SY` is matched in the current swap interval.

Resolution

This issue has been fixed by `a473a18`.

EXPONENT_CLMM

[H-06] Incorrect reversal in partial liquidity removal

Identified in commit [fc070e9](#).

The function `reverse_crossing_liquidity_corrections` in `remove_liquidity.rs` is used to revert, either fully or proportionally, the crossing-tick liquidity corrections introduced for `CrossingSplit` positions. These corrections are intended to adjust `liquidity_net` at the crossing boundaries so swaps observe `L_actual` in the crossing segment and `L_target` outside it.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
175 | /// Reverse the liquidity corrections at crossing tick boundaries during removal.
176 | fn reverse_crossing_liquidity_corrections(
177 |     ticks: &mut Ticks,
178 |     crossing_split: &crate::CrossingSplit,
179 |     position_lp_balance: u64,
180 |     liquidity_to_remove: u64,
181 |     is_full_removal: bool,
182 | ) -> Result<()> {
183 |     // Calculate the original correction amount
184 |     let l_reduction = position_lp_balance.checked_sub(crossing_split.lp_balance_crossing).unwrap();
186 |     // Calculate proportional correction to reverse
187 |     let crossing_correction = if is_full_removal {
188 |         l_reduction
189 |     } else {
190 |         // Proportional correction based on removal ratio
191 |         let ratio = liquidity_to_remove as u128 * l_reduction as u128 / position_lp_balance as u128;
192 |         ratio as u64
193 |     };
195 |     if crossing_correction > 0 {
196 |         let correction_i128 = i128::try_from(crossing_correction).unwrap();
198 |         // Reverse the corrections:
199 |         // At cross_left: we originally subtracted, now add back
200 |         ticks.tick_add_liquidity_net_gross(
201 |             crossing_split.cross_left_idx,
202 |             correction_i128,
203 |             false,
204 |             0,
205 |         )?;
207 |         // At cross_right: we originally added, now subtract
208 |         ticks.tick_sub_liquidity_net_gross(
209 |             crossing_split.cross_right_idx,
210 |             correction_i128,
211 |             true,
212 |         )?;
214 |         // Increase prefix_sum back (we decreased it during add)
215 |         ticks.current_prefix_sum = ticks
216 |             .current_prefix_sum
217 |             .checked_add(crossing_correction)
218 |             .unwrap();
219 |     }
221 |     Ok(())
222 | }

```

During partial liquidity removal, the code applies the wrong reversal operation at `cross_left_idx`. The correction originally applied at `cross_left_idx` is "upper-like", meaning it decreases `liquidity_net`. Reverting it should therefore increase `liquidity_net`. Instead, the current implementation calls `tick_add_liquidity_net_gross(..., lower=false)`, which further decreases `liquidity_net` and compounds the correction rather than undoing it.

In short, `tick_add_liquidity_net_gross` was mistakenly used where `tick_sub_liquidity_net_gross` should have been applied.

This can accumulate state drift across repeated partial removals, causing swaps to observe incorrect active liquidity and potentially breaking pricing and fee accounting assumptions.

If the crossing-tick boundary corrections are not reverted correctly during partial removals, the protocol can end up with persistently distorted `liquidity_net` and `current_prefix_sum` at crossing boundaries. This may cause swaps to use incorrect active liquidity in the crossing segment, leading to mispricing and incorrect fee accrual. In extreme cases, it can also trigger unexpected arithmetic edge cases and transaction failures, causing DoS around the active tick.

On the other hand, since the AMM commits to liquidity beyond what is actually backed by tokens, swap-pers may get abnormally favorable prices in certain price ranges. The protocol will suffer losses as a result.

The following unit test demonstrates the anomaly on a minimal state: after a partial removal, the code path intended to undo the `cross_left_idx` correction applies an operation that further decreases `liquidity_net` instead of increasing it.

```
use super::*;
use sokoban::NodeAllocatorMap;

/// PoC for: incorrect reversal direction at `cross_left_idx` on partial removal.
///
/// This test is written as `#[should_panic]` so it can run while clearly showing
/// the unexpected state transition.
#[test]
#[should_panic(expected = "BUG: expected cross_left liquidity_net to increase")]
fn poc_partial_removal_cross_left_reversal_direction() {
    let mut ticks = crate::Ticks::default();
    // Simulate state *after* a crossing correction was applied at deposit time:
    // - cross_left (upper-like): liquidity_net decreased by L_reduction
    // - cross_right (lower-like): liquidity_net increased by L_reduction
    let l_reduction: i128 = 200;
    let mut tick_left = crate::Tick::default();
    tick_left.liquidity_net = -l_reduction;
    tick_left.liquidity_gross = 1_000_000;
    let mut tick_right = crate::Tick::default();
```

```

tick_right.liquidity_net = l_reduction;
tick_right.liquidity_gross = 1_000_000;
let cross_left_idx = ticks.ticks_tree.insert(100u32, tick_left).unwrap();
let cross_right_idx = ticks.ticks_tree.insert(200u32, tick_right).unwrap();
let crossing_split = crate::CrossingSplit {
    cross_left_idx,
    cross_right_idx,
    // L_actual < L_target => correction exists
    lp_balance_crossing: 800,
    ..Default::default()
};
let position_lp_balance: u64 = 1000;
let liquidity_to_remove: u64 = 500; // partial removal => crossing_correction = 100
// This should *increase* cross_left liquidity_net by 100 (from -200 to -100).
// The current buggy code applies an operation that further *decreases* it.
super::reverse_crossing_liquidity_corrections(
    &mut ticks,
    &crossing_split,
    position_lp_balance,
    liquidity_to_remove,
    false,
)
.unwrap();
let left_net = ticks.ticks_tree.get_node(cross_left_idx).value.liquidity_net;
let right_net = ticks.ticks_tree.get_node(cross_right_idx).value.liquidity_net;
// cross_right behaves as expected for partial undo: +200 -> +100
assert_eq!(right_net, 100);
// Expected correct behavior at cross_left: -200 -> -100.
// Buggy behavior produces -300, so we panic with a stable message.
if left_net != -100 {
    panic!(
        "BUG: expected cross_left liquidity_net to increase (from -200 to -100), got {left_net}"
    );
}
}

```

The PoC file path is `solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity_poc.rs`, which can be run from the `solana/` workspace root:

```

cd solana
cargo test -p exponent_clmm --target aarch64-apple-darwin poc_partial_removal_cross_left_reversal_direction --
↳ --nocapture

```

The test produces:

```
BUG: expected cross_left liquidity_net to increase (from -200 to -100), got -300
```

To address this, consider changing `tick_add_liquidity_net_gross` to `tick_sub_liquidity_net_gross` in the `reverse_crossing_liquidity_corrections` function.

The current version is:

```
ticks.tick_add_liquidity_net_gross(  
    crossing_split.cross_left_idx,  
    correction_i128,  
    false,  
    0,  
)?;
```

The suggested version is:

```
ticks.tick_sub_liquidity_net_gross(  
    crossing_split.cross_left_idx,  
    correction_i128,  
    false,  
)?;
```

Resolution

This issue has been fixed by [cadab4c](#).

EXPONENT_CLMM

[H-07] Incorrect crossing split reversal mechanism

Identified in commit [fc070e9](#).

During the initial liquidity deposit, when opening a position, if the LP position's tick range contains the current spot price, the protocol records special crossing information in the `crossing_split` struct. It updates the `liquidity_net` and `liquidity_gross` values of the boundary ticks, and adjusts `current_prefix_sum` to accurately reflect the actual active liquidity within the crossing range.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
827 | /// Apply liquidity corrections for crossing tick to ensure swap correctness.
828 | ///
829 | /// When a position has a crossing split, we need to adjust liquidity_net at the
830 | /// crossing tick boundaries so that swaps use the correct L_actual in that segment.
831 | ///
832 | /// The liquidity_net changes are:
833 | /// - cross_left: decrease by (L_target - L_actual)
834 | /// - cross_right: increase by (L_target - L_actual)
835 | ///
836 | /// This ensures that when price enters the crossing segment, L_active decreases
837 | /// from L_target to L_actual, and when it leaves, L_active increases back.
838 | fn apply_crossing_liquidity_corrections(
839 |     ticks: &mut Ticks,
840 |     crossing_info: &CrossingTickInfo,
841 | ) -> Result<()> {
842 |     let l_reduction = crossing_info.lp_balance_target
843 |         .checked_sub(crossing_info.lp_balance_crossing)
844 |         .unwrap();
845 |
846 |     if l_reduction > 0 {
847 |         let delta_l = i128::try_from(l_reduction).unwrap();
848 |
849 |         // Same convention as external ticks: only tick_add_liquidity_net_gross for apply.
850 |         // At cross_left: when moving right into crossing segment, L decreases (upper-like).
851 |         ticks.tick_add_liquidity_net_gross(
852 |             crossing_info.cross_left_idx,
853 |             delta_l,
854 |             false, // upper-like: -ΔL at this boundary
855 |             0,
856 |         )?;
857 |
858 |         // At cross_right: when moving right out of crossing segment, L increases back (lower-like).
859 |         ticks.tick_add_liquidity_net_gross(
860 |             crossing_info.cross_right_idx,
861 |             delta_l,
862 |             true, // lower-like: +ΔL at this boundary
863 |             0,
864 |         )?;
865 |
866 |         // Reduce current_prefix_sum since position crosses current price
867 |         // and the crossing segment has reduced liquidity
868 |         ticks.current_prefix_sum = ticks
869 |             .current_prefix_sum

```

```

870 |         .checked_sub(l_reduction)
871 |         .unwrap();
872 |     }
873 |
874 |     Ok(())
875 | }

```

Example (Initial Open):

Consider an LP position opened over the tick range `[10, 20, 30, 40]`, with the current spot price at tick `25`. The sub-range `[20, 30]` becomes the `crossing_split` range, with `lp_balance_target = 50` and `lp_balance_crossing` in `[20, 30] = 40`.

The protocol adjusts `tick20.liquidity_net` to `-10` and `tick30.liquidity_net` to `+10`. Although the LP position records total liquidity as `50`, only `40` liquidity exists in the crossing range `[20, 30]`.

Accordingly, `current_prefix_sum` is updated to `50 - (50 - 40) = 40`. This ensures correct active liquidity accounting.

However, when increasing liquidity via `add_liquidity`, if the LP position contains valid `crossing_split` data, the protocol first calls `reverse_crossing_corrections_fully` to undo the previous crossing adjustment and then recalculates a new `crossing_split`.

The problem occurs when the LP position's tick range no longer contains the current spot price at the time of `add_liquidity`. In this case, the new `crossing_split` will be `None`, but the reversal step still executes.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
877 | /// Reverse ALL crossing corrections for an existing crossing split.
878 | /// Used when re-adding liquidity to a position that already had a crossing split.
879 | /// This fully undoes the liquidity_net and prefix_sum changes from the original split.
880 | fn reverse_crossing_corrections_fully(
881 |     ticks: &mut Ticks,
882 |     crossing_split: &CrossingSplit,
883 |     position_lp_balance: u64,
884 | ) -> Result<()> {
885 |     let l_reduction = position_lp_balance
886 |         .checked_sub(crossing_split.lp_balance_crossing)
887 |         .unwrap();
888 |
889 |     if l_reduction > 0 {
890 |         let delta_l = i128::try_from(l_reduction).unwrap();
891 |
892 |         // Same convention as external ticks: only tick_sub_liquidity_net_gross for reverse.
893 |         // At cross_left: undo the upper-like correction.
894 |         ticks.tick_sub_liquidity_net_gross(crossing_split.cross_left_idx, delta_l, false)?;
895 |
896 |         // At cross_right: undo the lower-like correction.
897 |         ticks.tick_sub_liquidity_net_gross(crossing_split.cross_right_idx, delta_l, true)?;
898 |

```

```

899 |         // Restore current_prefix_sum
900 |         ticks.current_prefix_sum = ticks
901 |             .current_prefix_sum
902 |             .checked_add(1_reduction)
903 |             .unwrap();
904 |     }
905 |
906 |     Ok(())
907 | }

```

This results in two issues. First, incorrect `tick.liquidity_net` updates cause the liquidity in the previously recorded crossing tick range to become incorrect. Second, when the LP position's tick range does not include the current spot price, `current_prefix_sum` is updated incorrectly.

Example (After Price Movement):

Continuing the same LP position `[10, 20, 30, 40]`, suppose a swap moves the current spot price to tick `40`. The LP position becomes fully one-sided and inactive, so active liquidity is effectively `0`.

If the user now calls `add_liquidity`, the reversal logic executes and updates the state as follows:

- `tick20.liquidity_net = -10 + 10 = 0`
- `tick30.liquidity_net = 10 - 10 = 0`
- `current_prefix_sum = 0 + (50 - 40) = 10`

Since the current spot price is no longer inside the LP's tick range, the new `crossing_split` becomes `None`.

As a result, active liquidity is incorrectly updated to `10`, and the tick range `[20, 30]`, which previously contained only `40` liquidity, is now treated as having `50` liquidity.

This leads to incorrect active liquidity accounting and breaks the intended invariant between position liquidity, boundary ticks, and `current_prefix_sum`.

The `current_prefix_sum` bug also exists in `reverse_crossing_liquidity_corrections` during the `remove_liquidity` process.

To address this, consider redesigning the reversal mechanism so tick liquidity is correctly recorded and updated.

Resolution

This issue has been fixed by [7bc61ee](#).

EXPONENT_CLMM

[H-08] Stale lp_balance_crossing breaks removal accounting

Identified in commit [fc070e9](#).

Function `add_liquidity_idempotent` handles re-adding liquidity to an existing position that may already have an active `CrossingSplit`.

When the new addition also spans the current price and produces a new `crossing_info`, the code fully reverses the old crossing corrections, applies new corrections based only on the new `delta_l`, and stores a new `CrossingSplit`.

This flow introduces two compounding errors.

1. Tick corrections only reflect the new addition's reduction. After `reverse_crossing_corrections_fully` zeroes out the old `+/-` corrections, `apply_crossing_liquidity_corrections` only applies the new addition's `l_reduction_new = delta_l - l_actual_new`. The old reduction, `l_reduction_old = old_lp_balance - old_lp_balance_crossing`, is permanently lost from the ticks, inflating apparent liquidity in the crossing segment.
2. The `lp_balance_crossing` only records the new addition's `l_actual`. The newly created `crossing_split.lit.lp_balance_crossing` stores `l_actual_new` instead of the correct accumulated value, `old_lp_balance_crossing + l_actual_new`. This creates a mismatch between the stored state and the already incorrect tick corrections, causing `remove_liquidity` to compute an `l_reduction` far exceeding what exists on the ticks.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
272 | pub fn add_liquidity_idempotent(
273 |     &mut self,
274 |     ticks: &mut Ticks,
275 |     position: &mut LpPosition,
276 |     args: AddLiquidityArgs,
277 | ) -> Result<AddLiquidityOutcome> {
381 |     let final_liquidity = if let Some(ref crossing_info) = accrue_result.crossing_info {
385 |         // Create new crossing split with updated values
386 |         let crossing_split = create_crossing_split(
387 |             ticks,
388 |             lower_tick_idx,
389 |             upper_tick_idx,
390 |             crossing_info,
391 |             market_farm_indices,
392 |         )?;
393 |         position.crossing_split = crossing_split;
```

The root cause is in `create_crossing_split`, which directly stores `crossing_info.lp_balance_crossing`, meaning only the new addition's `l_actual` is recorded:

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
645 | let l_target = position_liquidity as f64;
646 | let l_pt_cap = if du_pt_part > 0.0 { (used_pt as f64) / du_pt_part } else { l_target };
647 | let l_sy_cap = if sy_per_l_1 > 0.0 { (used_sy as f64) / sy_per_l_1 } else { l_target };
648 | let l_actual = l_pt_cap.min(l_sy_cap).min(l_target);
649 |
650 | // Record crossing info for CrossingSplit creation
651 | crossing_info = Some(CrossingTickInfo {
654 |     lp_balance_crossing: l_actual.floor() as u64,
656 | });
```

After the old corrections are reversed, the ticks treat the old liquidity as "fully active", but this is only an accounting artifact. The actual token backing in the crossing segment has not changed: it still only supports `old_lp_balance_crossing`, not `old_lp_balance`. Combined with the new addition's `l_actual_new`, the true token-backed liquidity in the crossing segment is `old_lp_balance_crossing + l_actual_new`, not `l_actual_new` alone. Meanwhile, the tick corrections after the reverse-then-apply flow only reflect `+(delta_l - l_actual_new)`, losing the old `+/(old_lp_balance - old_lp_balance_crossing)` entirely.

The mismatch between stored `lp_balance_crossing` and the actual tick-level correction causes `remove_liquidity` to compute an `l_reduction` far exceeding what exists on the ticks. This can trigger arithmetic underflow in `liquidity_gross.checked_sub(...)`, causing the transaction to panic and creating a DoS. It also distorts `liquidity_net` and `current_prefix_sum` at crossing boundaries, leading to incorrect active liquidity during swaps, mispricing, wrong fee/farm reward accrual, and even losses to the protocol's funds.

Consider a position over `[tick_A, tick_D)` with crossing segment `[tick_B, tick_C)`.

First, with an initial deposit where `l_target = 1000` and `l_actual = 700`, the state is:

tick	boundary liquidity_net	crossing correction	final liquidity_net
tick_A	+1000	—	+1000
tick_B	—	-300	-300
tick_C	—	+300	+300
tick_D	-1000	—	-1000

Stored values are `lp_balance = 1000` and `lp_balance_crossing = 700`. Swap traversal produces the correct active liquidity: `1000 -> 700 -> 1000 -> 0`.

Next, `add_liquidity_idempotent` is called with `delta_l = 500` and new `l_actual_new = 450`:

operation	tick_B	tick_C	note
reverse old ± 300	$-300 + 300 = 0$	$+300 - 300 = 0$	old correction removed
apply new ± 50	$0 - 50 = -50$	$0 + 50 = +50$	new correction for <code>delta_1</code> only
actual tick correction	-50	+50	

The buggy stored values become `lp_balance = 1500` and `lp_balance_crossing = 450`.

For the first error, the correct total token-backed liquidity in [B, C) is `old_lp_balance_crossing + l_actual_new = 700 + 450 = 1150`. The correct `l_reduction` should be $1500 - 1150 = 350$, so tick corrections should be ± 350 . Instead, the current code produces only ± 50 , because the old reduction of 300 is lost.

	correct	actual (buggy)
token-backed liquidity in [B, C)	$700 + 450 = 1150$	treated as $1500 - 50 = 1450$
<code>l_reduction</code> on ticks	± 350	± 50
phantom liquidity	0	$300 (1450 - 1150)$

This inflates apparent liquidity in the crossing segment by 300 units that have no token backing, causing swappers to extract more value than the vault supports.

For the second error, the stored `lp_balance_crossing` is stale:

	correct	actual (buggy)
<code>lp_balance_crossing</code>	$700 + 450 = 1150$	450
implied <code>l_reduction</code> at removal	$1500 - 1150 = 350$	$1500 - 450 = 1050$

The stored `lp_balance_crossing` value of 450 is inconsistent with both the correct value of 1150 and the actual tick corrections of ± 50 . This creates a three-way mismatch.

On full `remove_liquidity`, the mismatch becomes:

	correct	actual (buggy)
<code>l_reduction</code>	$1500 - 1150 = 350$	$1500 - 450 = 1050$
reverse at tick_B	+350 (from -350 to 0)	+1050 (from -50 to +1000)
reverse at tick_C	-350 (from +350 to 0)	-1050 -> <code>liquidity_gross.checked_sub(1050)</code> underflows

The transaction reverts, permanently locking the position.

The underflow-and-revert path above is the optimistic failure mode. In two common scenarios, the inflated `l_reduction` does not trigger an underflow and the state is silently corrupted.

1. During partial removal, the crossing correction is scaled proportionally as `crossing_correction = liquidity_to_remove * l_reduction / lp_balance`. For a small `liquidity_to_remove`, the correction stays within `liquidity_gross`, so the transaction succeeds. However, the correction is oversized relative to the correct value, such as 7 instead of 2 when removing 10 units, causing tick corrections and `prefix_sum` to drift with every partial removal.
2. During full removal, when other positions share the crossing ticks, other LP positions may contribute sufficient `liquidity_gross` at `tick_B / tick_C`, which is a common case. Even the full 1050 correction will not underflow `liquidity_gross` in that case. The transaction succeeds, but `liquidity_net` at the crossing boundaries is reversed in sign, such as `tick_B: -50 -> +1000` and `tick_C: +50 -> -1000`, flipping the crossing boundaries' direction entirely.

In both scenarios, swappers trade against phantom liquidity and extract real tokens from the vault, causing losses to the protocol.

The runnable proof of concept can be put in `solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity_poc.rs`.

```
use super::*;
use crate::state::ticks::{FarmYieldTrackers, EmissionYieldTrackers, Tick, Ticks};
use crate::PrincipalShareTrackers;
use sokoban::{NodeAllocatorMap, SENTINEL};

/// PoC: after `add_liquidity_idempotent`, two compounding errors occur:
/// 1. Tick corrections only reflect the new addition's reduction (+/-50 instead of +/-350).
/// 2. Stored `lp_balance_crossing` only records l_actual_new (450 instead of 1150).
/// Together, these cause `remove_liquidity` to compute a vastly inflated `l_reduction`,
/// triggering arithmetic underflow when reversing tick corrections.
#[test]
#[should_panic(expected = "BUG: l_reduction mismatch")]
fn poc_stale_lp_balance_crossing_after_idempotent_add() {
    let mut ticks: Ticks = unsafe { std::mem::zeroed() };
    ticks.ticks_tree.initialize();
    ticks.current_prefix_sum = 0;

    // Insert crossing boundary ticks (simulating state after initial deposit)
    let mut tick_b = Tick::default();
    tick_b.spot_price = 1.05;
    tick_b.liquidity_net = -300; // old crossing correction: l_reduction = 300
    tick_b.liquidity_gross = 1_000_000;
    tick_b.farms = FarmYieldTrackers::default();
    tick_b.emissions = EmissionYieldTrackers::default();

    let mut tick_c = Tick::default();
    tick_c.spot_price = 1.08;
    tick_c.liquidity_net = 300; // old crossing correction
    tick_c.liquidity_gross = 1_000_000;
    tick_c.farms = FarmYieldTrackers::default();
    tick_c.emissions = EmissionYieldTrackers::default();

    let cross_left_idx = ticks.ticks_tree.insert(500u32, tick_b).unwrap();
```

```

let cross_right_idx = ticks.ticks_tree.insert(800u32, tick_c).unwrap();

// Also need a current_tick for prefix_sum operations
let mut tick_cur = Tick::default();
tick_cur.spot_price = 1.06;
tick_cur.farms = FarmYieldTrackers::default();
tick_cur.emissions = EmissionYieldTrackers::default();
let cur_idx = ticks.ticks_tree.insert(600u32, tick_cur).unwrap();
ticks.current_tick = cur_idx;

// --- Simulate the old position state ---
let old_lp_balance: u64 = 1000;
let old_crossing_split = CrossingSplit {
    cross_left_idx,
    cross_right_idx,
    lp_balance_crossing: 700, // l_actual_old = 700, so old l_reduction = 300
    ..Default::default()
};

// --- Step 1: reverse old corrections (as add_liquidity_idempotent does) ---
reverse_crossing_corrections_fully(
    &mut ticks,
    &old_crossing_split,
    old_lp_balance,
)
.unwrap();

// After reversal, tick corrections should be zeroed
let net_b = ticks.ticks_tree.get_node(cross_left_idx).value.liquidity_net;
let net_c = ticks.ticks_tree.get_node(cross_right_idx).value.liquidity_net;
assert_eq!(net_b, 0, "tick_B should be 0 after reversal");
assert_eq!(net_c, 0, "tick_C should be 0 after reversal");

// --- Step 2: apply new crossing corrections (delta_l = 500, l_actual_new = 450) ---
let new_crossing_info = CrossingTickInfo {
    cross_left_idx,
    cross_right_idx,
    lp_balance_crossing: 450, // l_actual for new addition only
    lp_balance_target: 500, // delta_l
};
apply_crossing_liquidity_corrections(&mut ticks, &new_crossing_info).unwrap();

// Tick corrections now reflect only the new addition: +/- (500 - 450) = +/-50
let actual_tick_correction: u64 = 500 - 450; // = 50
let net_b = ticks.ticks_tree.get_node(cross_left_idx).value.liquidity_net;
assert_eq!(net_b, -(actual_tick_correction as i128));

// --- Step 3: simulate what create_crossing_split stores (the bug) ---
let stored_lp_balance_crossing = new_crossing_info.lp_balance_crossing; // 450 (buggy)
let new_lp_balance = old_lp_balance + 500; // 1500

// What remove_liquidity would compute:
let implied_l_reduction = new_lp_balance - stored_lp_balance_crossing; // 1500 - 450 = 1050

// The correct lp_balance_crossing should accumulate old + new token-backed liquidity:
let correct_lp_balance_crossing = old_crossing_split.lp_balance_crossing
    + new_crossing_info.lp_balance_crossing; // 700 + 450 = 1150
let correct_l_reduction = new_lp_balance - correct_lp_balance_crossing; // 1500 - 1150 = 350

// Note: even the correct l_reduction (350) does not match the actual tick correction (50),

```

```

// because the tick corrections are also buggy (Error 1: old +/-300 reduction was lost).
// In a fully correct implementation, tick corrections would also be +/-350.
assert_ne!(correct_l_reduction, actual_tick_correction,
    "tick corrections are also wrong: should be +/-{} but only +/-{}", correct_l_reduction,
    ↪ actual_tick_correction);

// The buggy l_reduction (1050) vastly exceeds the actual tick correction (50)
if implied_l_reduction != correct_l_reduction {
    panic!(
        "BUG: l_reduction mismatch - implied {} but correct is {} (tick correction is only {}). \
        stored lp_balance_crossing={} should have been {}",
        implied_l_reduction,
        correct_l_reduction,
        actual_tick_correction,
        stored_lp_balance_crossing,
        correct_lp_balance_crossing,
    );
}
}

```

Run: `cargo test -p exponent_clmm --target aarch64-apple-darwin --lib poc_stale_lp_balance_crossing_after_idempotent_add -- --nocapture`

The test simulates the exact three-step scenario above:

1. It inserts crossing ticks with an initial correction of `+/-300`, where `lp_balance = 1000` and `lp_balance_crossing = 700`.
2. It calls `reverse_crossing_corrections_fully` and then `apply_crossing_liquidity_corrections` for a new `delta_l = 500` with `l_actual_new = 450`, producing `+/-50` tick corrections.
3. It verifies that tick corrections are incomplete, with `+/-50` instead of the correct `+/-350`, confirming Error 1.
4. It compares the stored `lp_balance_crossing`, which is `450` and buggy, against the correct value of `1150`, confirming Error 2.

The test panics with:

```

BUG: l_reduction mismatch - implied 1050 but correct is 350 (tick correction is only 50).
stored lp_balance_crossing=450 should have been 1150

```

This confirms both errors: the tick corrections lost the old `+/-300` reduction, and the stored `lp_balance_crossing` is stale. Together, `remove_liquidity` would attempt to reverse a correction of `1050` against ticks that only carry `+/-50`, causing an arithmetic underflow.

Both errors stem from the same root cause: `reverse_crossing_corrections_fully` erases the old tick corrections, and the subsequent `apply_crossing_liquidity_corrections` only rebuilds corrections for the new addition's `delta_l`, permanently losing the old reduction.

The preferred fix is to skip the full reversal and use an incremental update. When the new `crossing_split` tick range is the same as the previous one, do not call `reverse_crossing_corrections_fully`. Instead, apply only the new addition's crossing corrections incrementally on top of the existing ones, and accumulate `lp_balance_crossing`:

```
// Do NOT call reverse_crossing_corrections_fully when crossing range is unchanged.
// Just apply the new addition's corrections incrementally:
apply_crossing_liquidity_corrections(ticks, &crossing_info)?;

// Accumulate lp_balance_crossing to preserve the old reduction:
crossing_split.lp_balance_crossing = old_crossing_split.lp_balance_crossing
    .checked_add(crossing_info.lp_balance_crossing)
    .unwrap();
position.crossing_split = crossing_split;
```

With this approach, using the example above:

- Old tick corrections `+/-300` are preserved.
- New corrections `+/-50` are added, producing final corrections of `+/-350`.
- `lp_balance_crossing = 700 + 450 = 1150`.
- `l_reduction = 1500 - 1150 = 350`, which matches the tick corrections.

Alternatively, if the full reverse-then-apply pattern must be retained, such as when the crossing tick range changes, the re-application must use the combined `l_reduction`:

```
// After reverse_crossing_corrections_fully:
// Recompute total lp_balance_crossing
let total_lp_balance_crossing = old_crossing_split.lp_balance_crossing
    .checked_add(crossing_info.lp_balance_crossing)
    .unwrap();

// Apply corrections using combined reduction
let combined_crossing_info = CrossingTickInfo {
    cross_left_idx: crossing_info.cross_left_idx,
    cross_right_idx: crossing_info.cross_right_idx,
    lp_balance_crossing: total_lp_balance_crossing,
    lp_balance_target: new_lp_balance, // = old_lp_balance + delta_l
};
apply_crossing_liquidity_corrections(ticks, &combined_crossing_info)?;

crossing_split.lp_balance_crossing = total_lp_balance_crossing;
position.crossing_split = crossing_split;
```

Resolution

This issue has been fixed by `0827ccd`.

EXPONENT_CLMM

[M-01] Cumulative rounding loss in sub tick token calculation

Identified in commit [818a6b8](#).

During the add liquidity process, the `compute_liquidity_target_and_token_needs` function calculates the delta liquidity amount (`liquidity_target`) by dividing the expected input `PT/SY` amount by the tick price range.

```

/* solana/programs/exponent_clmm/src/utils/math.rs */
038 | pub fn compute_liquidity_target_and_token_needs(
039 |     snap: EffSnap,
040 |     spot_price_current: f64,
041 |     price_eff_lower: f64,
042 |     price_eff_upper: f64,
043 |     lower_price: f64,
044 |     upper_price: f64,
045 |     lower_tick_idx: u32,
046 |     upper_tick_idx: u32,
047 |     current_index: u32,
048 |     max_sy: u64,
049 |     max_pt: u64,
050 |     epsilon_clamp: f64,
051 | ) -> LiquidityNeeds {
052 |     // Decide region and compute target ΔL and token needs under A-anchored formulas
053 |     if spot_price_current <= lower_price {
054 |         // Below range: SY only
055 |         let delta_c_total = (price_eff_lower - price_eff_upper).max(epsilon_clamp);
056 |         let liquidity_from_sy = (max_sy as f64) / delta_c_total;
057 |         let liquidity_target = liquidity_from_sy;
058 |         let sy_need = (liquidity_target) * delta_c_total;
059 |
060 |         return LiquidityNeeds {
061 |             liquidity_target: liquidity_target.floor() as u64,
062 |             sy_needed: sy_need as u64,
063 |             pt_needed: 0,
064 |             price_split_for_need: lower_price,
065 |             price_split_tick_idx: lower_tick_idx,
066 |         };
067 |     } else if spot_price_current >= upper_price {
068 |         // Above range: PT only
069 |         let du_total = (upper_price - lower_price).max(epsilon_clamp);
070 |         let liquidity_from_pt = (max_pt as f64) / du_total;
071 |         let liquidity_target = liquidity_from_pt;
072 |         let pt_need = liquidity_target * du_total;
073 |
074 |         return LiquidityNeeds {
075 |             liquidity_target: liquidity_target.floor() as u64,
076 |             sy_needed: 0,
077 |             pt_needed: pt_need.floor() as u64,
078 |             price_split_for_need: upper_price,
079 |             price_split_tick_idx: upper_tick_idx,
080 |         };
081 |     } else {
082 |         // Inside range: both sides

```

```

083 |     let du_left = (spot_price_current - lower_price).max(epsilon_clamp);
084 |     let price_eff_current = snap.get_effective_price(spot_price_current);
085 |     let delta_c_right = (price_eff_current - price_eff_upper).max(epsilon_clamp);
086 |
087 |     let liquidity_from_pt = (max_pt as f64) / du_left;
088 |     let liquidity_from_sy = (max_sy as f64) / delta_c_right;
089 |
090 |     let liquidity_target = liquidity_from_pt.min(liquidity_from_sy);
091 |     let pt_need = liquidity_target * du_left;
092 |     let sy_need = liquidity_target * delta_c_right;
093 |
094 |     return LiquidityNeeds {
095 |         liquidity_target: liquidity_target.floor() as u64,
096 |         sy_needed: sy_need.floor() as u64,
097 |         pt_needed: pt_need.floor() as u64,
098 |         price_split_for_need: spot_price_current,
099 |         price_split_tick_idx: current_index,
100 |     };
101 | }
102 | }

```

The computed `liquidity_target` is then used to determine how much PT/SY should be allocated to each sub tick range. The actual required `PT/SY` token amount is calculated as the sum of: `liquidity_target × (sub-tick range length)`.

However, the `PT/SY` requirement for each sub tick range is calculated using rounding down (floor). This introduces a precision loss accumulation issue.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
217 | self.for_each_interval_between(
218 |     lower_price_idx,
219 |     pt_slice_start,
220 |     pt_slice_end,
221 |     |segment: IntervalSegment| {
222 |         let segment_start = segment.price_spot_left.max(pt_slice_start);
223 |         let segment_end = segment.price_spot_right.min(pt_slice_end);
224 |         let segment_length_u = segment_end - segment_start;
225 |         if segment_length_u > 0.0 {
226 |             let is_crossing = segment.price_spot_left < price_split_for_need
227 |                 && segment.price_spot_right > price_split_for_need;
228 |
229 |             let principal_pt_delta =
230 |                 (position_liquidity as f64 * segment_length_u).floor() as u64;
231 |
232 |             if principal_pt_delta > 0 {
233 |                 if is_crossing {
234 |                     // save for later resolve
235 |                     pending_cross = Some((
236 |                         segment.left_index,
237 |                         principal_pt_delta,
238 |                         segment_length_u,
239 |                     ));
240 |                     return Ok(());
241 |                 }
242 |             }

```

```

243 |         let (_minted, used_pt, _used_sy) =
244 |             segment.ticks.mint_shares_for_tick_prestate_calc_used(
245 |                 segment.left_index,
246 |                 principal_pt_delta,
247 |                 0,
248 |                 lp_position,
249 |                 market_emission_indices,
250 |             )?;
251 |         total_pt_spend = total_pt_spend.checked_add(used_pt).unwrap();
252 |         add_principal_pt(segment.ticks, segment.left_index, used_pt)?;
253 |     }

```

When calculating the required amount over the entire tick range, the expected value would be: `floor(liquidity_target × total_tick_range_length)`, which incurs rounding loss only once. In contrast, when summing over multiple sub-tick ranges, the floor operation is applied multiple times, causing the total required PT/SY amount to be smaller than expected.

Example. In PT only mode, a user provides 50 PT tokens for the tick price range `[10, 20)`.

- The liquidity target is `liquidity_target = 50 / (20 - 10) = 5`. The expected PT requirement is 50.
- The tick range `[10, 20)` is split into two sub-ranges `[10, 14.9)` and `[14.9, 20)`. The actual PT requirement is calculated as `floor(5 × (14.9 - 10)) + floor(5 × (20 - 14.9)) = floor(24.5) + floor(25.5) = 49`.
- This is less than the expected 50 PT tokens.

It is recommended to use rounding ceil when calculating the required PT/SY amount for each sub-range to prevent precision loss accumulation.

In addition, the expected PT/SY amount calculation should also apply rounding up, so that the overall rounding behavior is in favor of the protocol. And update the final `used_PT/Sy <= args.max_PT/SY` accordingly.

Resolution

This issue has been fixed by [462f9be](#).

EXPONENT_CLMM

[M-02] Split discontinuity at lower and upper boundaries

Identified in commit [818a6b8](#).

The function `map_ledger_and_supply_for_split_anchored` is used by `split_principals_after_insert` to split principal ledgers when inserting a new boundary tick. It computes the PT split ratio as $r_{pt} = \frac{du_{pt_left}}{du_{pt_tot}}$, where $du_{pt_tot} = \min(\text{upper}, \text{current}) - \text{lower}$ and $du_{pt_left} = \min(\text{inserted}, \text{current}) - \text{lower}$.

Due to the `skip` behavior during swaps, the tick above the current price may still contain PT tokens, and the tick below the current price may still contain SY tokens.

When `current < lower`, all PT remains on the left tick. However, when `current == lower` is true, both numerator and denominator become 0 so that the `r_pt` becomes NaN.

This `r_pt` is then passed through `floor()` as `u64`, which saturates to 0, making `pt_new_lower=0` and moving all PT to the newly inserted tick.

Thus, the allocation direction at the equality boundary flips compared to the immediate left neighborhood. A symmetric discontinuity exists for SY at `current == upper`.

```
/* solana/programs/exponent_clmm/src/utils/math.rs */
117 | // PT-space: [u_l, min(u_r, u_cur)]
118 | let u_pt_r = spot_price_upper.min(spot_price_current);
119 | let du_pt_tot = u_pt_r - spot_price_lower;
120 | let du_pt_left = spot_price_inserted.min(spot_price_current) - spot_price_lower;
121 | let r_pt = du_pt_left / du_pt_tot;
...
135 | let dc_tot = c_l - c_r;
136 | let dc_left = c_l - c_m;
137 | let r_sy = dc_left / dc_tot;
138 |
139 | let sy_l = ((sy_lower as f64) * r_sy).floor() as u64;
140 | let sy_r = sy_lower.checked_sub(sy_l).unwrap();
```

It is recommended to add zero- denominator guards and clamp the split ratios `r_pt` and `r_sy` to `[0, 1]`, making the equality case follow a clear policy.

Resolution

This issue has been fixed by [387e2e8](#).

EXPONENT_CLMM

[M-03] kappa not clamped in exact-in swap calculation

Identified in commit [818a6b8](#).

In the current swap calculation, the variable `kappa` represents the proportion of effective liquidity that can be utilized within the current `[anchor_u_left, anchor_u_right]` range after accounting for inventory constraints from PT and SY, relative to the active liquidity.

However, the `kappa` is not clamped against `1`, which may cause the value to exceed `1`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
497 | // Y_max = (L/τ) * (C(u_old) - C(u_right))
498 | let c_eff_at_boundary = snapshot.get_effective_price(anchor_u_right);
499 | let y_max_to_boundary_f = (active_liquidity_f64) * (c_eff_old - c_eff_at_boundary);
500 |
501 | let eps = self.configuration_options.epsilon_clamp;
502 | let kappa_sy = principal_sy_in_interval as f64 / y_max_to_boundary_f;
503 |
504 | let du_to_left = current_price_spot - anchor_u_left;
505 | let pt_max_to_left_f = active_liquidity_f64 * du_to_left;
506 | let kappa_pt = (principal_pt_in_interval as f64) / (pt_max_to_left_f);
507 | let kappa = kappa_pt.min(kappa_sy);
```

Since the `l_trade` is larger than the amount of liquidity that can actually be used for trading, the computed `sy_out_gross_f` and `pt_out_gross_f` may be overstated.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
536 | // Token flows for this segment
537 | let pt_in_segment = (l_trade_f64 * du_actual).floor() as u64;
538 | let anchor_u_new = current_price_spot + (du_actual as f64);
539 | let c_eff_new = snapshot.get_effective_price(anchor_u_new);
540 | let sy_out_gross_f = (l_trade_f64 as f64) * (c_eff_old - c_eff_new);
541 |
542 | // Clamp by SY principal again after rounding (defensive)
543 | let sy_out_gross_u64 = sy_out_gross_f.floor() as u64;
544 | let sy_out_gross_u64 = sy_out_gross_u64.min(principal_sy_in_interval);
...
640 | // New effective price and anchor after consuming ΔC
641 | let c_eff_new = c_eff_old + delta_c_actual;
642 | let spot_price_new = snapshot.spot_price_from_effective_price(c_eff_new); // decreases
643 |
644 | // Token flows
645 | let sy_in_segment_f = (l_trade_f64 as f64) * (c_eff_new - c_eff_old);
646 | let du_abs = current_price_spot - spot_price_new;
647 | let pt_out_gross_f = l_trade_f64 * du_abs;
648 |
649 | // Clamp gross PT by available principal (defensive)
650 | let pt_out_gross_u64 =
651 |     (pt_out_gross_f.floor() as u64).min(principal_pt_in_interval);
```

This overstated value may result in losses to either the pool or users:

- When the `principal*_in_interval` constraint is not triggered, the user may receive more output for the same input, while the price advances less than it should.
- When the `principal*_in_interval` constraint is triggered, the user may suffer input losses, as the swap output is smaller than expected. If the user has correctly set `amount_out_min`, the swap may instead fail.

In either scenario, the incorrect calculations lead to instability in the pool's accounting, resulting in inaccurate pricing.

It is recommended to align the implementation with the exact-out logic by modifying `kappa = kappa_pt.min(kappa_sy);` to `kappa = kappa_pt.min(kappa_sy).min(1.0);`.

Resolution

This issue has been fixed by `8e87865`.

EXPONENT_CLMM

[M-04] Incorrect rounding direction in user provided amount calculation

Identified in commit [818a6b8](#).

In the swap function, `pt_in_segment` and `sy_in_segment_u64` represent the amount of `PT` or `SY` tokens that the user must provide for each swap segment in the `PtToSy` or `SyToPt` mode. These values are computed using `f64` arithmetic and then converted to `u64` using rounding down (floor).

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
537 | let pt_in_segment = (l_trade_f64 * du_actual).floor() as u64;

591 | // Consume input and advance state
592 | amount_in_left = amount_in_left.checked_sub(pt_in_segment).unwrap();

652 | let sy_in_segment_u64 = sy_in_segment_f.floor() as u64;

699 | // Consume input and advance state
700 | amount_in_left = amount_in_left.checked_sub(sy_in_segment_u64).unwrap();
```

Since rounding down favors the user, this behavior creates systematic under-collection by the protocol. It is recommended to switch to rounding up (ceil) so that the conversion favors the protocol.

The same issue exists in the `swap_exact_out` mode within the `calculate_swap_segment` function, where the `sy_in_segment_u64` value should also be converted using ceiling rounding instead of floor.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
826 | fn calculate_swap_segment(
827 |     gross_candidate: u64,
828 |     l_trade_f64: f64,
829 |     current_price_spot: f64,
830 |     du_to_left: f64,
831 |     anchor_u_left: f64,
832 |     snapshot: &EffSnap,
833 |     c_eff_old: f64,
834 |     lp_fee_rate: f64,
835 | ) -> (f64, u64, u64, u64) {
836 |     let du_candidate = (gross_candidate as f64) / l_trade_f64;
837 |
838 |     let du_clamped = du_candidate.min(du_to_left);
839 |     let mut spot_price_new = current_price_spot - du_clamped;
840 |     if spot_price_new < anchor_u_left {
841 |         spot_price_new = anchor_u_left;
842 |     }
843 |
844 |     let c_eff_new = snapshot.get_effective_price(spot_price_new);
845 |     let sy_in_segment_f = l_trade_f64 * (c_eff_new - c_eff_old);
846 |     let sy_in_segment_u64 = sy_in_segment_f.floor() as u64;
```

Resolution

This issue has been fixed by [8f84361](#).

EXPONENT_CLMM

[M-05] Stranded emissions from LP fee tokens

Identified in commit 818a6b8.

The CLMM protocol deposits **SY** tokens into the **SY** program to earn emission incentives. It queries the claimable emission amount from the **SY** program and divides it by the total deposited **SY** token amount to compute the per **SY** emission index (**lp_share_index**).

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
397 | pub fn update_emissions_from_position_state(
398 |     &mut self,
399 |     position_state: &PositionState,
400 |     lp_staked: u64,
401 | ) {
402 |     for (index, current_position) in position_state.emissions.iter().enumerate() {
403 |         let difference =
404 |             current_position.amount_claimable - self.emissions.trackers[index].last_seen_staged;
406 |         let amount_to_increase = Number::from_natural_u64(difference)
407 |             .checked_div(&Number::from_natural_u64(lp_staked))
408 |             .unwrap_or(Number::ZERO);
410 |         self.emissions.trackers[index].lp_share_index += amount_to_increase;
412 |         self.emissions.trackers[index].last_seen_staged = current_position.amount_claimable;
413 |     }
414 | }
```

The CLMM protocol splits the deposited **SY** tokens across sub tick ranges. Each sub tick range stores a **last_position_index**, which represents the emission index of the **SY** tokens in that range. When a tick is updated, the protocol distributes emissions to that tick by taking the difference between the current **lp_share_index** and **last_position_index**, then updating **last_position_index**.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
092 | pub fn update_tick_emissions(
093 |     &mut self,
094 |     market_indices: [Number; PERSONAL_TICK_YIELD_TRACKER_SIZE],
095 | ) {
096 |     if self.principal_sy == 0 {
097 |         for i in 0..PERSONAL_TICK_YIELD_TRACKER_SIZE {
098 |             self.emissions.trackers[i].last_seen_index = market_indices[i];
099 |         }
100 |         return;
101 |     }
103 |     let denom_sy = self.principal_sy as f64;
104 |     let denom_lp = self.principal_share_supply as f64;
105 |     if denom_sy != 0.0 && denom_lp != 0.0 {
106 |         for i in 0..PERSONAL_TICK_YIELD_TRACKER_SIZE {
107 |             let market_idx = market_indices[i];
108 |             let e = &mut self.emissions.trackers[i];
109 |             let delta_market = (market_idx - e.last_seen_index).to_f64().unwrap();
110 |             let sy_earned = delta_market * denom_sy;
111 |             let incr_per_lp = sy_earned / denom_lp;
```

```

112 |         e.last_position_index = e
113 |         .last_position_index
114 |         .checked_add(&DNum::from(incr_per_lp).to_precise_number())
115 |         .unwrap();
116 |         e.last_seen_index = market_idx;
117 |     }
118 | }
119 | }

```

Each user's LP position maintains an emission tracker that records the `last_seen_index` for the sub tick ranges in which the position was created. When the user claims rewards, the protocol computes the difference between the sub tick range's `last_position_index` and the tracker's `last_seen_index` to determine how many emission tokens the user is entitled to for the `SY` they supplied.

```

/* solana/programs/exponent_clmm/src/state/position.rs */
087 | pub fn accrue_fees_emissions_when_remove(
088 |     &mut self,
089 |     tick: &Tick,
090 |     share_to_remove: u64,
091 | ) -> Result<()> {
092 |     let share_amount = Number::from_natural_u64(share_to_remove);
093 |     for index in 0..PERSONAL_TICK_YIELD_TRACKER_SIZE {
094 |         let diff = tick.emissions.trackers[index].last_position_index
095 |             - self.emissions.trackers[index].last_seen_index;
096 |         let staged_amount = (diff * share_amount).floor_u64();
097 |         self.emissions.trackers[index].staged = self.emissions.trackers[index]
098 |             .staged
099 |             .checked_add(staged_amount)
100 |             .unwrap();
101 |         self.emissions.trackers[index].last_seen_index =
102 |             tick.emissions.trackers[index].last_position_index;
103 |     }
104 |     Ok(())
105 | }
106 | }

```

However, not all `SY` tokens owned by the CLMM protocol are held within tick ranges. During swaps, the `SY` portion of LP fees is removed from tick ranges and retained by the protocol. These fee-derived `SY` tokens are still counted in the total `SY` deposited into the `SY` program and therefore contribute to the protocol's claimable emission amount.

Yet the emission rewards generated by these fee `SY` tokens cannot be claimed by users, because user rewards are calculated only from the `SY` balances recorded inside tick ranges.

This creates a mismatch where some emission rewards become permanently locked.

It is recommended to add an emission claim and distribution mechanism for `SY` tokens collected as LP fees.

Resolution

This issue has been fixed by [b2caf15](#).

EXPONENT_CLMM

[M-06] Tick skip breaks liquidity invariant

Identified in commit [818a6b8](#).

In the CLMM protocol, liquidity is derived from the user's deposit over a price-distance formula: $\text{liquidity} = \text{input_token_amount} / \text{tick_price_range_length}$.

```
/* solana/programs/exponent_clmm/src/utils/math.rs */
081 | } else {
082 |     // Inside range: both sides
083 |     let du_left = (spot_price_current - lower_price).max(epsilon_clamp);
084 |     let price_eff_current = snap.get_effective_price(spot_price_current);
085 |     let delta_c_right = (price_eff_current - price_eff_upper).max(epsilon_clamp);
087 |     let liquidity_from_pt = (max_pt as f64) / du_left;
088 |     let liquidity_from_sy = (max_sy as f64) / delta_c_right;
090 |     let liquidity_target = liquidity_from_pt.min(liquidity_from_sy);
091 |     let pt_need = liquidity_target * du_left;
092 |     let sy_need = liquidity_target * delta_c_right;
094 |     return LiquidityNeeds {
095 |         liquidity_target: liquidity_target.floor() as u64,
096 |         sy_needed: sy_need.floor() as u64,
097 |         pt_needed: pt_need.floor() as u64,
098 |         price_split_for_need: spot_price_current,
099 |         price_split_tick_idx: current_index,
100 |     };
101 | }
```

During swaps, the user's input and output token amounts are also computed from the same relationship:

$\text{token_amount} = \text{liquidity} * \text{price_range_length}$.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
824 | /// Calculate all swap segment values for a given gross candidate.
825 | /// Returns: (spot_price_new, sy_in_segment, pt_out_gross_actual, total_fee_out)
826 | fn calculate_swap_segment(
827 |     gross_candidate: u64,
828 |     l_trade_f64: f64,
829 |     current_price_spot: f64,
830 |     du_to_left: f64,
831 |     anchor_u_left: f64,
832 |     snapshot: &EffSnap,
833 |     c_eff_old: f64,
834 |     lp_fee_rate: f64,
835 | ) -> (f64, u64, u64, u64) {
836 |     let du_candidate = (gross_candidate as f64) / l_trade_f64;
838 |     let du_clamped = du_candidate.min(du_to_left);
839 |     let mut spot_price_new = current_price_spot - du_clamped;
840 |     if spot_price_new < anchor_u_left {
841 |         spot_price_new = anchor_u_left;
842 |     }
844 |     let c_eff_new = snapshot.get_effective_price(spot_price_new);
845 |     let sy_in_segment_f = l_trade_f64 * (c_eff_new - c_eff_old);
846 |     let sy_in_segment_u64 = sy_in_segment_f.floor() as u64;
```

```

848 |     let pt_out_gross_actual = (l_trade_f64 * (current_price_spot - spot_price_new)).floor() as u64;
849 |     let total_fee_out = get_fee_from_amount(pt_out_gross_actual, lp_fee_rate);
851 |     (spot_price_new, sy_in_segment_u64, pt_out_gross_actual, total_fee_out)
852 | }

```

Together, these define the core CLMM invariant: `tick_range_token_amount = liquidity * price_range_length`.

However, swap execution introduces a "skip" behavior that can break this invariant. Specifically, if the remaining price distance from the current spot price to the current tick boundary is less than or equal to `epsilon_clamp`, the swap skips that final price segment and crosses the tick directly into the next range. This boundary crossing moves the spot price but does not adjust the corresponding token amounts for the skipped segment, creating an accounting gap between "where the price is" and "what has been swapped."

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
518 | if du_to_boundary <= self.configuration_options.epsilon_clamp {
519 |     // Cross boundary to unlock the next interval
520 |     // msg!("cross boundaries: {}, {}, {}", du_by_principal, du_by_input, du_to_boundary);
521 |     self.cross_one_boundary_and_update_core(
522 |         ticks,
523 |         SwapDirection::PtToSy,
524 |         right_boundary_index,
525 |         &mut current_left_boundary_index,
526 |         &mut current_price_spot,
527 |         &mut active_liquidity_u64,
528 |         &mut active_liquidity_f64,
529 |         mutate_tick_state,
530 |         market_farm_indices,
531 |         args.modified_ticks,
532 |     )?;
533 |     continue;
534 | }

740 | fn cross_one_boundary_and_update_core(
741 |     &self,
742 |     ticks: &mut Ticks,
743 |     direction: SwapDirection,
744 |     boundary_index_to_cross: u32,
745 |     current_left_boundary_index_io: &mut u32,
746 |     current_price_spot: &mut f64,
747 |     active_liquidity_u64: &mut u64,
748 |     active_liquidity_f64: &mut f64,
749 |     mutate_tick_state: bool,
750 |     market_farm_indices: [Number; PERSONAL_TICK_YIELD_TRACKER_SIZE],
751 |     modified_ticks: &mut ModifiedTicks,
752 | ) -> Result<()> {
753 |     if mutate_tick_state {
754 |         let node = ticks.ticks_tree.get_node_mut(boundary_index_to_cross);
755 |         node.value.cross_tick_update_outside_u128(
756 |             ticks.fee_growth_index_global_sy,
757 |             ticks.fee_growth_index_global_pt,
758 |             market_farm_indices,
759 |         )?;
760 |         modified_ticks.ticks.push(ModifiedTick {

```

```

761 |         tick_idx: boundary_index_to_cross,
762 |         fee_growth_pt_after: node.value.fee_growth_outside_pt,
763 |         fee_growth_sy_after: node.value.fee_growth_outside_sy,
764 |     });
765 | }
766 |
767 | // Update active liquidity by liquidity_net at the boundary
768 | let delta_net_i64 = ticks
769 |     .ticks_tree
770 |     .get_node(boundary_index_to_cross)
771 |     .value
772 |     .liquidity_net;
773 | let l_old_i64 = *active_liquidity_u64 as i64;
774 | let l_new_i64 = match direction {
775 |     SwapDirection::PtToSy => l_old_i64.checked_add(delta_net_i64).unwrap(), // crossing upward boundary
776 |     SwapDirection::SyToPt => l_old_i64.checked_sub(delta_net_i64).unwrap(), // crossing downward boundary
777 | };
778 | *active_liquidity_u64 = l_new_i64 as u64;
779 | *active_liquidity_f64 = l_new_i64 as f64;
780 |
781 | // Move spot to the boundary we cross
782 | let boundary_sqrt = ticks.get_spot_price(boundary_index_to_cross);
783 | *current_price_spot = boundary_sqrt;
784 |
785 | // Advance current left boundary index
786 | *current_left_boundary_index_io = match direction {
787 |     SwapDirection::PtToSy => boundary_index_to_cross,
788 |     SwapDirection::SyToPt => ticks
789 |         .predecessor_idx(*current_left_boundary_index_io)
790 |         .ok_or(ExponentCLMMErr::EndOfRange)?,
791 | };
792 |
793 | if mutate_tick_state {
794 |     ticks.current_prefix_sum = *active_liquidity_u64;
795 | }
796 |
797 | Ok(())
798 | }

```

Example. Assume a tick range `[10, 20)` contains both PT and SY liquidity, and the current spot price is 15. The range holds 50 PT and 50 SY.

1. A swap moves the spot price close to tick 20, leaving only 0.01 SY remaining in the range.
 - Because the remaining price distance to the boundary is below `epsilon_clamp`, the protocol skips the remainder and crosses the tick, updating the spot price to 20 **without** swapping out the remaining 0.01 SY and **without** collecting the corresponding PT input for that price segment.
 - At this point, the range violates the invariant on the PT side: `tick range PT token amount < liquidity * price range length`.
2. Now consider a reverse swap that moves the price back to 15.
 - Based on the pricing formula, traversing `[15, 20)` corresponds to 50 SY. But because 0.01 SY was left behind by the prior skip, the tick range now contains `50 + 0.01 = 50.01 SY`.

- At this point: $\text{tick range SY token amount} > \text{liquidity} * \text{price range length}$. This violates the core invariant.

Over time, this discrepancy can accumulate and produce unintended behaviors. For example, during add-liquidity, the LP position's share-tracker values for that tick range can be lower than expected. Two LPs providing the same token amounts to the same tick range may end up with different share-tracker values, resulting in unfair outcomes.

When crossing a tick due to the skip logic, the protocol should swap out the remaining token balance in the tick range and require the user to provide the corresponding input token amount for that price segment (with the input rounded up via `ceil` to avoid under-collection).

Resolution

This issue has been fixed by [828d44f](#).

EXPONENT_CLMM

[M-07] Incorrect max_sy_in parameter in wrapper_buy_yt

Identified in commit [818a6b8](#).

In the `wrapper_buy_yt` instruction, the user first mints `SY` tokens and then uses those minted `SY` tokens to execute the `buy_yt` instruction. The minted `SY` token amount is `mint_sy_return_data.sy_out_amount`.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_buy_yt.rs */
050 | let mint_sy_accounts = &ctx.remaining_accounts[..mint_sy_accounts_length as usize];
051 | let mint_sy_return_data = sy_cpi::cpi_mint_sy(
052 |     ctx.accounts.sy_program.key(),
053 |     asset_spend,
054 |     mint_sy_accounts,
055 |     mint_sy_accounts.to_vec().to_account metas(None),
056 | )?;
057 |
058 | let buy_yt_result = self_cpi::do_cpi_buy_yt(
059 |     self_cpi::BuyYtAccounts {
060 |         trader: ctx.accounts.buyer.to_account_info(),
061 |         market: ctx.accounts.market.to_account_info(),
062 |         token_sy_trader: ctx.accounts.token_sy_trader.to_account_info(),
063 |         token_yt_trader: ctx.accounts.token_yt_trader.to_account_info(),
064 |         token_pt_trader: ctx.accounts.token_pt_trader.to_account_info(),
065 |         token_sy_escrow: ctx.accounts.token_sy_escrow.to_account_info(),
066 |         token_pt_escrow: ctx.accounts.token_pt_escrow.to_account_info(),
067 |         address_lookup_table: ctx.accounts.market_address_lookup_table.to_account_info(),
068 |         token_program: ctx.accounts.token_program.to_account_info(),
069 |         sy_program: ctx.accounts.sy_program.to_account_info(),
070 |         token_fee_treasury_sy: ctx.accounts.token_fee_treasury_sy.to_account_info(),
071 |         event_authority: ctx.accounts.event_authority.to_account_info(),
072 |         program: ctx.accounts.program.to_account_info(),
073 |         ticks: ctx.accounts.ticks.to_account_info(),
074 |         token_fee_treasury_pt: ctx.accounts.token_fee_treasury_pt.to_account_info(),
075 |         token_yt_escrow: ctx.accounts.token_yt_escrow.to_account_info(),
076 |         exponent_core_program: ctx.accounts.exponent_core_program.to_account_info(),
077 |     },
078 |     interface_cpi_rem_accounts,
079 |     mint_sy_return_data.sy_out_amount + 1,
080 |     yt_out,
081 |     None,
082 | )?;
```

Inside `buy_yt`, the protocol calculates two values: `sy_to_borrow` (the amount of `SY` borrowed from the protocol) and `net_sy_spend` (the amount of `SY` the user must provide).

In practice, based on the calculation logic, `net_sy_spend` is equal to the `max_sy_in` parameter.

- `sy_to_borrow = sy_to_strip - max_sy_in`
- `net_sy_spend = sy_to_strip - sy_to_borrow`
- `net_sy_spend = max_sy_in`

```

/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
194 | let sy_to_strip = py_to_sy(sy_exchange_rate, yt_out) + 1;
195 | assert!(
196 |     sy_to_strip > max_sy_in,
197 |     "sy_to_strip must be greater than max_sy_in"
198 | );
199 |
200 | let sy_to_borrow = sy_to_strip - max_sy_in;
201 |
202 | // Check borrow amount is less than target strip amount
203 | assert!(
204 |     sy_to_borrow < sy_to_strip,
205 |     "sy_to_borrow must be less than sy_to_strip"
206 | );
207 |
208 | // The total amount of SY spent by the trader is the amount of SY stripped minus the amount of SY borrowed
209 | let net_sy_spend = sy_to_strip - sy_to_borrow;
210 |
211 | // Check slippage
212 | assert!(
213 |     net_sy_spend <= max_sy_in,
214 |     "net_sy_spend must be less than or equal to max_sy_in"
215 | );
216 |
228 |
229 | ctx.accounts
230 |     .do_repay_sy(net_sy_spend)
231 |     .expect("deposit sy failed");

```

However, the `max_sy_in` passed into `buy_yt` is incorrect. It is set to `mint_sy_return_data.sy_out_amount + 1`, which exceeds the actual amount of `SY` minted for the user. As a result, the instruction will always fail unless the user already holds extra `SY` tokens, since they cannot provide more `SY` than they just minted.

It's recommended to set `max_sy_in` to the actual minted amount `mint_sy_return_data.sy_out_amount`.

Resolution

This issue has been fixed by [a9c261f](#).

EXPONENT_CLMM

[M-08] Exact SY swap out check breaks buy_yt

Identified in commit [818a6b8](#).

In the `buy_yt` instruction, the user can first borrow some `SY` from the protocol, combine it with their own `SY`, and strip it into `PT` and `YT`. The `PT` is then swapped back to `SY` via the CLMM so the borrowed `SY` can be repaid. This flow allows users to effectively swap `SY` into `YT`.

```

/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
217 | // ===== Perform the borrow =====
218 | do_withdraw_sy(
219 |     sy_to_borrow,
220 |     &ctx.accounts.address_lookup_table.to_account_info(),
221 |     &ctx.accounts.market.cpi_sy_accounts,
222 |     &ctx.accounts.to_account_infos(),
223 |     ctx.remaining_accounts,
224 |     ctx.accounts.market.sy_program,
225 |     [&ctx.accounts.market.signer_seeds()],
226 | )
227 | .expect("withdraw failed");
228 |
229 | ctx.accounts
230 |     .do_repay_sy(net_sy_spend)
231 |     .expect("deposit sy failed");
232 |
233 | let pt_out = do_cpi_strip_sy(
234 |     sy_to_strip,
235 |     &ctx.accounts.address_lookup_table.to_account_info(),
236 |     &ctx.accounts.market.cpi_core_accounts,
237 |     &ctx.accounts.to_account_infos(),
238 |     ctx.remaining_accounts,
239 |     ctx.accounts.exponent_core_program.key(),
240 |     [&ctx.accounts.market.signer_seeds()],
241 | )
242 | .map(|x| x.amount_py_out)
243 | .expect("Strip failed");
244 |
245 | // transfer out yt and pt amounts before trade
246 | ctx.accounts
247 |     .do_transfer_pt_out(pt_out)
248 |     .expect("tranfet PT failed");
249 |
250 | ctx.accounts
251 |     .do_transfer_yt_out(yt_out)
252 |     .expect("tranfet YT failed");
253 |
254 | let res = do_cpi_trade_pt(
255 |     ctx.accounts.trade_pt_accounts(),
256 |     ctx.remaining_accounts,
257 |     yt_out,
258 |     SwapDirection::PtToSy,
259 |     Some(sy_to_borrow),
260 |     price_spot_limit,
261 | )

```

```
274 | .expect("sell PT failed");
```

However, after swapping **PT** to **SY** via the CLMM, the protocol requires the exact amount of **SY** received to equal the borrowed **SY**. This creates a self-inflicted DoS condition that makes **buy_yt** almost unusable in practice.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
277 | assert!(
278 |     received_sy <= sy_to_borrow + 2,
279 |     "received SY must be less than or equal to borrowed SY (with a buffer of 2)"
280 | );
281 |
282 | assert!(
283 |     received_sy >= sy_to_borrow,
284 |     "received SY must be greater than or equal to borrowed SY"
285 | );
```

While the strip step ensures that the base value of **PT** equals the borrowed **SY**, the actual **SY** obtained from swapping **PT** depends on the CLMM's current spot price and available liquidity.

As a result, the **SY** received will almost never match the borrowed amount exactly, except under extremely specific market conditions.

It's recommended to change the check to require only `received_sy >= sy_to_borrow`, allowing any excess **SY** to be retained by the user instead of requiring strict equality.

Resolution

This issue has been fixed by [99709a4](#).

EXPONENT_CLMM

[M-09] Rounding assertion between pt_out and yt_out can fail

Identified in commit [818a6b8](#).

In the current `buy_yt` computation flow, the amount of `SY` to be stripped is calculated as `sy_to_strip = floor(yt_out / sy_exchange_rate) + 1`. Correspondingly, during the stripping process, the `PT` output is computed as `pt_out = floor(sy_to_strip * r)`.

After these steps, the code asserts that the `PT` and `YT` outputs are either equal or differ by at most one. However, once `sy_exchange_rate >= 2`, that invariant is no longer preserved, and the assertion can fail.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
194 | let sy_to_strip = py_to_sy(sy_exchange_rate, yt_out) + 1;

/* solana/programs/exponent_core/src/utis/sy_cpi.rs */
284 | /// Convert PT & YT to SY
285 | pub fn py_to_sy(sy_exchange_rate: Number, amount_py: u64) -> u64 {
286 |     let sy = Number::from_natural_u64(amount_py) / sy_exchange_rate;
287 |     sy.floor_u64()
288 | }

/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
256 | assert!(
257 |     pt_out == yt_out || pt_out == yt_out + 1,
258 |     "PT out must equal YT out"
259 | );
```

Scenarios when the invariant breaks

Let $x = yt_out / r = f + \delta$, where $f = \text{floor}(x)$, $0 \leq \delta < 1$ and $r = sy_exchange_rate$.

- If $\delta > 0$: `sy_to_strip = f + 1`, `pt_out = floor((f + 1) * r) = yt_out + floor((1 - delta) * r)`.
- If $\delta = 0$: `sy_to_strip = x + 1`, `pt_out = floor((x + 1) * r) = yt_out + floor(r)`.

When $r < 2$, `pt_out < yt_out + 2`. Therefore, the condition `pt_out == yt_out || pt_out == yt_out + 1` always holds so the assertion at L257 passes.

However, when $r \geq 2$, there exist valid `delta` such that `floor((1 - delta) * r) >= 2` or `floor(r) >= 2`, which implies `pt_out >= yt_out + 2`. In this case, the assertion at L257 fails.

Practical impact

As the `sy_exchange_rate` increases over time, once $r \geq 2$, the `buy_yt` instruction may consistently revert due to this rounding assertion, effectively blocking `YT` purchases and resulting in a denial of service

condition.

It is recommended to relax the assertion to accommodate the full rounding range by replacing it with `assert!(pt_out >= yt_out && pt_out <= yt_out + floor(r))`.

Resolution

This issue has been fixed by [d633742](#).

EXPONENT_CLMM

[M-10] DoS add_liquidity calls by zero lp_balance_crossing

Identified in commit [7bc61ee](#).

A partial withdrawal can floor `position.crossing_split.lp_balance_crossing` down to `0` without clearing the split.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
267 | // Partial removal: proportionally reduce lp_balance_crossing
268 | if position.crossing_split.is_active() {
269 |     // Position lp_balance has already been reduced at this point
270 |     let original_balance = position.lp_balance + liquidity_to_remove;
271 |     let ratio = position.lp_balance as u128
272 |         * position.crossing_split.lp_balance_crossing as u128
273 |         / original_balance as u128;
274 |     position.crossing_split.lp_balance_crossing = ratio as u64;
275 | }
```

For example, assume a position with:

- `original_balance = 2`
- `lp_balance_crossing = 1`

If the user performs a partial withdrawal of `1`, then:

- `position.lp_balance = 1`
- `lp_balance_crossing = floor(1 * 1 / 2) = 0`

So a single partial withdrawal can round the `position.crossing_split.lp_balance_crossing` down to zero.

At the same time, whether a split is considered active is determined solely by the two crossing indices.

```
/* solana/programs/exponent_clmm/src/state/position.rs */
051 | pub fn is_active(&self) -> bool {
052 |     self.cross_left_idx != SENTINEL && self.cross_right_idx != SENTINEL
053 | }
```

Therefore, in the `add_liquidity` flow, once execution reaches `equalize_previous_crossing_split_if_needed`, it proceeds to `build_crossing_equalization_plan`, which immediately rejects `lp_balance_crossing == 0`:

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
1078 | let l_actual = position.crossing_split.lp_balance_crossing;
1079 | if position.lp_balance == l_actual {
1080 |     return Ok(vec![]);
1081 | }
1082 |
1083 | require!(
1084 |     l_actual > 0,
1085 |     ExponentCLMMError::CrossingEqualizationInvariantViolated
1086 | );

```

As a result, a user can permanently lose the ability to add liquidity to a position after a partial withdrawal. This is especially easy to trigger on low-liquidity crossing positions, where proportional floor division can zero out the crossing side in a single step.

It is recommended to clear the split whenever partial removal reduces `lp_balance_crossing` to `0`.

Resolution

This issue has been fixed by [56747c9](#).

EXPONENT_CLMM

[M-11] Zero min_l_to_activate disables tick initialization protection

Identified in commit [7bc61ee](#).

In `add_liquidity.rs`, both the fresh add path and the idempotent add path pass `min_l_to_activate = 0` into `tick_add_liquidity_net_gross` even when a boundary tick is newly initialized.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
155 | // When a tick is initialized, min_l_to_activate is frozen and must be deducted from user's LP.
156 | let frozen_lower = ticks.tick_add_liquidity_net_gross(
157 |     lower_tick_idx,
158 |     delta_l_i128,
159 |     true,
160 |     0, // no min_l_to_activate in add_liquidity path (existing position or new)
161 | );
162 | let frozen_upper = ticks.tick_add_liquidity_net_gross(
163 |     upper_tick_idx,
164 |     delta_l_i128,
165 |     false,
166 |     0, // no min_l_to_activate in add_liquidity path (existing position or new)
167 | );
```

The tick-level helper explicitly documents that this frozen amount is the mechanism intended to stop a minimum-liquidity bypass attack:

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
283 | /// Returns the amount of liquidity that was frozen (only non-zero when initializing a new tick).
284 | /// When a tick is newly initialized (liquidity_gross was 0), we freeze min_l_to_activate
285 | /// to prevent the bypass attack where users add liquidity and immediately withdraw most of it.
313 | // When initializing a new tick, freeze min_l_to_activate to prevent withdrawal below minimum.
314 | // This frozen liquidity is deducted from the initializing user's LP amount.
315 | let frozen_amount = if is_initializing {
316 |     node.frozen_liquidity = min_l_to_activate;
317 |     min_l_to_activate
318 | } else {
319 |     0
320 | };
```

However, because the add path always supplies `0`, every newly initialized tick ends up with `frozen_liquidity = 0`. The later withdrawal check therefore becomes ineffective:

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
271 | // Ensure liquidity_gross doesn't go below frozen_liquidity
272 | // This prevents users from withdrawing liquidity that was frozen during tick initialization
273 | assert!(
274 |     new_liquidity_gross >= node.frozen_liquidity,
275 |     "cannot withdraw below frozen liquidity"
276 | );
```

With `node.frozen_liquidity == 0`, the assertion imposes no minimum at all.

As a result, the documented tick-initialization protection is disabled on the `add_liquidity` flow. A user can initialize previously empty boundary ticks with arbitrarily small liquidity and then later remove the position back down to zero, even though the tick code is explicitly structured around freezing a non-zero minimum when a tick is first created.

It is recommended to pass a real non-zero `min_l_to_activate` from the `add_liquidity` path.

Resolution

This issue has been fixed by `50931a6`.

EXPONENT_CLMM

[M-12] Dust-sized shares can DoS subsequent add_liquidity calls

Identified in commit [7bc61ee](#).

After repeated tick splits, the shares corresponding to an active historical crossing range can be fragmented into small shares.

Before both `add_liquidity` and `withdraw_liquidity`, the program calls `update_lp_position_shares`, which in turn calls `project_anchor_shares_to_current_ticks` to project the position's existing shares onto the current tick structure. During this projection, a single old share can be split into multiple child shares.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/update_lp_shares.rs */
017 | let mut recomputed_shares_for_current_ticks = ticks
018 |     .project_anchor_shares_to_current_ticks(position.share_trackers.clone())
019 |     .unwrap();

/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
311 | let left_share = if splited_full_range <= 0.0 {
312 |     Number::ZERO
313 | } else {
314 |     let ratio = (current_split_range / splited_full_range).clamp(0.0, 1.0);
315 |     let scaled_num = (ratio * Number::DENOM as f64).round() as u128;
316 |     principal_share
317 |         .lp_share
318 |         .fast_mul_ratio(scaled_num, Number::DENOM)
319 | };
333 | let migrated_share = principal_share.lp_share.fast_sub_u128(&left_share);

/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
337 | stack.push(PrincipalShare {
338 |     tick_idx: splited_index,
339 |     right_tick_idx: right_index,
340 |     split_epoch: principal_share.split_epoch,
341 |     lp_share: migrated_share,
342 |     emissions: new_emissions,
343 | });
```

Later in the `add_liquidity` flow, `build_crossing_equalization_plan` processes each projected share independently, and any share whose floored size is zero is skipped immediately:

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
1107 | for (share_index, share) in position.share_trackers.trackers.iter().enumerate() {
1123 |     let share_u128 = share.lp_share.fast_floor_u128();
1124 |     if share_u128 == 0 {
1125 |         continue;
1126 |     }
1137 |     let share_delta = share
1138 |         .lp_share
1139 |         .fast_mul_ratio(liquidity_delta as u128, l_actual as u128);
```

At the same time, the `add_liquidity` flow requires the final equalization plan to be non-empty:

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
1201 | require!(
1202 |     !plans.is_empty(),
1203 |     ExponentCLMMError::CrossingEqualizationInvariantViolated
1204 | );
```

As a result, if the old crossing range has been fragmented finely enough, all shares inside that range may be skipped during equalization. Once this happens, `plans` becomes empty and `add_liquidity` reverts with `CrossingEqualizationInvariantViolated`, even though the position still appears to carry an active historical crossing split.

For example:

- A small position has an active historical crossing split on `[20,30)`, and that crossing interval originally contains `1` minted share for the position.
- A new tick `25` is inserted, and `update_lp_position_shares` projects that share into `0.5` on `[20,25)` and `0.5` on `[25,30)`.
- Both shares satisfy `lp_share.fast_floor_u128() == 0` and are therefore skipped, `plans` becomes empty, and the next `add_liquidity` reverts.

This issue may be triggered in the following scenarios:

1. A user adds only a small amount of liquidity, while multiple new ticks are inserted within the range, causing each share to carry only a very small amount of liquidity.
2. A `withdraw_liquidity` operation leaves the position with only a small amount of remaining liquidity.

In both cases, a later `add_liquidity` call can revert when checking whether the equalization `plans` is empty, leaving the position unable to add liquidity again.

It is recommended to handle an empty equalization plan caused by dust-sized shares rather than reverting.

Resolution

This issue has been fixed by [e041f04](#).

EXPONENT_CLMM

[M-13] Potentially inaccurate equalization estimates

Identified in commit [7bc61ee](#).

In `wrapper_provide_liquidity_base`, the function first estimates the additional `SY` and `PT` token amounts required for equalization (crossing split) via `expand_user_budgets_for_existing_position_equalization`. This estimation depends on the current tick state, and the resulting amounts are added to `external_sy_constraint` and `external_pt_to_buy` to form `adjusted_external_sy_constraint` and `adjusted_external_pt_to_buy`.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_base.rs */
207 | let (_existing_position_before_trade, equalization_precalc) =
208 |     with_ticks_snapshot(&ctx.accounts.ticks, |ticks_snapshot_before_trade| {
209 |         load_existing_position_precalc(
210 |             ticks_snapshot_before_trade,
211 |             &ctx.accounts.lp_position,
212 |         )
213 |     })?;
214 | let (adjusted_external_sy_constraint, adjusted_external_pt_to_buy) =
215 |     expand_user_budgets_for_existing_position_equalization(
216 |         &equalization_precalc,
217 |         external_sy_constraint,
218 |         external_pt_to_buy,
219 |     )
220 |     .unwrap_or((external_sy_constraint, external_pt_to_buy));

/* solana/programs/exponent_clmm/src/instructions/wrappers/add_liquidity_precalc.rs */
080 | pub(crate) fn expand_user_budgets_for_existing_position_equalization(
081 |     precalc: &ExistingPositionAddLiquidityPrecalc,
082 |     effective_max_sy: u64,
083 |     effective_max_pt: u64,
084 | ) -> Result<(u64, u64)> {
085 |     Ok((
086 |         precalc.required_user_max_sy(effective_max_sy)?,
087 |         precalc.required_user_max_pt(effective_max_pt)?,
088 |     ))
089 | }

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
532 | pub(crate) fn required_user_max_sy(&self, effective_max_sy: u64) -> Result<u64> {
533 |     effective_max_sy
534 |     .checked_add(self.outcome.sy_spent)
535 |     .ok_or(ExponentCLMMEError::CrossingEqualizationInvariantViolated)?
536 |     .checked_sub(self.outcome.sy_released)
537 |     .ok_or(ExponentCLMMEError::CrossingEqualizationInvariantViolated.into())
538 | }
539 |
540 | pub(crate) fn required_user_max_pt(&self, effective_max_pt: u64) -> Result<u64> {
541 |     effective_max_pt
542 |     .checked_add(self.outcome.pt_spent)
543 |     .ok_or(ExponentCLMMEError::CrossingEqualizationInvariantViolated)?
544 |     .checked_sub(self.outcome.pt_released)
```

```

545 |         .ok_or(ExponentCLMMEError::CrossingEqualizationInvariantViolated.into())
546 |     }

```

Next, the function borrows `adjusted_external_sy_constraint SY` tokens and swaps them into `adjusted_external_pt_to_buy PT` tokens via `do_cpi_buy_pt`. Any unused `SY`, together with the acquired `PT`, is then used to add liquidity.

```

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_base.rs */
246 | let trade_result = ctx.accounts.do_cpi_buy_pt(
247 |     ctx.remaining_accounts,
248 |     adjusted_external_sy_constraint,
249 |     adjusted_external_pt_to_buy,
250 |     None,
251 | )?;

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_base.rs */
261 | let deposit_budget_effect = compute_existing_position_budget_effect_after_sy_to_pt_trade(
262 |     &equalization_precalc_after_trade,
263 |     adjusted_external_sy_constraint,
264 |     trade_result.amount_in,
265 |     trade_result.amount_out,
266 | )?;
267 |
268 | // CPI to deposit liquidity
269 | let deposit_liquidity_return_data = self_cpi::do_cpi_deposit_liquidity_idempotent(
270 |     ctx.accounts.to_deposit_liquidity_accounts(),
271 |     interface_cpi_rem_accounts,
272 |     lower_tick_key,
273 |     upper_tick_key,
274 |     deposit_budget_effect.user_max_sy,
275 |     deposit_budget_effect.user_max_pt,
276 |     &ctx.accounts.rent,
277 | )?;

/* solana/programs/exponent_clmm/src/instructions/wrappers/add_liquidity_precalc.rs */
067 | pub(crate) fn compute_existing_position_budget_effect_after_sy_to_pt_trade(
068 |     precalc: &ExistingPositionAddLiquidityPrecalc,
069 |     total_sy_budget: u64,
070 |     trade_sy_spent: u64,
071 |     trade_pt_out: u64,
072 | ) -> Result<ExistingPositionBudgetEffect> {
073 |     let sy_remainder = total_sy_budget
074 |         .checked_sub(trade_sy_spent)
075 |         .ok_or(ExponentCLMMEError::InsufficientBudgetSY)?;
076 |
077 |     compute_existing_position_budget_effect(precalc, sy_remainder, trade_pt_out)
078 | }

/* solana/programs/exponent_clmm/src/instructions/wrappers/add_liquidity_precalc.rs */
050 | pub(crate) fn compute_existing_position_budget_effect(
051 |     precalc: &ExistingPositionAddLiquidityPrecalc,
052 |     user_max_sy: u64,
053 |     user_max_pt: u64,
054 | ) -> Result<ExistingPositionBudgetEffect> {
055 |     Ok(ExistingPositionBudgetEffect {
056 |         user_max_sy,
057 |         user_max_pt,

```

```

058 |         effective_max_sy: precalc.effective_max_sy(user_max_sy)?,
059 |         effective_max_pt: precalc.effective_max_pt(user_max_pt)?,
060 |         fixed_sy_spent: precalc.sy_spent(),
061 |         fixed_pt_spent: precalc.pt_spent(),
062 |         fixed_sy_released: precalc.sy_released(),
063 |         fixed_pt_released: precalc.pt_released(),
064 |     })
065 | }

```

However, the `do_cpi_buy_pt` self CPI modifies the `CLMM` tick state. As a result, the previously estimated `adjusted_external_sy_constraint` and `adjusted_external_pt_to_buy` may no longer be accurate. Consequently, the final token amounts used for liquidity provision cannot be guaranteed to match the intended `external_pt_to_buy`.

Since it is not possible to accurately estimate the equalization requirements before `do_cpi_buy_pt`, this introduces inconsistency in the liquidity provisioning process.

It is recommended to move the equalization estimation (`expand_user_budgets_for_existing_position_equalization`) to after `do_cpi_buy_pt`, so that calculations are based on the updated tick state.

Resolution

This issue has been fixed by `cdc37b0`.

EXPONENT_CLMM

[L-01] Unsafe u64 to i64 type casting

Identified in commit [818a6b8](#).

1. compute_liquidity_target_and_token_needs

During the add liquidity process, the delta liquidity amount is calculated by dividing the expected input **P** **T/SY** amount by the tick price range.

The expected input **PT/SY** amount is of type **u64**, while the price range is of type **f64**, and the final **liquidity_target** is of type **u64**.

```
/* solana/programs/exponent_clmm/src/utils/math.rs */
038 | pub fn compute_liquidity_target_and_token_needs(
039 |     snap: EffSnap,
040 |     spot_price_current: f64,
041 |     price_eff_lower: f64,
042 |     price_eff_upper: f64,
043 |     lower_price: f64,
044 |     upper_price: f64,
045 |     lower_tick_idx: u32,
046 |     upper_tick_idx: u32,
047 |     current_index: u32,
048 |     max_sy: u64,
049 |     max_pt: u64,
050 |     epsilon_clamp: f64,
051 | ) -> LiquidityNeeds {
052 |     // Decide region and compute target ΔL and token needs under A-anchored formulas
053 |     if spot_price_current <= lower_price {
054 |         // Below range: SY only
055 |         let delta_c_total = (price_eff_lower - price_eff_upper).max(epsilon_clamp);
056 |         let liquidity_from_sy = (max_sy as f64) / delta_c_total;
057 |         let liquidity_target = liquidity_from_sy;
058 |         let sy_need = (liquidity_target) * delta_c_total;
059 |
060 |         return LiquidityNeeds {
061 |             liquidity_target: liquidity_target.floor() as u64,
062 |             sy_needed: sy_need as u64,
063 |             pt_needed: 0,
064 |         };
065 |     } else if spot_price_current >= upper_price {
066 |         // Above range: PT only
067 |         let du_total = (upper_price - lower_price).max(epsilon_clamp);
068 |         let liquidity_from_pt = (max_pt as f64) / du_total;
069 |         let liquidity_target = liquidity_from_pt;
070 |         let pt_need = liquidity_target * du_total;
071 |
072 |         return LiquidityNeeds {
073 |             liquidity_target: liquidity_target.floor() as u64,
074 |             sy_needed: 0,
075 |             pt_needed: pt_need.floor() as u64,
076 |         };
077 |     }
078 | }
```

```

081 |     } else {
082 |         // Inside range: both sides
083 |         let du_left = (spot_price_current - lower_price).max(epsilon_clamp);
084 |         let price_eff_current = snap.get_effective_price(spot_price_current);
085 |         let delta_c_right = (price_eff_current - price_eff_upper).max(epsilon_clamp);
086 |
087 |         let liquidity_from_pt = (max_pt as f64) / du_left;
088 |         let liquidity_from_sy = (max_sy as f64) / delta_c_right;
089 |
090 |         let liquidity_target = liquidity_from_pt.min(liquidity_from_sy);
091 |         let pt_need = liquidity_target * du_left;
092 |         let sy_need = liquidity_target * delta_c_right;
093 |
094 |         return LiquidityNeeds {
095 |             liquidity_target: liquidity_target.floor() as u64,
096 |             sy_needed: sy_need.floor() as u64,
097 |             pt_needed: pt_need.floor() as u64,
100 |         };
101 |     }
102 | }

```

However, `liquidity_target` is then unsafely cast to an `i64` (`delta_l_i64`) using the `as` keyword. This value is subsequently used to update both the tick's liquidity net and liquidity gross.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
093 | // Decide region and per-budget ΔL
094 | let liquidity_needs = compute_liquidity_target_and_token_needs(
095 |     snap,
096 |     current_spot,
097 |     price_eff_lower,
098 |     price_eff_upper,
099 |     lower_price,
100 |     upper_price,
101 |     lower_tick_idx,
102 |     upper_tick_idx,
103 |     ticks.current_tick,
104 |     args.max_sy,
105 |     args.max_pt,
106 |     self.configuration_options.epsilon_clamp,
107 | );
108 |
109 | // 5) Materialize integer ΔL units and update liquidity_net at boundaries
110 | let delta_l_i64 = liquidity_needs.liquidity_target as i64;
111 | ticks.tick_add_liquidity_net_gross(
112 |     lower_tick_idx,
113 |     delta_l_i64,
114 |     true,
115 |     self.configuration_options.min_lp_tick_amount,
116 | ); // +ΔL at lower
117 | ticks.tick_add_liquidity_net_gross(
118 |     upper_tick_idx,
119 |     delta_l_i64,
120 |     false,
121 |     self.configuration_options.min_lp_tick_amount,
122 | ); // -ΔL at upper
123 |

```

If `liquidity_target` exceeds `i64::MAX`, this cast will overflow and produce a negative value, causing the tick's liquidity net and liquidity gross to be updated incorrectly.

The likelihood of this scenario depends on how strictly the `epsilon_clamp` configuration bounds the minimum price range.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
261 | pub fn tick_add_liquidity_net_gross(
262 |     &mut self,
263 |     idx: u32,
264 |     delta_l_i64: i64,
265 |     lower: bool,
266 |     min_l_to_activate: u64,
267 | ) -> Result<()> {
268 |     let node = &mut self.ticks_tree.get_node_mut(idx).value;
269 |     if lower {
270 |         node.liquidity_net = node.liquidity_net.checked_add(delta_l_i64).unwrap();
271 |     } else {
272 |         node.liquidity_net = node.liquidity_net.checked_sub(delta_l_i64).unwrap();
273 |     }
274 |
275 |     node.liquidity_gross = node
276 |         .liquidity_gross
277 |         .checked_add(delta_l_i64.abs().try_into().unwrap())
278 |         .unwrap();
279 |
280 |     node.liquidity_gross
281 |         .checked_sub(min_l_to_activate)
282 |         .expect("not enough liquidity for tick to be initialized");
283 |
284 |     Ok(())
285 | }
```

It is recommended to change the `liquidity_net` variable to `i128` and use type-safe conversions.

2. remove_liquidity

A similar issue also exists in the `remove_liquidity` process.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
034 | // Bound and round the liquidity to remove
035 | let liquidity_to_remove_u64 = args.l_to_remove;
036 | let liquidity_to_remove_i64 = liquidity_to_remove_u64 as i64;
```

3. More unsafe type conversion cases

In addition, there are 5 more unsafe type conversion instances.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
322 | let delta_l: i64 = (l_target - l_actual).floor() as i64;

/* solana/programs/exponent_clmm/src/instructions/market_three/withdraw_liquidity.rs */
```

```

143 | self.market.liquidity_net_balance_limits.verify_limits(
144 |     Clock::get()?.unix_timestamp as u32,
145 |     liquidity_balance,
146 |     -(lp_in as i64),
147 | );

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_sell_yt.rs */
021 | let sy_constraint = py_to_sy(sy_state.exchange_rate, min_base_amount).saturating_add(1) as i64;

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
773 | let l_old_i64 = *active_liquidity_u64 as i64;

/* solana/programs/exponent_clmm/src/instructions/market_three/admin/modify_farm.rs */
119 | let old_undistributed_tokens = old_remaining_time * current_farm.token_rate as i64;
127 | (new_expiration_timestamp as i64 - current_timestamp as i64) * new_rate as i64;

```

Resolution

This issue has been fixed by [3f44304](#).

EXPONENT_CLMM

[L-02] Incorrect compute_ticks_amount_in_range implementation

Identified in commit [818a6b8](#).

During `lp_position` account initialization, the program calculates the number of ticks within the LP's tick range using the `compute_ticks_amount_in_range` function. The LP position then stores `share_trackers` records for each sub tick range.

This calculation relies on `get_amount_ticks_in_range`, which contains two issues:

1. Off-by-one when counting ticks:

In `get_amount_ticks_in_range`, the tick counter is initialized to 1. The function fetches the leftmost tick index from the LP's lower tick key and then iteratively searches to the right for larger tick indices. Each discovered tick index is counted toward `counter_tick_amount`.

However, for the first discovered `counter_index`, the function does not compare it against the tick range upper key. If this first `counter_index` is already greater than or equal to the upper key, the function still continues scanning to the right, causing the final `counter_tick_amount` to be overstated by one. It is recommended to compare the first discovered `counter_index` against the tick range upper key before continuing the search.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
643 | pub fn get_amount_ticks_in_range(&mut self, lower_key: u32, upper_key: u32) -> Result<usize> {
644 |     let left_idx_optional = self.ticks_tree.get_addr(&lower_key);
645 |     let mut counter: usize = 1;
646 |     let mut counter_index = if left_idx_optional != SENTINEL {
647 |         left_idx_optional
648 |     } else {
649 |         counter += 1;
650 |         let start_key = self
651 |             .ticks_tree
652 |             .iter()
653 |             .find(|tick| *tick.0 > lower_key)
654 |             .map(|entry| *entry.0)
655 |             .unwrap_or(SENTINEL);
656 |         self.ticks_tree.get_addr(&start_key)
657 |     };
658 |
659 |     // Totally new interval
660 |     if counter_index == SENTINEL {
661 |         counter = 2;
662 |         return Ok(counter);
663 |     } else {
664 |         while let Some(successor) = self.successor_idx(counter_index) {
665 |             if self.ticks_tree.get_node(successor).key < upper_key {
```

```

666 |         counter += 1;
667 |         counter_index = successor;
668 |     } else if self.ticks_tree.get_node(successor).key == upper_key {
669 |         counter += 1;
670 |         return Ok(counter);
671 |     } else {
672 |         counter += 1; // no upper tick exist
673 |         return Ok(counter);
674 |     }
675 | }
676 | }
677 |
678 | counter += 1; // no upper tick exist
679 | return Ok(counter);
680 | }

```

2. Mismatch between tick count and share_trackers allocation:

In the actual `share_trackers` recording process, the number of `share_trackers` written is one less than the computed `tick_in_range_amount`. This is because N sub-ranges require $N + 1$ ticks to define their boundaries. As a result, the `share_trackers` array is overallocated by one entry.

It is recommended to subtract one from the value returned by `get_amount_ticks_in_range` inside `compute_ticks_amount_in_range` so that the allocated `share_trackers` count correctly matches the number of sub tick ranges.

Resolution

This issue has been fixed by `e6cfbeb`.

EXPONENT_CLMM

[L-03] create_token_account DoS

Identified in commit [818a6b8](#).

The `system_program::create_account` will fail if the new account's Lamport is not 0.

```
/* solana/libraries/token_util/src/lib.rs */
058 | pub fn create_token_account<'a>(<
059 |     authority: &AccountInfo<'a>,
060 |     payer: &AccountInfo<'a>,
061 |     token_account: &AccountInfo<'a>,
062 |     mint_account: &AccountInfo<'a>,
063 |     system_program: &AccountInfo<'a>,
064 |     token_program: &AccountInfo<'a>,
065 |     signer_seeds: &[&[u8]]],
066 | ) -> Result<> {
083 |     let lamports = Rent::get()?.minimum_balance(space);
084 |     let cpi_accounts = anchor_lang::system_program::CreateAccount {
085 |         from: payer.to_account_info(),
086 |         to: token_account.to_account_info(),
087 |     };
088 |     let cpi_context = CpiContext::new(system_program.to_account_info(), cpi_accounts);
089 |     anchor_lang::system_program::create_account(
090 |         cpi_context.with_signer(signer_seeds),
091 |         lamports,
092 |         space as u64,
093 |         token_program.key,
094 |     )?;
104 | }
```

During the `MarketThreeInit` instruction, the program creates the `escrow_yt` and `escrow_sy` token accounts using the `create_token_account` function.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/admin/market_three_init.rs */
133 | /// Generic function to create token accounts for the market
134 | fn create_market_token_account(
135 |     &self,
136 |     mint: &AccountInfo<'i>,
137 |     token_account: &AccountInfo<'i>,
138 |     seed: &[u8],
139 | ) -> Result<Pubkey> {
140 |     let (addr, bump) = Pubkey::find_program_address(&[seed, self.market.key().as_ref()], &ID);
141 |     assert_eq!(addr, token_account.key());
142 |
143 |     create_token_account(
144 |         &self.market.to_account_info(),
145 |         &self.payer.to_account_info(),
146 |         token_account,
147 |         mint,
148 |         &self.system_program.to_account_info(),
149 |         &self.token_program.to_account_info(),
150 |         &[&[seed, self.market.key().as_ref(), &bump]]],
151 |     )?;
```

```
152 |  
153 |     Ok(addr)  
154 | }
```

Both `escrow_yt` and `escrow_sy` are program-derived addresses, meaning their public keys can be deterministically precomputed. The `create_token_account` function does not handle the case where the target account already has a non-zero lamport balance.

As a result, a malicious user could precompute the `escrow_yt` or `escrow_sy` PDA and transfer `SOL` to that address in advance. This would cause the subsequent `create_account` CPI to fail, leading to a DoS condition for the `MarketThreeInit` instruction.

It is recommended to add logic in `MarketThreeInit` to handle cases where `escrow_yt` or `escrow_sy` already hold non-zero lamports.

Resolution

This issue has been fixed by [6f5044c](#).

EXPONENT_CLMM

[L-04] The floor operation on pt_in_segment leads to trade_pt failure

Identified in commit [818a6b8](#).

The `swap` function handles exact-in swaps by iterating across tick intervals, computing segment outputs, and consuming `amount_in_left`.

In the `PtToSy` branch, the `pt_in_segment` is calculated as `floor(l_trade * du_actual)`:

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
537 | let pt_in_segment = (l_trade_f64 * du_actual).floor() as u64;
```

When this value becomes `0`, the function does not reduce `amount_in_left` and may also skip the output/fee updates.

This can cause the main `while amount_in_left > 1` loop to make no progress, repeatedly executing without consuming inputs or advancing states, which

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
457 | while amount_in_left > 1 {
537 |     let pt_in_segment = (l_trade_f64 * du_actual).floor() as u64;
592 |     amount_in_left = amount_in_left.checked_sub(pt_in_segment).unwrap();
```

The potential impact is a compute-unit exhaustion caused by a non-terminating loop in a user-invoked swap. Under specific low-liquidity or near-boundary conditions, this can cause the instruction to fail and effectively block trades along the affected path:

1. Alice submits a trade with a very small `amount_in` (greater than 1) while the market is positioned near a boundary with minimal effective liquidity.
2. The swap calculation produces `pt_in_segment = 0`, meaning no input is consumed during the iteration.
3. Because no progress is made, the loop continues until the compute budget is exhausted, causing the transaction to fail and resulting in a denial of service for that trade path.

To mitigate this, consider introducing a minimal-progress guard: `if pt_in_segment == 0` (or the net output is `0`). The logic should either force a boundary crossing or exit with an explicit error to prevent a zero-progress loop.

Additionally, validating `l_trade_f64` and the effective deltas before applying `floor` would help prevent

segments that produce zero output.

```
let pt_in_segment = (l_trade_f64 * du_actual).floor() as u64;
if pt_in_segment == 0 {
    // either cross boundary or return an error
    self.cross_one_boundary_and_update_core(...)?;
    continue;
}
// proceed with output/fee updates
```

Resolution

This issue has been fixed by [923c6c8](#).

EXPONENT_CLMM

[L-05] Trade fee event misreports total fee

Identified in commit [818a6b8](#).

The `TradePt` event in `trade_pt.rs` is used to report fee metrics for swaps.

Currently, it assigns the protocol fee to `total_fee`, which omits the LP fee component. This results in on-chain event data that does not fully reflect the total fee charged from traders.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/trade_pt.rs */
306 | let event = TradePtEvent {
316 |     total_fee: trade_result.protocol_fee_charged_out_token,
317 |     treasury_fee: trade_result.protocol_fee_charged_out_token,
321 | };
427 | let event = TradePtEvent {
437 |     total_fee: trade_result.protocol_fee_charged_out_token,
438 |     treasury_fee: trade_result.protocol_fee_charged_out_token,
442 | };
```

Consider updating the event assignment to sum the LP fee and protocol fee for `total_fee`, while retaining the protocol fee for `treasury_fee`:

```
total_fee: trade_result.lp_fee_charged_out_token + trade_result.protocol_fee_charged_out_token,
treasury_fee: trade_result.protocol_fee_charged_out_token,
```

Resolution

This issue has been fixed by [f3f02fc](#).

EXPONENT_CLMM

[L-06] Inconsistent epsilon clamp check in SyToPt mode

Identified in commit [818a6b8](#).

In the `swap` function under `SyToPt` mode, if the difference between the effective target price and the effective current spot price is smaller than `epsilon_clamp`, the swap skips that part of the tick range and crosses the tick.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
613 | SwapDirection::SyToPt => {
614 |     // PT_max_left = L * (u_old - u_left)
615 |     let c_eff_left = snapshot.get_effective_price(anchor_u_left);
616 |     // k_pt = min(1, PT_rem / (PT_max_left))
617 |     let l_trade_f64 = active_liquidity_f64 * kappa;
618 |     // Final ΔC in this interval
619 |     let delta_c_by_input = (amount_in_left as f64) / (l_trade_f64);
620 |     let delta_c_to_left_boundary = c_eff_left - c_eff_old;
621 |     let delta_c_actual = delta_c_by_input.min(delta_c_to_left_boundary);
622 |
623 |     if delta_c_to_left_boundary <= eps {
624 |         // Cross boundary to the left
625 |         self.cross_one_boundary_and_update_core(
626 |             ticks,
627 |             SwapDirection::SyToPt,
628 |             current_left_boundary_index, // crossing the current lower boundary
629 |             &mut current_left_boundary_index,
630 |             &mut current_price_spot,
631 |             &mut active_liquidity_u64,
632 |             &mut active_liquidity_f64,
633 |             mutate_tick_state,
634 |             market_farm_indices,
635 |             args.modified_ticks,
636 |         )?;
637 |         continue;
638 |     }
```

After a swap segment completes, the function checks the price difference between the `left_boundary_index` tick price and the current spot price. If this difference is smaller than `epsilon_clamp`, it also skips the remaining portion of the tick range and crosses the tick.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
701 | current_price_spot = spot_price_new;
702 |
703 | // If we hit boundary, cross
704 | if (current_price_spot - anchor_u_left).abs() <= eps && amount_in_left > 0 {
705 |     self.cross_one_boundary_and_update_core(
706 |         ticks,
707 |         SwapDirection::SyToPt,
708 |         current_left_boundary_index,
709 |         &mut current_left_boundary_index,
```

```
710 |         &mut current_price_spot,  
711 |         &mut active_liquidity_u64,  
712 |         &mut active_liquidity_f64,  
713 |         mutate_tick_state,  
714 |         market_farm_indices,  
715 |         args.modified_ticks,  
716 |     )?;  
717 | }
```

However, in this post-swap `epsilon_clamp` check, the price difference is computed using the raw price, not the effective price. Since in `SyToPt` mode, the actual SY input is calculated using `liquidity * effective_price` rather than the raw price, this inconsistency can cause the `epsilon_clamp` protection to be bypassed.

It is recommended to use an effective price consistently for all `epsilon_clamp` checks in `SyToPt` mode.

Resolution

This issue has been fixed by [154f795](#).

EXPONENT_CLMM

[L-07] Missing tick cross when price hits boundary

Identified in commit 818a6b8.

In the `swap` function, after completing a swap step within a tick range, the protocol decides whether to cross the tick based on (i) the spot price is at (or within `eps` of) the tick boundary, and (ii) there is still remaining input (`amount_in_left > 0`).

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
596 | // If we hit boundary, cross
597 | if (anchor_u_right - current_price_spot) <= eps && amount_in_left > 0 {
598 |     self.cross_one_boundary_and_update_core(
599 |         ticks,
600 |         SwapDirection::PtToSy,
601 |         right_boundary_index,
602 |         &mut current_left_boundary_index,
603 |         &mut current_price_spot,
604 |         &mut active_liquidity_u64,
605 |         &mut active_liquidity_f64,
606 |         mutate_tick_state,
607 |         market_farm_indices,
608 |         args.modified_ticks,
609 |     )?;
610 | }

703 | // If we hit boundary, cross
704 | if (current_price_spot - anchor_u_left).abs() <= eps && amount_in_left > 0 {
705 |     self.cross_one_boundary_and_update_core(
706 |         ticks,
707 |         SwapDirection::SyToPt,
708 |         current_left_boundary_index,
709 |         &mut current_left_boundary_index,
710 |         &mut current_price_spot,
711 |         &mut active_liquidity_u64,
712 |         &mut active_liquidity_f64,
713 |         mutate_tick_state,
714 |         market_farm_indices,
715 |         args.modified_ticks,
716 |     )?;
717 | }

```

However, one case is missing from this logic. If the `current_price_spot` has already reached the tick boundary but the remaining input amount has been fully consumed (so `amount_in_left = 0`), the swap loop exits without crossing the tick, even though the price has already moved into the next tick range.

It's recommended to add an explicit check for whether the current spot price has reached the tick boundary. If it has, the protocol should perform a tick cross, regardless of whether any input amount remains.

Resolution

This issue has been fixed by [fab6e1c](#).

EXPONENT_CLMM

[L-08] Incorrect tick cross condition in swap_exact_out

Identified in commit [818a6b8](#).

In `swap_exact_out`, several branches determine whether the swap should cross a tick boundary. Some of these are valid optimizations for skipping ranges with no effective liquidity. However, the `net_actual == 0` branch is not an appropriate reason to force a tick cross.

```

314 | let net_actual = pt_out_gross_actual.checked_sub(total_fee_out).unwrap();
315 | if net_actual == 0 {
316 |     self.cross_one_boundary_and_update_core(
317 |         ticks,
318 |         SwapDirection::SyToPt,
319 |         current_left_boundary_index,
320 |         &mut current_left_boundary_index,
321 |         &mut current_price_spot,
322 |         &mut active_liquidity_u64,
323 |         &mut active_liquidity_f64,
324 |         mutate_tick_state,
325 |         market_farm_indices,
326 |         args.modified_ticks,
327 |     )?;
328 |     continue;
329 | }

```

When `net_actual == 0`, the entire `PT` output of the swap is collected as fees. This is still a valid swap (and is favorable to the protocol/LPs) and does not indicate that a tick cross should be triggered.

It's recommended to remove the skip logic under `net_actual == 0` condition so that the swap continues normally.

Resolution

This issue has been fixed by [1cffc3a](#).

EXPONENT_CLMM

[L-09] Incorrect SY slice traversal due to index-based start node selection

Identified in commit [818a6b8](#).

In `accrue_principal_for_deposit`, the SY-slice traversal selects its starting node via: `start_idx = price_split_tick_idx.max(lower_price_idx)`.

This is not a valid ordering primitive for a red-black tree. In an RB tree, the node "index" is an allocation handle (or internal pointer surrogate) and is not ordered by the node's key. Therefore, taking max over indices compares allocation/handle values rather than tick keys, which can cause traversal to start from an incorrect node and potentially skip intervals.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
261 | let sy_slice_start = price_split_for_need.max(price_spot_lower);
262 | let sy_slice_end = price_spot_upper;
263 | let start_idx = price_split_tick_idx.max(lower_price_idx);
264 | if sy_slice_start < sy_slice_end {
265 |     self.for_each_interval_between(
266 |         start_idx,
267 |         sy_slice_start,
268 |         sy_slice_end,
```

The `max` operation compares the node index, not the tick keys.

- For `price_split_for_need < lower`: Since L270-272 clamps with `segment_start = max(left, sy_slice_start)` and `segment_end = min(right, sy_slice_end)`, when `start_idx` lies left of `lower`, early segments clamp to `[lower, lower)` and skipped until the iterator reaches the interval whose left boundary is `lower`.
- For `lower < price_split_for_need < upper`: If `start_idx` lies left of `lower_price_idx`, the initial segments with `right split` are clamped to `[split, split)` and skipped. Traversal continues until the crossing interval `[split, right)` where actual accounting occurs and `pending_cross` is consumed.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
269 | |segment: IntervalSegment| {
270 |     let segment_start = segment.price_spot_left.max(sy_slice_start);
271 |     let segment_end = segment.price_spot_right.min(sy_slice_end);
272 |     if segment_end > segment_start {
273 |         let sy_per_l_1 = snapshot.delta_eff_price_to_sy(
274 |             1.0,
275 |             snapshot.get_effective_price(segment_start),
276 |             snapshot.get_effective_price(segment_end),
277 |         );
```

Due to per-segment price clamping, an incorrect `start_idx` generally does not cause a severe impact.

However, it leads to unnecessary traversal between the misselected index and the actual starting node.

It is recommended to determine `start_idx` based on key ordering rather than index values. For example:

```
let key_split = self.ticks_tree.get_node(price_split_tick_idx).key;
let key_lower = self.ticks_tree.get_node(lower_price_idx).key;
let start_idx = if key_split >= key_lower { price_split_tick_idx } else { lower_price_idx };
```

Resolution

This issue has been fixed by [dc73443](#).

EXPONENT_CLMM

[L-10] Undistributed emissions due to lp_share split rounding

Identified in commit [818a6b8](#).

When splitting a position's LP shares across newly inserted ticks, the implementation rounds the left-side allocation down using `floor`. This introduces a rounding discrepancy between the left and right sub-intervals. If the per-LP emission increments differ across these sub-intervals, a user may receive a slightly incorrect total amount of emissions compared to an ideal continuous split due to this discrete rounding.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
167 | let splited_full_range =
168 |     self.ticks_tree.get_node(right_index).value.spot_price - tick_spot_price; // N: old_right - old_left
169 | let current_split_range =
170 |     self.ticks_tree.get_node(splited_index).value.spot_price - tick_spot_price; // N: new_right-old_left
171 | let left_share = (principal_share.lp_share as f64
172 |     * (current_split_range / splited_full_range))
173 |     .floor() as u64;

/* solana/programs/exponent_clmm/src/utils/math.rs */
142 | // SUPPLY
143 | let new_supply_lower = ((share_supply_lower as f64) * r_len).floor() as u64;
144 | let supply_share_for_inserted = share_supply_lower.checked_sub(new_supply_lower).unwrap();
```

Applying `floor` on the left side ensures that `left_share + right_share = total`. But it introduces a systematic left-side shortfall of up to one share per split. If per-LP emission increments differ between the left and right sub-intervals, this discrete rounding can cause a user's total realized emissions to deviate slightly from the ideal proportional (continuous) split.

At the node level, the left supply is computed as `floor(total_supply * r_len)`. Aggregating position-level splits yields $\sum \text{floor}(x_i * r)$, which can be less than `floor(total_supply * r_len)`. The missing shares are effectively shifted to the right sub-interval. When the left sub-interval has a higher per-LP emission increment than the right, users collectively receive slightly less than the node proportional baseline across these sub-intervals.

Example:

Assume a node with total supply `S = 100` and a split ratio `r = 0.5`. Node-level split:

- `left = 50, right = 50`.

Two positions cross the split: `P1 = 11` and `P2 = 89` (`sum = 100`). Position-level splits:

- P1: $\text{left} = \text{floor}(11 * 0.5) = 5$, $\text{right} = 6$.
- P2: $\text{left} = \text{floor}(89 * 0.5) = 44$, $\text{right} = 45$.

Aggregated positions yield $\text{left} = 49$ and $\text{right} = 51$, whereas the node records 50 and 50 . At this point, the `principal_share_supply` value at the node may no longer exactly represent the total `lp_share` positions minted at the current tick due to rounding.

If future per-LP emission increments differ (e.g., $\text{left increment} = 3$, $\text{right increment} = 1$ due to higher SY on the left):

- For node emission: $\text{total} = 50 * 3 + 50 * 1 = 200$.
- Aggregated positions: $\text{total} = 49 * 3 + 51 * 1 = 198$.

While the loss or gain per split is bounded, repeated splits across multiple lp positions can cause these small discrepancies to accumulate. As a result, a small portion of emissions may remain undistributed and permanently retained in the pool.

It is recommended to consider representing `principal_share.lp_share` using fractional precision to eliminate rounding discrepancies during splits.

Resolution

This issue has been fixed by [00c44f9](#).

EXPONENT_CLMM

[L-11] Zero PT contribution on crossing leads to DoS

Identified in commit [818a6b8](#).

When adding liquidity across a split point, the first **PT** slice only sets **pending_cross** when **principal_pt_delta > 0**. If the segment crosses the split but the **PT** slice contributes zero due to tiny segment length or small liquidity (**principal_pt_delta == 0**), **pending_cross** is never set. The subsequent **SY** slice assumes it exists and unconditionally unwraps it, causing a panic.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
226 | let is_crossing = segment.price_spot_left < price_split_for_need
227 |     && segment.price_spot_right > price_split_for_need;
228 |
229 | let principal_pt_delta =
230 |     (position_liquidity as f64 * segment_length_u).floor() as u64;
231 |
232 | if principal_pt_delta > 0 {
233 |     if is_crossing {
234 |         // save for later resolve
235 |         pending_cross = Some((
236 |             segment.left_index,
237 |             principal_pt_delta,
238 |             segment_length_u,
239 |         ));
240 |         return Ok(());
241 |     }
```

The **SY** phase then unwraps **None**, causing the instruction to panic. The only scenario that avoids the unwrap is when **principal_sy_delta** is also **0**, in which case execution bypasses the split-crossing branch.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
281 | let is_crossing = segment.price_spot_left < price_split_for_need
282 |     && segment.price_spot_right > price_split_for_need;
283 |
284 | if principal_sy_delta > 0 {
285 |     if is_crossing {
286 |         // active tick case we deposit liquidity based on proportions of current distribution
287 |         let (pt_left_index, dpt_in, du_pt_part) =
288 |             pending_cross.take().unwrap();
289 |     }
```

This can cause panics along legitimate **add_liquidity** paths and result in a denial of service.

It is recommended to set **pending_cross** appropriately when **is_crossing** is true and use round up when calculating **principal_pt_delta** and **principal_sy_delta**.

Resolution

This issue has been fixed by [5cf4bca](#).

EXPONENT_CLMM

[L-12] swap_exact_out input accounting undercounts max_in

Identified in commit [818a6b8](#).

In the `swap_exact_out` and `swap` functions, the handling differs in extreme cases where `pt_out_net_boundary == 0` or `net_actual == 0`. In these cases, the exact-out path crosses the boundary without input consumption.

In `swap`, before crossing abnormal ticks, `amount_in_left` is deducted.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
547 | if sy_out_gross_u64 > 0 {
548 |     amount_in_left = amount_in_left.checked_sub(pt_in_segment).unwrap();
549 | }
550 |
551 | // If we hit boundary, cross
552 | if (anchor_u_right - current_price_spot) <= eps && amount_in_left > 0 {
553 |     self.cross_one_boundary_and_update_core(
554 |         ticks,
555 |         SwapDirection::PtToSy,
556 |         right_boundary_index,
557 |         &mut current_left_boundary_index,
558 |         &mut current_price_spot,
559 |         &mut active_liquidity_u64,
560 |         &mut active_liquidity_f64,
561 |         mutate_tick_state,
562 |         market_farm_indices,
563 |         args.modified_ticks,
564 |     )?;
565 | }
```

In `swap_exact_out`, boundaries are crossed directly without deducting `amount_in`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
252 | if pt_out_net_boundary == 0 {
253 |     self.cross_one_boundary_and_update_core(
254 |         ticks,
255 |         SwapDirection::SyToPt,
256 |         current_left_boundary_index,
257 |         &mut current_left_boundary_index,
258 |         &mut current_price_spot,
259 |         &mut active_liquidity_u64,
260 |         &mut active_liquidity_f64,
261 |         mutate_tick_state,
262 |         market_farm_indices,
263 |         args.modified_ticks,
264 |     )?;
265 |     continue;
266 | }
315 | if net_actual == 0 {
316 |     self.cross_one_boundary_and_update_core(
```

```

317 |         ticks,
318 |         SwapDirection::SyToPt,
319 |         current_left_boundary_index,
320 |         &mut current_left_boundary_index,
321 |         &mut current_price_spot,
322 |         &mut active_liquidity_u64,
323 |         &mut active_liquidity_f64,
324 |         mutate_tick_state,
325 |         market_farm_indices,
326 |         args.modified_ticks,
327 |     )?;
328 |     continue;
329 | }

```

In these edge cases, `swap_exact_out` can skip input and fee accounting while still advancing, which makes `amount_in_consumed` smaller than it would be in `swap`. This weakens the `max_in` constraint and can yield slightly more PT output.

It's recommended to ensure that zero-net cases in `swap_exact_out` apply fee, principal, and input accounting consistently with `swap`.

In addition, after addressing issue #66, it is recommended to update the fee, principal, and `amount_out_left` state before calling `cross_one_boundary_and_update_core` when `pt_out_net_boundary == 0` condition.

Resolution

This issue has been fixed by [41a0b8d](#).

EXPONENT_CLMM

[L-13] Incorrect pt_asset_value formula

Identified in commit [818a6b8](#).

In `principal_asset_value`, the protocol computes the position's total value as the sum of the `PT` leg value and the `SY` leg value. The `PT` leg is valued based on the CLMM effective price model implemented in `pt_asset_value`.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
529 | #[inline]
530 | pub fn principal_asset_value(
531 |     &self,
532 |     market: &MarketThree,
533 |     sy_exchange_rate: Number,
534 |     principal_pt: u64,
535 |     principal_sy: u64,
536 | ) -> Number {
537 |     if principal_pt == 0 && principal_sy == 0 {
538 |         return Number::ZERO;
539 |     }
540 |
541 |     const ONE_E18: u128 = 1_000_000_000_000_000_000;
542 |
543 |     let pt_asset_value_per_token = self.pt_asset_value(market.sec_remaining_normalized());
544 |     let pt_value_ratio =
545 |         Number::from_ratio((pt_asset_value_per_token * ONE_E18 as f64) as u128, ONE_E18);
546 |
547 |     let principal_pt_value = Number::from_natural_u64(principal_pt) * pt_value_ratio;
548 |     let principal_sy_value = Number::from_natural_u64(principal_sy) * sy_exchange_rate;
549 |
550 |     principal_pt_value + principal_sy_value
551 | }
```

However, the `pt_asset_value` comment and implementation currently assume the effective price is $C = \frac{e^{-\tau \times u}}{r}$.

Under this assumption, the asset value per `PT` (`asset_value_per_pt`) is computed as $C \times r = e^{-\tau \times u}$.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
513 | /// Calculate PT asset value based on current spot price and time to expiry
514 | ///
515 | /// Uses CLMM's effective price model: C = e^(-τ * u) / r
516 | /// At the current spot price u, 1 PT can be sold for C SY tokens
517 | /// Therefore, asset_value_per_pt = C * r = e^(-τ * u)
518 | ///
519 | /// # Arguments
520 | /// * `time_factor` - τ (tau), normalized time remaining until expiry
521 | ///
522 | /// # Returns
523 | /// Asset value of 1 PT token (not scaled by 1e18)
```

```

524 | #[inline]
525 | pub fn pt_asset_value(&self, time_factor: f64) -> f64 {
526 |     (-(time_factor * self.current_spot_price)).exp()
527 | }

```

This is inconsistent with the actual implementation of `get_effective_price`, which returns: $\frac{u^{1-\tau}}{r(\tau-1)}$.

This expression corresponds to the integral of the effective price curve. The underlying effective price used by the model is $C = \frac{u^{-\tau}}{r}$ (with the sign handling the interval direction). Therefore, the correct asset value per PT (`asset_value_per_pt`) should be $C \times r = u^{-\tau}$.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
034 | #[inline]
035 | pub fn get_effective_price(&self, u: f64) -> f64 {
036 |     assert!(self.time_factor != 1.0);
037 |     return u.powf(-(self.time_factor) + 1.0)
038 |         / ((self.sy_exchange_rate) * (self.time_factor - 1.0));
039 | }

```

Using the incorrect exponential formula in `pt_asset_value` will lead to an incorrect `principal_asset_value`, which in turn affects every instruction that relies on `principal_asset_value` to compute a position's value.

It's recommended to update the PT asset value formula in `pt_asset_value` to `self.current_spot_price.powf(-time_factor)`.

Resolution

This issue has been fixed by [ee8929a](#).

EXPONENT_CLMM

[L-14] Hardcoded parameters in mock liquidity calculation

Identified in commit [818a6b8](#).

In the `wrapper_provide_liquidity` instruction, the program performs a "mock" calculation to estimate how much liquidity a user can add within the specified tick range based on their provided base-token amount.

The `compute_token_needs` function is used to compute the expected liquidity amount. However, the `max_x_sy` and `max_pt` parameters passed into `compute_token_needs` are currently hardcoded to `MOCK_AMOUNT`, which is a fixed value of `1e9`. As a result, the computed liquidity amount does not reflect the actual liquidity amount implied by their base-token input.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs */
231 |
232 | let (sy_mock, pt_mock) = compute_token_needs(
233 |     snap,
234 |     spot_price_current,
235 |     price_eff_lower,
236 |     price_eff_upper,
237 |     lower_price,
238 |     upper_price,
239 |     MOCK_AMOUNT,
240 |     MOCK_AMOUNT,
241 |     ctx.accounts.market.configuration_options.epsilon_clamp,
242 | );
```

It's recommended to replace the hard-coded `max_sy` and `max_pt` values in `compute_token_needs` with user-input parameters.

Resolution

This issue has been fixed by [72c7783](#).

EXPONENT_CLMM

[L-15] Minimum node liquidity check bypass

Identified in commit [818a6b8](#).

In the `add_liquidity` instruction, the `tick_add_liquidity_net_gross` function enforces a minimum liquidity requirement, ensuring that a node's liquidity must be at least `min_l_to_activate`.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */  
280 | node.liquidity_gross  
281 |     .checked_sub(min_l_to_activate)  
282 |     .expect("not enough liquidity for tick to be initialized");
```

However, this check can be easily bypassed. The `withdraw_liquidity` instruction does not enforce any minimum liquidity constraint, so a user can add liquidity above `min_l_to_activate` and then immediately withdraw most of it, leaving the node with liquidity below `min_l_to_activate`.

It's recommended to deduct `min_l_to_activate` from the user's LP amount when initializing a new tick, freeze that portion into the newly created tick range.

Resolution

This issue has been fixed by [bdbf523](#).

EXPONENT_CLMM

[L-16] Extra tick crossing due to inconsistent swap loop conditions

Identified in commit [818a6b8](#).

In both the `swap` and `swap_exact_out` functions, the termination conditions of the `while` loop differ from the conditions that trigger the final boundary crossing inside the loop:

- In `swap_exact_out`, the loop continues while `amount_out_left > 2`, yet a boundary crossing may still be triggered when `amount_out_left > 0`.
- The exact in `swap` has the same pattern: the loop guard is `amount_in_left > 1`, but a boundary crossing can occur as long as `amount_in_left > 0`.

As a result, the implementation can cross a tick boundary and advance the current price even when no further swap execution is possible. This creates a scenario where the price state advances by one additional interval beyond what the loop guard is intended to permit.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
099 | while amount_out_left > 2 {
100 |     let anchor_u_left = ticks.get_spot_price(current_left_boundary_index);
101 |     let c_eff_old = snapshot.get_effective_price(current_price_spot);
102 |     let c_eff_left = snapshot.get_effective_price(anchor_u_left);
370 | if amount_out_left > 0
371 |     && ((spot_price_new - anchor_u_left).abs() <= eps
372 |         || pt_out_gross_actual == pt_out_gross_boundary_u64)
373 | {
374 |     self.cross_one_boundary_and_update_core(

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
457 | while amount_in_left > 1 {
458 |     // Right boundary of current interval (must exist to have a half-open [left, right))
459 |     let right_boundary_index_optional = ticks.successor_idx(current_left_boundary_index);
456 | // If we hit boundary, cross
457 | if (anchor_u_right - current_price_spot) <= eps && amount_in_left > 0 {
458 |     self.cross_one_boundary_and_update_core(
459 |         ticks,
```

This inconsistency may introduce a small price discrepancy when `amount_out_left` equals `1` or `2` (or `amount_in_left` equals `1`). As a result, subsequent user operations may observe a `current_spot_price` and `tick` state that do not correspond to the actual end of the swap, but instead reflect an additional advancement by one interval.

It is recommended to align the condition used to advance to the next tick interval at the end of the while loop with the condition used at the start of the while loop.

Resolution

This issue has been fixed by [07eb822](#).

EXPONENT_CLMM

[L-17] Unclaimable CrossingSplit rewards due to mismatched domain

Identified in commit [fc070e9](#).

After `CrossingSplit` was introduced, farm reward accounting and farm reward claiming no longer use the same storage path.

During accrual, a crossing position writes farm rewards into the segmented trackers `crossing_split.farms_a/b/c`. During claim, however, the program reads only the legacy root tracker's `lp_position.farms`.

As a result, rewards accrued while `crossing_split` is active are not claimable through `claim_farm_emissions`, and they can be silently lost when `crossing_split` is overwritten or cleared.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
338 | pub fn accrue_fees_farms_for_position(...)
343 |     if pos.has_crossing_split() {
344 |         self.accrue_fees_farms_for_position_segmented(pos, market_farm_indices)

369 | fn accrue_fees_farms_for_position_segmented(...)
384 |     split.farms_a.ensure_trackers_and_earn_all(&inside_a, l_target);
392 |     split.farms_b.ensure_trackers_and_earn_all(&inside_b, l_actual);
400 |     split.farms_c.ensure_trackers_and_earn_all(&inside_c, l_target);

/* solana/programs/exponent_clmm/src/instructions/market_three/claim_farm_emissions.rs */
74 | let amount_to_send = amount.to_u64(ctx.accounts.lp_position.farms.trackers[index].staged)?;
83 | ctx.accounts.lp_position.farms.trackers[index].dec_staged(amount_to_send);
```

1. One loss path occurs when crossing_split is overwritten.

After `accrue_fees_farms_for_position` accrues rewards into the current split trackers, the code rebuilds and overwrites `position.crossing_split` in the `accrue_result.crossing_info` branch. Because the new split is reinitialized from current indices, previously staged rewards in `crossing_split.farms_a/b/c` are not carried forward and can be silently discarded.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
386 | let crossing_split = create_crossing_split(
387 |     ticks,
388 |     lower_tick_idx,
389 |     upper_tick_idx,
390 |     crossing_info,
391 |     market_farm_indices,
392 | );
393 | position.crossing_split = crossing_split;
```

2. Another loss path occurs when crossing_split is cleared.

The split trackers are never merged back into `lp_position.farms` before claim, and there are code paths that clear the split state without migrating staged split rewards.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
230 | if is_full_removal {
231 |     // Clear crossing split on full removal
232 |     position.crossing_split = crate::CrossingSplit::inactive();
233 |     return;
234 | }

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
397 | // No crossing in this add - clear any existing crossing_split
398 | position.crossing_split = CrossingSplit::inactive();
```

This leads to three outcomes:

1. Unclaimable farm rewards: users can accrue farm rewards in crossing mode but receive zero from `claim_farm_emissions`.
2. Silent reward loss: staged rewards in split trackers can be dropped when `crossing_split` is reset to inactive.
3. State/reporting inconsistency: events and indexers using `lp_position.farms` under-report rewards for crossing positions.

To address this, add a merge step that moves rewards from `crossing_split.farms_a/b/c` into `lp_position.farms` before claim and before any path that clears split state.

Resolution

This issue has been fixed by `56f64d2`.

EXPONENT_CLMM

[L-18] Equalization calculation ignored in liquidity deposit

Identified in commit [7bc61ee](#).

In `wrapper_provide_liquidity_classic`, after converting the user provided `amount_base` into `SY` tokens, the instruction calls `compute_existing_position_budget_effect` to estimate the additional `SY` and `PT` required for equalization (crossing split).

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs */
109 | let (sy_exchange_rate, sy_out_amount) = if amount_base != 0 {
110 |     let mint_sy_return_data = sy_cpi::cpi_mint_sy(
111 |         ctx.accounts.market.sy_program,
112 |         amount_base,
113 |         mint_sy_rem_accounts,
114 |         mint_sy_rem_accounts.to_vec().to_account metas(None),
115 |     )?;
116 |
117 |     (mint_sy_return_data.exchange_rate, mint_sy_return_data.sy_out_amount)
118 | } else {
119 |     (Number::ZERO, 0)
120 | };
```

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs */
122 | let (_, equalization_precalc) =
123 |     with_ticks_snapshot(&ctx.accounts.ticks, |ticks_snapshot| {
124 |         load_existing_position_precalc(ticks_snapshot, &ctx.accounts.lp_position)
125 |     })?;
126 | let deposit_budget_effect =
127 |     compute_existing_position_budget_effect(&equalization_precalc, sy_out_amount, amount_pt)?;

/* solana/programs/exponent_clmm/src/instructions/wrappers/add_liquidity_precalc.rs */
050 | pub(crate) fn compute_existing_position_budget_effect(
051 |     precalc: &ExistingPositionAddLiquidityPrecalc,
052 |     user_max_sy: u64,
053 |     user_max_pt: u64,
054 | ) -> Result<ExistingPositionBudgetEffect> {
055 |     Ok(ExistingPositionBudgetEffect {
056 |         user_max_sy,
057 |         user_max_pt,
058 |         effective_max_sy: precalc.effective_max_sy(user_max_sy)?,
059 |         effective_max_pt: precalc.effective_max_pt(user_max_pt)?,
060 |         fixed_sy_spent: precalc.sy_spent(),
061 |         fixed_pt_spent: precalc.pt_spent(),
062 |         fixed_sy_released: precalc.sy_released(),
063 |         fixed_pt_released: precalc.pt_released(),
064 |     })
065 | }
```

However, during the actual liquidity addition, the `do_cpi_deposit_liquidity_idempotent` call still uses the original `SY` and `PT` amounts, without incorporating the additional amounts computed for equalization. As a result, the output of `compute_existing_position_budget_effect` is effectively unused and redundant.

```

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs */
129 | let deposit_liquidity_return_data = self_cpi::do_cpi_deposit_liquidity_idempotent(
130 |     ctx.accounts.to_deposit_liquidity_accounts(),
131 |     interface_cpi_rem_accounts,
132 |     lower_tick,
133 |     upper_tick,
134 |     deposit_budget_effect.user_max_sy,
135 |     deposit_budget_effect.user_max_pt,
136 |     &ctx.accounts.rent,
137 | )?;

```

This also means the final **PT** amount used for liquidity provisioning may not match the intended **amount_pt** specified in the instruction.

It is recommended to remove the redundant **compute_existing_position_budget_effect** calculation and pass **sy_out_amount** and **amount_pt** directly as parameters.

Resolution

This issue has been fixed by [18f35d1](#).

EXPONENT_CLMM

[L-19] Missing cap on secant_step in pt_heavy path

Identified in commit [a8ede4d](#).

In `next_to_strip_one_sided_pt_heavy`, the function estimates the `SY` token amount to be stripped(`next`) in the next iteration based on historical iteration data. For the `pt_heavy` side, it handles the case where the planned `SY` strip amount is too high and therefore needs to be reduced.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs */
551 | fn next_to_strip_one_sided_pt_heavy(
552 |     args: &WrapperPrecalcArgs,
553 |     eval: &StripEvaluation,
554 |     to_strip: u64,
555 |     gradient_delta: i64,
556 |     gradient_step: u64,
557 |     prev_one_sided_pt_heavy: Option<(u64, u64)>,
558 | ) -> u64 {
559 |     let next = prev_one_sided_pt_heavy.and_then(|(prev_to_strip, prev_pt_excess)| {
560 |         let strip_delta = prev_to_strip.checked_sub(to_strip)?;
561 |         let excess_delta = prev_pt_excess.checked_sub(eval.pt_excess)?;
562 |         let secant_step =
563 |             checked_secant_step_u64(eval.pt_excess, strip_delta, excess_delta, gradient_step)?;
564 |         to_strip.checked_sub(secant_step)
565 |     });
566 |
567 |     next.unwrap_or_else(|| apply_signed_to_strip_delta(to_strip, gradient_delta, args.total_sy))
568 | }
```

After computing the amount to reduce via `checked_secant_step_u64`, the function subtracts this `secant_step` value from the total planned strip amount, `to_strip`.

However, based on the implementation of `checked_secant_step_u64`, `secant_step` may be greater than `to_strip`, and the subsequent `checked_sub` operation does not include any protection against this case. If the computed `secant_step` exceeds `to_strip`, the resulting `next` value becomes `None` and the logic ultimately falls through to `apply_signed_to_strip_delta`, causing the iteration to lose its gradient precision. It may stop at a suboptimal `to_strip`, resulting in a non optimal solution.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs */
507 | fn checked_secant_step_u64(
508 |     current_excess: u64,
509 |     strip_delta: u64,
510 |     excess_delta: u64,
511 |     gradient_step: u64,
512 | ) -> Option<u64> {
513 |     if strip_delta == 0 || excess_delta == 0 {
514 |         return None;
515 |     }
```

```

516 |
517 |     let raw_secant_step = checked_div_ceil_u128(
518 |         (current_excess as u128).checked_mul(strip_delta as u128)?,
519 |         excess_delta as u128,
520 |     )?;
521 |     let capped_secant_step =
522 |         raw_secant_step.min((gradient_step.checked_mul(4)? as u128));
523 |     let secant_step = u64::try_from(capped_secant_step).ok()?;
524 |
525 |     Some(secant_step.max(1))
526 | }

```

By contrast, the functionally similar `next_to_strip_one_sided_sy_heavy` function already caps the computed `secant_step` at an upper bound.

```

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs */
529 | fn next_to_strip_one_sided_sy_heavy(
530 |     args: &WrapperPrecalcArgs,
531 |     eval: &StripEvaluation,
532 |     to_strip: u64,
533 |     gradient_delta: i64,
534 |     gradient_step: u64,
535 |     prev_one_sided_sy_heavy: Option<(u64, u64)>,
536 | ) -> u64 {
537 |     let next = prev_one_sided_sy_heavy.and_then(|| (prev_to_strip, prev_sy_excess)| {
538 |         let strip_delta = to_strip.checked_sub(prev_to_strip)?;
539 |         let excess_delta = prev_sy_excess.checked_sub(eval.sy_excess)?;
540 |         let secant_step =
541 |             checked_secant_step_u64(eval.sy_excess, strip_delta, excess_delta, gradient_step)?;
542 |         let max_forward_step = args.total_sy.checked_sub(to_strip)?;
543 |         let applied_step = secant_step.min(max_forward_step);
544 |         to_strip.checked_add(applied_step)
545 |     });
546 |
547 |     next.unwrap_or_else(|| apply_signed_to_strip_delta(to_strip, gradient_delta, args.total_sy))
548 | }

```

It is recommended to add a `secant_step.min(to_strip)` cap in `next_to_strip_one_sided_pt_heavy`.

Resolution

This issue has been fixed by [6284b7d](#).

EXPONENT_CLMM

[I-01] Missing alignment check in emissions tracker update

Identified in commit [f2bfe37](#).

The `update_emissions_from_position_state` function updates `market.emissions` using the provided `position_state`.

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
397 | pub fn update_emissions_from_position_state(
398 |     &mut self,
399 |     position_state: &PositionState,
400 |     lp_staked: u64,
401 | ) {
402 |     for (index, current_position) in position_state.emissions.iter().enumerate() {
403 |         let difference =
404 |             current_position.amount_claimable - self.emissions.trackers[index].last_seen_staged;
405 |
406 |         let amount_to_increase = Number::from_natural_u64(difference)
407 |             .checked_div(&Number::from_natural_u64(lp_staked))
408 |             .unwrap_or(Number::ZERO);
409 |
410 |         self.emissions.trackers[index].lp_share_index += amount_to_increase;
411 |
412 |         self.emissions.trackers[index].last_seen_staged = current_position.amount_claimable;
413 |     }
414 | }
```

At line 404, the code directly accesses `self.emissions.trackers` using the same index as `position_state.emissions`. This operation implicitly assumes alignment between `position_state` and `market.emissions.trackers`. However, `position_state` originates from the `do_get_position_state` CPI call:

```
/* solana/programs/exponent_clmm/src/instructions/market_three/deposit_liquidity.rs */
157 | let position_state = do_get_position_state(
158 |     &ctx.accounts.address_lookup_table,
159 |     &ctx.accounts.market.cpi_sy_accounts,
160 |     ctx.remaining_accounts,
161 |     ctx.accounts.sy_program.key(),
162 |     [&ctx.accounts.market.signer_seeds()],
163 | )?;
171 | ctx.accounts
172 |     .market
173 |     .update_emissions_from_position_state(&position_state, sy_balance);
```

Since `position_state.emissions` depends solely on `SY_PROGRAM` management, while `market.emissions.trackers` is maintained by the `CLMM` admin, potential misalignment may occur. For instance, `SY_PROGRAM` might omit an emission that the `CLMM` admin added. This inconsistency could trigger incorrect updates within `update_emissions_from_position_state`, leading to functional disruptions in the CLMM.

It's recommended to explicitly verify `position_state.emissions.len() == self.emissions.trackers.len()` in the `update_emissions_from_position_state` function.

Resolution

This issue has been fixed by `ac6091f`.

EXPONENT_CLMM

[I-02] Inconsistent MarketThree::size_of

Identified in commit [f2bfe37](#).

1. Incorrect ConfigurationOptions::SIZE_OF

In `market_three.rs`, the `ConfigurationOptions::SIZE_OF` is defined to represent the size of the `ConfigurationOptions`.

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
144 | impl ConfigurationOptions {
145 |     pub const SIZE_OF: usize =
154 |         // max_lp_supply
155 |         16 +
160 | }
```

It currently assumes the `max_lp_supply` occupies 16 bytes even though the field is declared as a `u64`, which should be 8 bytes instead.

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
137 |     pub tick_space: u32,
142 | }
143 |
144 | impl ConfigurationOptions {
145 |     pub const SIZE_OF: usize =
156 |         // tick_space
157 |         2 +
160 | }
```

In addition, the `tick_space: u32` is marked as 2 bytes, which should be 4 instead.

2. MarketThree::size_of does not account for the is_current_flash_swap field

The `MarketThree::size_of` function is defined to calculate the size of the market PDA accounts.

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
190 | pub fn size_of(
191 |     cpi_accounts: &CpiAccounts,
192 |     cpi_core_accounts: &CpiCoreAccounts,
193 |     emissions_len: usize,
194 |     farm_emissions_len: usize,
195 | ) -> usize {
273 |     cpi_core_accounts_size +
274 |
275 |     // liquidity_net_balance_limits
276 |     LiquidityNetBalanceLimits::SIZE_OF +
280 | }
```

```

/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
029 | #[derive(Default)]
030 | #[account]
031 | pub struct MarketThree {
032 |     /// Record of CPI core accounts
033 |     pub cpi_core_accounts: CpiCoreAccounts,
034 |
035 |     pub is_current_flash_swap: bool,
036 |
037 |     pub liquidity_net_balance_limits: LiquidityNetBalanceLimits,
038 | }

```

However, it does not include the size for the `is_current_flash_swap: bool` field.

Since the previously mentioned `MarketThree::size_of` method overestimates the account size, this issue won't lead to insufficient space failures.

It is recommended to modify the `MarketThree::size_of` method to include the space allocation for the `is_current_flash_swap` field.

Resolution

This issue has been fixed by [3f090b9](#).

EXPONENT_CLMM

[I-03] Inaccurate tick key ordering check

Identified in commit [818a6b8](#).

In the `DepositLiquidity` instruction, the validation of `lower_tick_key` and `upper_tick_key` is inaccurate. It only checks that `lower_tick_key != upper_tick_key`, but it should enforce that `lower_tick_key < upper_tick_key`.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/deposit_liquidity.rs */
135 | #[access_control(ctx.accounts.validate())]
136 | pub fn handler<'info>(
137 |     ctx: Context<'_, '_, '_, 'info, DepositLiquidity<'info>>,
138 |     lower_tick_key: u32,
139 |     upper_tick_key: u32,
140 |     max_sy_in: u64,
141 |     max_pt_in: u64,
142 | ) -> Result<DepositLiquidityEvent> {
143 |     require!(
144 |         lower_tick_key != upper_tick_key,
145 |         ExponentCLMMError::InvalidTickRange
146 |     );
```

It is recommended to update the check to require `lower_tick_key < upper_tick_key`.

Resolution

This issue has been fixed by [170fc42](#).

EXPONENT_CLMM

[I-04] Redundant successor lookup

Identified in commit 818a6b8.

In the `swap` function, the `ticks.successor_idx(current_left_boundary_index)` is called in line 459 and is subsequently invoked with the same argument again in line 481. This results in redundant tree traversal on every loop iteration, introducing unnecessary overhead.

It's recommended to use the previously computed successor index instead of calling `successor_idx` a second time. The unwrap logic can be refactored to consume the already obtained value directly. Consider changing

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
459 | let right_boundary_index_optional = ticks.successor_idx(current_left_boundary_index);
461 | let right_boundary_index = if right_boundary_index_optional.is_none() {
480 | } else {
481 |     ticks.successor_idx(current_left_boundary_index).unwrap()
482 | };
```

to:

```
let right_boundary_index_optional = ticks.successor_idx(current_left_boundary_index);
let right_boundary_index = if let Some(idx) = right_boundary_index_optional {
    idx
} else {
    // ...
};
```

Resolution

This issue has been fixed by 14512e8.

EXPONENT_CLMM

[I-05] Unsafe account closure procedure

Identified in commit [818a6b8](#).

In `close_market.rs`, the account close flow transfers remaining lamports out of `market` and `ticks`. However, it does not reassign account ownership to the system program and resize the data to zero.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/admin/close_market.rs */
051 | //close accounts and withdraw rent
052 | let market_lamports = market.to_account_info().lamports();
053 | **ctx.accounts.payer.to_account_info().lamports.borrow_mut() += market_lamports;
054 | **market.to_account_info().lamports.borrow_mut() = 0;
055 |
056 | let ticks_lamports = ticks.to_account_info().lamports();
057 | **ctx.accounts.payer.to_account_info().lamports.borrow_mut() += ticks_lamports;
058 | **ticks.to_account_info().lamports.borrow_mut() = 0;
```

This is identical to the unsafe example shown in the Anchor Sealevel Attack [9-closing-accounts/insecure-still](#).

It's recommended to follow [the account closure procedure in Anchor](#) instead.

Resolution

The team acknowledged this finding. They clarified that this instruction was only used for testing and will be removed later for production.

EXPONENT_CLMM

[I-06] Rounding causes zero mint with principal loss

Identified in commit [818a6b8](#).

In tick-level minting, the calculation performed by the function `mint_shares_for_tick_prestate_calculated` involves rounding down, which can produce a result of `minted == 0` while still reporting a non-zero amount of principal consumed. The caller then records the principal via `add_principal_pt/add_principal_sy`, but there is no corresponding increase to the `principal_share_supply` or position shares.

Single-sided:

- PT-only: `minted = floor(supply * dpt_in / pt_before)`, `used_pt = dpt_in`.
- SY-only: `minted = floor(supply * dsy_in / sy_before)`, `used_sy = dsy_in`.
- If `supply * dpt_in < pt_before` or `supply * dsy_in < sy_before`, `minted == 0` but `used > 0`. The function then skips minting, while the caller still adds principal.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
594 | } else {
595 |     minted = supply
596 |         .checked_mul(dpt_in)
597 |         .unwrap()
598 |         .checked_div(pt_before)
599 |         .unwrap();
600 | }

608 | } else {
609 |     minted = supply
610 |         .checked_mul(dsy_in)
611 |         .unwrap()
612 |         .checked_div(sy_before)
613 |         .unwrap();
614 | }
```

Double-sided:

- `lp_tokens_out` is rounded down to u64. In the `lp_from_pt < lp_from_sy` branch it sets `pt_in = intent_pt.to_u64()` even when `lp_tokens_out.to_u64() == 0`. In the other branch, `sy_in` can be > 0 while `lp_tokens_out == 0`.
- As a result, `minted == 0` with `used_pt > 0` or `used_sy > 0` can also occur in this path.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
580 | let add_result = exponent_time_curve::math::add_liquidity::(<f64>(
581 |     dsy_in, dpt_in, supply, sy_before, pt_before,
582 | ));
```

```

583 |     used_pt = add_result.pt_in;
584 |     used_sy = add_result.sy_in;
585 |     minted = add_result.lp_tokens_out;
586 | }

/* solana/libraries/time_curve/src/math.rs */
144 | let lp_from_pt = market_total_lp * intent_pt / market_total_pt;
145 | let lp_from_sy = market_total_lp * intent_sy / market_total_sy;
147 | if lp_from_pt < lp_from_sy {
148 |     let lp_tokens_out = lp_from_pt;
149 |     let pt_in = intent_pt.to_u64();
150 |     let sy_in = ((market_total_sy * lp_tokens_out + market_total_lp - N::one())
151 |                 / market_total_lp)
152 |                 .to_u64();
154 |     AddLiquidityResult {
155 |         lp_tokens_out: lp_tokens_out.to_u64(),
156 |         sy_in,
157 |         pt_in,
158 |     }
159 | } else {
160 |     let lp_tokens_out = lp_from_sy;
161 |     let sy_in = intent_sy.to_u64();
162 |     let pt_in = ((market_total_pt * lp_tokens_out + market_total_lp - N::one())
163 |                 / market_total_lp)
164 |                 .to_u64();
166 |     AddLiquidityResult {
167 |         lp_tokens_out: lp_tokens_out.to_u64(),
168 |         sy_in,
169 |         pt_in,
170 |     }
171 | }

```

It is recommended that minted/used consistency is enforced at tick-minting. If `minted` is equal to zero, return `(0, 0, 0)` and only record the principal when some shares are minted.

Resolution

This issue has been fixed by [56ce3c1](#).

EXPONENT_CLMM

[I-07] Dead liquidity from epsilon clamp

Identified in commit [818a6b8](#).

In the `compute_liquidity_target_and_token_needs` function, the liquidity amount is calculated based on the difference between `upper_price` and `lower_price`. If this price difference is smaller than `epsilon_clamp`, the function uses `epsilon_clamp` instead of the actual price difference for the calculation. However, the liquidity is still assigned to the actual tick range defined by `upper_price` and `lower_price`.

```

/* solana/programs/exponent_clmm/src/utils/math.rs */
038 | pub fn compute_liquidity_target_and_token_needs(
039 |     snap: EffSnap,
040 |     spot_price_current: f64,
041 |     price_eff_lower: f64,
042 |     price_eff_upper: f64,
043 |     lower_price: f64,
044 |     upper_price: f64,
045 |     lower_tick_idx: u32,
046 |     upper_tick_idx: u32,
047 |     current_index: u32,
048 |     max_sy: u64,
049 |     max_pt: u64,
050 |     epsilon_clamp: f64,
051 | ) -> LiquidityNeeds {
052 |     // Decide region and compute target ΔL and token needs under A-anchored formulas
053 |     if spot_price_current <= lower_price {
054 |         // Below range: SY only
055 |         let delta_c_total = (price_eff_lower - price_eff_upper).max(epsilon_clamp);
056 |         let liquidity_from_sy = (max_sy as f64) / delta_c_total;
057 |         let liquidity_target = liquidity_from_sy;
058 |         let sy_need = (liquidity_target) * delta_c_total;
059 |
060 |         return LiquidityNeeds {
061 |             liquidity_target: liquidity_target.floor() as u64,
062 |             sy_needed: sy_need as u64,
063 |             pt_needed: 0,
064 |             price_split_for_need: lower_price,
065 |             price_split_tick_idx: lower_tick_idx,
066 |         };
067 |     } else if spot_price_current >= upper_price {
068 |         // Above range: PT only
069 |         let du_total = (upper_price - lower_price).max(epsilon_clamp);
070 |         let liquidity_from_pt = (max_pt as f64) / du_total;
071 |         let liquidity_target = liquidity_from_pt;
072 |         let pt_need = liquidity_target * du_total;
073 |
074 |         return LiquidityNeeds {
075 |             liquidity_target: liquidity_target.floor() as u64,
076 |             sy_needed: 0,
077 |             pt_needed: pt_need.floor() as u64,
078 |             price_split_for_need: upper_price,
079 |             price_split_tick_idx: upper_tick_idx,
080 |         };

```

```

081 |     } else {
082 |         // Inside range: both sides
083 |         let du_left = (spot_price_current - lower_price).max(epsilon_clamp);
084 |         let price_eff_current = snap.get_effective_price(spot_price_current);
085 |         let delta_c_right = (price_eff_current - price_eff_upper).max(epsilon_clamp);
086 |
087 |         let liquidity_from_pt = (max_pt as f64) / du_left;
088 |         let liquidity_from_sy = (max_sy as f64) / delta_c_right;
089 |
090 |         let liquidity_target = liquidity_from_pt.min(liquidity_from_sy);
091 |         let pt_need = liquidity_target * du_left;
092 |         let sy_need = liquidity_target * delta_c_right;
093 |
094 |         return LiquidityNeeds {
095 |             liquidity_target: liquidity_target.floor() as u64,
096 |             sy_needed: sy_need.floor() as u64,
097 |             pt_needed: pt_need.floor() as u64,
098 |             price_split_for_need: spot_price_current,
099 |             price_split_tick_idx: current_index,
100 |         };
101 |     }
102 | }

```

During actual swap execution, if the price difference between the current spot price and a tick boundary is smaller than `epsilon_clamp`, that range is skipped and the swap crosses the tick. This means that if a liquidity tick range is shorter than `epsilon_clamp`, that liquidity will never participate in swaps, effectively becoming "dead" liquidity.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
518 | if du_to_boundary <= self.configuration_options.epsilon_clamp {
519 |     // Cross boundary to unlock the next interval
520 |     // msg!("cross boundaries: {}, {}, {}", du_by_principal, du_by_input, du_to_boundary);
521 |     self.cross_one_boundary_and_update_core(
522 |         ticks,
523 |         SwapDirection::PtToSy,
524 |         right_boundary_index,
525 |         &mut current_left_boundary_index,
526 |         &mut current_price_spot,
527 |         &mut active_liquidity_u64,
528 |         &mut active_liquidity_f64,
529 |         mutate_tick_state,
530 |         market_farm_indices,
531 |         args.modified_ticks,
532 |     )?;
533 |     continue;
534 | }

623 | if delta_c_to_left_boundary <= eps {
624 |     // Cross boundary to the left
625 |     self.cross_one_boundary_and_update_core(
626 |         ticks,
627 |         SwapDirection::SyToPt,
628 |         current_left_boundary_index, // crossing the current lower boundary
629 |         &mut current_left_boundary_index,
630 |         &mut current_price_spot,
631 |         &mut active_liquidity_u64,

```

```
632 |         &mut active_liquidity_f64,  
633 |         mutate_tick_state,  
634 |         market_farm_indices,  
635 |         args.modified_ticks,  
636 |     )?;  
637 |     continue;  
638 | }
```

Instead of using `max(price_diff, epsilon_clamp)`, add a validation that prevents LPs from creating tick ranges whose price width is smaller than `epsilon_clamp`. This avoids creating liquidity ranges that can never be utilized.

Resolution

This issue has been fixed by [5b4d0f4](#).

EXPONENT_CLMM

[I-08] Unchecked liquidity_to_remove in remove_liquidity

Identified in commit 818a6b8.

In the `remove_liquidity` function, there is no validation to ensure that the requested removal amount (`liquidity_to_remove`) does not exceed the position's LP balance.

Specifically, `remove_liquidity` computes `burn_shares` as `my_shares * liquidity_to_remove / position.lp_balance`, and later subtracts this value from `share.lp_share` using `checked_sub().unwrap()`. If `liquidity_to_remove` is greater than `position.lp_balance`, `burn_shares` will exceed `my_shares`, causing the subtraction to panic via `unwrap`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
035 | let liquidity_to_remove_u64 = args.liquidity_to_remove;
036 | let liquidity_to_remove_i64 = liquidity_to_remove_u64 as i64;
```

It is therefore recommended to add explicit validation to reject zero or excessive removal requests and to prevent potential `u64` to `i64` overflow. For example, it's better to use `require!` checks and fallible conversions instead of unchecked `as` casts.

Resolution

This issue has been fixed by 165700e.

EXPONENT_CLMM

[I-09] Inaccurate PT balance check in sell_yt

Identified in commit [818a6b8](#).

In the `sell_yt` instruction, the user borrows some `PT` from the protocol, combines it with their own `YT`, and merges them into `SY`. That `SY` is then swapped (exact out) back into the previously borrowed amount of `PT` via the CLMM. So the borrowed `PT` can be repaid. This flow allows users to effectively swap `YT` into `SY`.

Because this instruction borrows `PT` and then swaps `SY` to obtain exactly that same amount of `PT`, the code includes a check requiring: `market.financials.pt_balance >= yt_in * 2 + 2`. Here, `market.financials.pt_balance` represents the total `PT` held by the CLMM, and `yt_in` represents the target borrowed `PT` amount.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/sell_yt.rs */
197 | // The market must have at least twice the amount of PT as YT in order to sell YT
198 | // This because the trader borrows the YT amount from the market
199 | // And then buys the YT amount back from the market
200 | // So they take 2x the YT amount of PT from the market
201 | assert!(
202 |     ctx.accounts.market.financials.pt_balance >= yt_in * 2 + 2,
203 |     "insufficient PT liquidity in the market"
204 | );
```

However, this check is inaccurate. The `market.financials.pt_balance` includes not only `PT` that is actually swappable within LP positions, but also `PT` collected as swap fees and held by the LP position. Because those fee `PT` tokens are not available for swapping, the `market.financials.pt_balance` overstates the amount of `PT` that can actually be used to repay the borrowed `PT`.

Since this check does not accurately reflect the usable `PT` liquidity and is not strictly necessary for the correctness of the instruction, it could be removed.

Resolution

This issue has been fixed by [a34fb0c](#).

EXPONENT_CLMM

[I-10] Incorrect amount_pt_in event parameter

Identified in commit [818a6b8](#).

In the `wrapper_provide_liquidity_classic` instruction, the `event.amount_pt_in` field is set incorrectly. It uses the user's expected maximum `PT` input instead of the actual `PT` amount consumed by the `deposit_liquidity` instruction.

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs */
106 | let deposit_liquidity_return_data = self.cpi::do_cpi_deposit_liquidity(
107 |     ctx.accounts.to_deposit_liquidity_accounts(),
108 |     interface_cpi_rem_accounts,
109 |     lower_tick,
110 |     upper_tick,
111 |     mint_sy_return_data.sy_out_amount,
112 |     amount_pt,
113 | )?;

132 | let event = WrapperProvideLiquidityClassicEvent {
133 |     user_address: *ctx.accounts.depositor.key,
134 |     market_address: ctx.accounts.market.key(),
135 |     amount_base_in: amount_base,
136 |     amount_lp_out: deposit_liquidity_return_data.delta_liquidity,
137 |     amount_pt_in: amount_pt,
138 |     asset_value_lp_in,
139 | };
```

It's recommended to set `amount_pt_in` to `deposit_liquidity_return_data.amount_pt_in` so the event reflects the true amount used.

Resolution

This issue has been fixed by [80e7241](#).

EXPONENT_CLMM

[I-11] Boundary condition mismatches in liquidity and fee checks

Identified in commit [818a6b8](#).

In the liquidity and configuration wrappers, several boundary checks use strict inequalities that incorrectly reject valid edge-case values. These checks should allow equality when the parameter represents an acceptable limit rather than an exclusive bound.

1. wrapper_provide_liquidity* incorrectly rejects delta_liquidity == min_lp_out

Both wrapper liquidity deposit flows enforce `min_lp_out` using a strict greater-than comparison:

```
/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs */
117 | assert!(
118 |     deposit_liquidity_return_data.delta_liquidity > min_lp_out,
119 |     "delta liquidity < min_lp_out"
120 | );

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs */
294 | assert!(
295 |     deposit_liquidity_result.delta_liquidity > min_lp_out,
296 |     "delta liquidity < min_lp_out"
297 | );
```

This logic rejects cases where the minted liquidity exactly equals `min_lp_out`, even though such a value should be considered acceptable.

This behavior is unintuitive from a UX perspective and contradicts the intent implied by the error message and parameter naming: `min_lp_out` should represent the minimum acceptable output, not a strictly lower bound that must be exceeded.

2. Inconsistent treasury_fee_bps bounds between initialization and admin updates

There is an inconsistency in how `treasury_fee_bps` is validated during market initialization versus subsequent admin updates.

During `MarketThreeInit`, the treasury fee is required to be strictly less than `100%`:

```
/* solana/programs/exponent_clmm/src/state/market_three/market_three.rs */
349 | // Make sure treasury fee bps is less than 100%
350 | assert!(treasury_fee_bps < 10000, "Treasury fee BPS is too high");
```

However, during `ModifyMarketSetting`, the treasury fee is allowed to be equal to `10,000 bps`:

```
/* solana/programs/exponent_clmm/src/instructions/market_three/admin/modify_market_setting.rs */
057 | MarketAdminAction::ChangeTreasuryTradeSyBpsFee(new_treasury_trade_bps_fee) => {
058 |     assert!(
059 |         new_treasury_trade_bps_fee <= 10000,
060 |         "Treasury trade SY BPS fee must be less than or equal to 10000"
061 |     );
062 |
063 |     market.configuration_options.treasury_fee_bps = new_treasury_trade_bps_fee;
064 | }
```

Resolution

This issue has been fixed by [d4a0143](#).

EXPONENT_CLMM

[I-12] Off-by-one farm index in AddFarmEvent

Identified in commit [818a6b8](#).

In the `add_farm` instruction, the `event.farm_index` field is set incorrectly due to an off-by-one error.

The `AddFarmEvent` is constructed after a new farm emission entry is pushed to `lp_farm.farm_emissions`. As a result, using `ctx.accounts.market.lp_farm.farm_emissions.len()` returns the post-push length, causing the emitted `farm_index` to be one greater than the actual index of the newly added farm.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/admin/add_farm.rs */
127 | let event = AddFarmEvent {
128 |     market_address: ctx.accounts.market.key(),
129 |     admin_address: ctx.accounts.signer.key(),
130 |     farm_index: ctx.accounts.market.lp_farm.farm_emissions.len() as u16,
131 |     emission_mint: farm_token_mint.key(),
132 |     token_rate,
133 |     expiry_timestamp: until_timestamp,
134 |     amount_deposited: required_amount,
135 | };
```

Consider adjusting the emitted index to reflect the actual position of the newly added farm by subtracting one from the length after the push:

```
farm_index: (ctx.accounts.market.lp_farm.farm_emissions.len() - 1) as u16,
```

Resolution

This issue has been fixed by [d29d6f1](#).

EXPONENT_CLMM

[I-13] Tick node exhaustion DoS risk

Identified in commit [818a6b8](#).

In the CLMM protocol, each pool has a dedicated ticks account that stores the pool's tick state. The ticks account has a maximum capacity of `MAX_TICK_NODES` (`100`). This means each CLMM pool can record at most `100` distinct tick nodes.

```
/* solana/programs/exponent_clmm/src/state/ticks.rs */
023 | pub struct Ticks {
024 |     /// Ticks RedBlackTree (fixed capacity)
025 |     /// Key is apy in base_points
026 |     pub ticks_tree: RedBlackTree<u32, Tick, MAX_TICK_NODES>,

028 | #[account(
029 |     mut,
030 |     has_one = token_pt_escrow,
031 |     has_one = token_sy_escrow,
032 |     has_one = sy_program,
033 |     has_one = address_lookup_table,
034 |     has_one = ticks,
035 | )]
036 | pub market: Box<Account<'info, MarketThree>>,
```

Since each tick node represents a price, users can initialize new tick nodes when opening positions. A malicious user could repeatedly add and then remove liquidity to populate all `100` tick nodes with unreasonable or extreme prices. Once the tick nodes are exhausted, legitimate users will no longer be able to initialize new reasonable price ticks, effectively causing a DoS scenario.

Consider adding validation in the open position process to ensure user-specified tick prices fall within a reasonable range.

Resolution

This issue has been fixed by [a9a855e](#).

EXPONENT_CLMM

[I-14] Overly restrictive sy_to_borrow check in buy_yt

Identified in commit [818a6b8](#).

In the `buy_yt` instruction, `sy_to_strip` represents the total amount of `SY` to be stripped, while `sy_to_borrow` represents the amount of `SY` borrowed from the protocol. There is a check enforcing: `sy_to_borrow < sy_to_strip`, which requires the user to provide some of their own `SY`.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs */
192 | // calculate how much SY must be stripped to get the target YT
193 | // Add 1 to counter-act the flooring function when converting from PY to SY
194 | let sy_to_strip = py_to_sy(sy_exchange_rate, yt_out) + 1;
195 | assert!(
196 |     sy_to_strip > max_sy_in,
197 |     "sy_to_strip must be greater than max_sy_in"
198 | );
199 |
200 | let sy_to_borrow = sy_to_strip - max_sy_in;
201 |
202 | // Check borrow amount is less than target strip amount
203 | assert!(
204 |     sy_to_borrow < sy_to_strip,
205 |     "sy_to_borrow must be less than sy_to_strip"
206 | );
```

However, this restriction can be unnecessarily restrictive. In the `buy_yt` flow, the user receives `PT` from stripping `SY` and then swaps that `PT` into `SY` via the CLMM. Depending on the liquidity distribution and current spot price in the CLMM, the amount of `SY` received from the swap can be greater than `sy_to_strip`. When this occurs, the user can legitimately complete an arbitrage and obtain "free" `YT`.

Requiring a positive user `SY` contribution is therefore not a correctness/safety condition; it is a policy constraint that blocks a valid and potentially desirable arbitrage path. If the protocol's intended behavior is to allow market forces to close mispricings between stripping and the CLMM, then forbidding `max_sy_in == 0` is unnecessarily restrictive.

Consider making the following changes:

1. Change the check from `sy_to_borrow < sy_to_strip` to `sy_to_borrow <= sy_to_strip`.
2. In the `wrapper_buy_yt` instruction, the `sim_buy_yt` function should handle cases where the obtained `sy_amount` exceeds `target_sy`, so users can successfully complete `buy_yt` in these scenarios.

```

/* solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_buy_yt.rs */
104 | fn sim_buy_yt(
105 |     yt_out: u64,
106 |     sy_exchange_rate: Number,
107 |     mf: &mut MarketThree,
108 |     ticks: &mut Ticks,
109 | ) -> u64 {
110 |     // calculate how much SY must be stripped to get the target YT
111 |     // Add 1 to counter-act the flooring function when converting from PY to SY
112 |     let target_sy = py_to_sy(sy_exchange_rate, yt_out) + 1;
113 |     let modified_ticks = &mut ModifiedTicks::default();
114 |     // Calculate required SY amount based on market price
115 |     let sy_amount = mf
116 |         .swap(
117 |             ticks,
118 |             SwapArgs {
119 |                 direction: SwapDirection::PtToSy,
120 |                 amount_in: yt_out,
121 |                 price_spot_limit: None,
122 |                 sy_exchange_rate: sy_exchange_rate.to_f64().unwrap(),
123 |                 is_current_flash_swap: false,
124 |                 modified_ticks,
125 |             },
126 |             false,
127 |         )
128 |         .unwrap()
129 |         .amount_out;
130 |
131 |     let sy_spend = target_sy - sy_amount;
132 |     // convert sy_spend to asset
133 |     (sy_exchange_rate * sy_spend.into()).ceil_u64()
134 | }

```

Resolution

This issue has been fixed by [0b35728](#).

EXPONENT_CLMM

[I-15] Unused emission_index instruction parameter

Identified in commit [818a6b8](#).

The `MarketAccrueEmission` instruction declares an `emission_index: u16` argument. But this value is never referenced in any account constraints or within the instruction handler itself.

```
/* solana/programs/exponent_clmm/src/instructions/market_three/market_accrue_emission.rs */  
009 | #[instruction(emission_index: u16)]  
010 | pub struct MarketAccrueEmission<'info> {
```

If the `emission_index` is not required, consider removing it from the instruction definition entirely by updating the instruction attribute accordingly.

Resolution

This issue has been fixed by [32a645b](#).

EXPONENT_CLMM

[I-16] Issues introduced by the new add_liquidity feature

Identified in commit [ce641d7](#).

The `clmm` protocol introduced a new `add_liquidity` feature during the fix process; however, this instruction currently has several issues.

1. Unsafe u64 to i64 casting

`liquidity_needs.liquidity_target`, `liquidity_reduction`, and `add_liquidity_result.delta_l` are all unsafely cast to `i64` (`delta_l_i64`) using the `as` keyword. If either value exceeds `i64::MAX`, this cast will overflow and produce a negative value.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
110 | let delta_l_i64 = liquidity_needs.liquidity_target as i64;
256 | let delta_l_i64 = liquidity_needs.liquidity_target as i64;
657 | let delta_l = liquidity_reduction as i64;

240 | ctx.accounts
241 |     .market
242 |     .liquidity_net_balance_limits
243 |     .verify_limits(
244 |         Clock::get()?.unix_timestamp as u32,
245 |         liquidity_balance,
246 |         add_liquidity_result.delta_l as i64,
247 |     )?;
```

It's recommended to use type-safe conversions and appropriate bounds checks.

2. Unnecessary minimum liquidity enforcement

The `add_liquidity` instruction allows users to increase the liquidity of an existing LP position. However, it still enforces the `min_l_to_activate` minimum liquidity check inside the `tick_add_liquidity_net_gross` function. For an already opened position, this check is unnecessary and may incorrectly reject valid attempts to increase liquidity.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
257 | ticks.tick_add_liquidity_net_gross(
258 |     lower_tick_idx,
259 |     delta_l_i64,
260 |     true,
261 |     self.configuration_options.min_lp_tick_amount,
262 | )?;
263 | ticks.tick_add_liquidity_net_gross(
264 |     upper_tick_idx,
265 |     delta_l_i64,
```

```

266 |     false,
267 |     self.configuration_options.min_lp_tick_amount,
268 | )?;

261 | pub fn tick_add_liquidity_net_gross(
262 |     &mut self,
263 |     idx: u32,
264 |     delta_l_i64: i64,
265 |     lower: bool,
266 |     min_l_to_activate: u64,
267 | ) -> Result<()> {
268 |     let node = &mut self.ticks_tree.get_node_mut(idx).value;
269 |     if lower {
270 |         node.liquidity_net = node.liquidity_net.checked_add(delta_l_i64).unwrap();
271 |     } else {
272 |         node.liquidity_net = node.liquidity_net.checked_sub(delta_l_i64).unwrap();
273 |     }
275 |     node.liquidity_gross = node
276 |         .liquidity_gross
277 |         .checked_add(delta_l_i64.abs().try_into().unwrap())
278 |         .unwrap();
280 |     node.liquidity_gross
281 |         .checked_sub(min_l_to_activate)
282 |         .expect("not enough liquidity for tick to be initialized");
283 |
284 |     Ok(())
285 | }

```

It's recommended to remove the `min_l_to_activate` minimum liquidity check from the `add_liquidity` instruction path.

3. Duplicate inside fee update

The `add_liquidity` instruction first updates the position's inside fees via `accrue_rewards_before_add`, but then recalculates the inside fee again at the end of the instruction.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
222 | accrue_rewards_before_add(ticks, position, market_farm_indices)?;
307 | let (inside_sy_fee, inside_pt_fee) =
308 |     ticks.compute_fee_inside_for_range(lower_tick_idx, upper_tick_idx)?;
309 |
310 | position.fee_inside_last_pt = inside_pt_fee;
311 | position.fee_inside_last_sy = inside_sy_fee;

```

It's recommended to remove the duplicate inside the fee update logic to avoid redundancy.

Resolution

This issue has been fixed by [eaec664](#), [cf8f449](#) and [99a8460](#).

EXPONENT_CLMM

[I-17] Division precision loss leads to state inconsistency

Identified in commit [fc070e9](#).

The `reverse_crossing_liquidity_corrections` function is used to reverse liquidity corrections at crossing tick boundaries when removing liquidity.

```

/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
176 | fn reverse_crossing_liquidity_corrections(
177 |     ticks: &mut Ticks,
178 |     crossing_split: &crate::CrossingSplit,
179 |     position_lp_balance: u64,
180 |     liquidity_to_remove: u64,
181 |     is_full_removal: bool,
182 | ) -> Result<()> {
183 |     // Calculate the original correction amount
184 |     let l_reduction = position_lp_balance.checked_sub(crossing_split.lp_balance_crossing).unwrap();
185 |
186 |     // Calculate proportional correction to reverse
187 |     let crossing_correction = if is_full_removal {
188 |         l_reduction
189 |     } else {
190 |         // Proportional correction based on removal ratio
191 |         let ratio = liquidity_to_remove as u128 * l_reduction as u128 / position_lp_balance as u128;
192 |         ratio as u64
193 |     };
194 |
195 |     if crossing_correction > 0 {
196 |         let correction_i128 = i128::try_from(crossing_correction).unwrap();
197 |
198 |         // Reverse the corrections:
199 |         // At cross_left: we originally subtracted, now add back
200 |         ticks.tick_add_liquidity_net_gross(
201 |             crossing_split.cross_left_idx,
202 |             correction_i128,
203 |             false,
204 |             0,
205 |         )?;
206 |
207 |         // At cross_right: we originally added, now subtract
208 |         ticks.tick_sub_liquidity_net_gross(
209 |             crossing_split.cross_right_idx,
210 |             correction_i128,
211 |             true,
212 |         )?;
213 |
214 |         // Increase prefix_sum back (we decreased it during add)
215 |         ticks.current_prefix_sum = ticks
216 |             .current_prefix_sum
217 |             .checked_add(crossing_correction)
218 |             .unwrap();
219 |     }
220 |
221 |     Ok(())

```

```
222 | }
```

However, when the removal amount(`liquidity_to_remove`) is small relative to the position balance, integer division truncation causes the correction ratio to become zero, resulting in state inconsistency between tick-level liquidity corrections and the position's actual `lp_balance`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs */
030 | pub fn remove_liquidity(
031 |     &mut self,
032 |     ticks: &mut Ticks,
033 |     position: &mut LpPosition,
034 |     args: RemoveLiquidityArgs,
035 | ) -> Result<RemoveLiquidityOutcome> {
103 |     // == Handle crossing split liquidity corrections ==
104 |     if position.crossing_split.is_active() {
105 |         reverse_crossing_liquidity_corrections(
106 |             ticks,
107 |             &position.crossing_split,
108 |             position.lp_balance,
109 |             liquidity_to_remove_u64,
110 |             is_full_removal,
111 |         )?;
112 |     }
152 |     // Finalize the position's L
153 |     position.lp_balance = position
154 |         .lp_balance
155 |         .checked_sub(liquidity_to_remove_u64)
156 |         .unwrap();
```

When `liquidity_to_remove` is very small, the proportional correction can floor to zero, so the crossing correction is not reversed while `position.lp_balance` is reduced. This leaves a small residual liquidity adjustment in tick state, which can accumulate over repeated small removals and cause the recorded liquidity corrections to drift from the true position balance.

The issue is limited to rounding drift and consistency errors; it does not directly enable fund theft or a deterministic profit path, but it will cause minor deviations in swap results.

Consider accumulating the division remainder or enforcing a minimum correction when `liquidity_to_remove > 0` and `l_reduction > 0` to avoid silent drift. Please change the proportional correction logic to preserve residuals across calls.

Resolution

The team acknowledged this finding and clarified that the impact is indeed minimal, approximately 1-2 units of liquidity drift per transaction. It is not economically viable to exploit, and it does not pose a

security risk from an attack standpoint. However, addressing it would require additional logic and state changes, so they would prefer to keep the current implementation for now.

EXPONENT_CLMM

[I-18] Exact input dust is reported as consumed

Identified in commit 483ae0c.

The `guarded_input_budget_f64()` may deliberately reduce the usable input budget by `1` raw unit when the `f64` roundtrip is too close to the user budget boundary. This is an internal safety guard to prevent segment-level rounding from overspending the caller's declared exact-in amount. That guarded budget is then used to compute the segment input that is actually reflected in swap pricing and principal updates.

However, `finalize_exact_in_amount_consumed()` later discards that distinction for remainders below `SWAP_DUST_LAMPORT_THRESHOLD = 2` and reports the full user input as consumed. For example, when the user transfers in `100` and has `1` remaining, even though the actual consumption is `99`, it will still be recorded as `100`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
009 | const SWAP_DUST_LAMPORT_THRESHOLD: u64 = 2;
105 | fn finalize_exact_in_amount_consumed(args_amount_in: u64, amount_in_left: u64) -> u64 {
106 |     if amount_in_left < SWAP_DUST_LAMPORT_THRESHOLD {
107 |         args_amount_in
108 |     } else {
109 |         args_amount_in.checked_sub(amount_in_left).unwrap_or(0)
110 |     }
111 | }
```

This creates a gap between:

- the amount that actually participates in pricing and interval principal accounting, and
- the amount later reported as `amount_in_consumed` and treated as economically consumed.

Concretely, if internal swap execution consumes `99` and leaves `amount_in_left = 1`, `finalize_exact_in_amount_consumed()` returns `100` instead of `99`. That extra `1` unit did not participate in pricing, but it is still surfaced downstream as consumed input. The `amount_in_consumed` is then reused for accounting, token transfer, and event emission:

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs */
631 | pub fn swap<'a>(<
636 | ) -> Result<SwapOutcome<'a>> {
1019 |     Ok(SwapOutcome {
1020 |         amount_in_consumed: finalize_exact_in_amount_consumed(
1021 |             args.amount_in,
1022 |             amount_in_left,
1023 |         ),
1028 |     })
1029 | }
```

```

/* solana/programs/exponent_clmm/src/instructions/market_three/trade_pt.rs */
195 | #[access_control(ctx.accounts.validate(swap_direction))]
196 | pub fn handler<'info>(<
202 | ) -> Result<TradePtEvent> {
290 |     // Transfer SY between trader & market & sy program
291 |     transfer_sy(
293 |         trade_result.amount_in_consumed,
304 |     )?;

```

As a result, the trader can be charged one extra raw unit of the input token even though that unit was not used in swap pricing or reflected in interval principal updates.

The core issue is that the economic destination of the dust is undefined, and it will be accumulated. In the current implementation, the residual unit is:

- not returned to the user,
- not booked as protocol fee,
- not added to the interval principal,
- and not tracked in any separate dust reserve.

This could leave pool-level balances and tick-level principal accounting potentially out of sync, while charging the user for input that did not affect execution.

It is recommended to explicitly define the destination of the residual dust that did not participate in swap pricing. If the protocol intentionally preserves exact-in semantics by charging the full input amount, the extra dust should be routed into a clearly owned balance, such as protocol fees or another well-defined market bucket, and the accounting should reflect that destination consistently. Otherwise, the dust could be refunded to the user.

Resolution

The team acknowledged this finding and clarified that this is the intended behavior: The plan is to leave 1-2 lamports that are accounted for in transfers but not in the trade, so they are not sent to the user and do not create dust.

EXPONENT_CLMM

[I-19] Inconsistent comments in add_liquidity

Identified in commit [a8ede4d](#).

The value of `GAP_TOKEN_REMINDER_DIVISOR` is inconsistent with its comment. `GAP_TOKEN_REMINDER_DIVISOR` should be changed to `1_000_000`.

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
013 | /// 0.0001 % expressed as a divisor (1 / 1_000_000).
014 | pub(crate) const GAP_TOKEN_REMINDER_DIVISOR: u64 = 1_000_00;
```

This difference relaxes the acceptable unused token remainder for the wrapper by 10 times; when making large deposits, users may transfer more PT/SY dust to the protocol without actually using it to mint LP, causing the quote/fund semantics to deviate from expectations. The relative proportion of single dust is still small, about 0.001%

The comment for `compute_gap_token_reminder` says `minimum 1`, but the actual code is `.max(20000)`

```
/* solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs */
016 | /// Compute the dust tolerance as 0.0001 % of the deposit amount (minimum 1).
017 | #[inline]
018 | pub(crate) fn compute_gap_token_reminder(deposit_amount: u64) -> u64 {
019 |     (deposit_amount / GAP_TOKEN_REMINDER_DIVISOR).max(20000)
020 | }
```

Resolution

This issue has been fixed by [5a90d8b](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Exponent Labs Ltd. (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

